Hourai's Template

哈尔滨工业大学 蓬莱人形

2024 年 11 月 7 日

目录

1 动态规划 **2**

2 数据结构 **13**

3 树论 **46**

4 图论 **61**

5 网络流 **76**

6 数学 **91**

7 多项式 **135**

8 字符串 **145**

9 计算几何 **160**

10 其他 **178**

1 动态规划

1.1 多重背包

1.1.1 用法

n 个物品，m 容量背包，第 i 个物品重量为 w_i 价值为 v_i 共有 c_i 个，计算不超过容量的情况下最多拿多少价值的物品。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

int qread();
const int MAXN = 4e4 + 3;

int F[MAXN];

int main(){
    int n, m;
    cin >> n >> m;
    for(int i = 1;i ≤ n;++ i){
        int w, v, c;
        cin >> w >> v >> c;
        // w: value, v: volume, c: count
        for(int j = 0;j < v;++ j){
            deque <tuple<int, int> > Q;
            for(int k = 0;j + k * v ≤ m;++ k){
                int x = j + k * v;
                int f = F[x] - (x / v) * w;
                while(!Q.empty() && get<0>(Q.back ()) ≤ f)
                    Q.pop_back ();
                Q.push_back({f, x});
                while(!Q.empty() && get<1>(Q.front()) <  x - c * v)
                    Q.pop_front();
                F[x] = get<0>(Q.front()) + (x / v) * w;
            }
        }
    }
    cout << F[m] << endl;
    return 0;
```

```
}
```

1.2 树形背包

```
#include<bits/stdc++.h>
using namespace std;
typedef long long i64;

const int MAXN = 2e3 + 3;

vector<int> E[MAXN];
int W[MAXN];
int F[MAXN][MAXN], S[MAXN];

void dfs(int u, int f){
    F[u][1] = W[u];
    S[u]    = 1;
    for(auto &v : E[u]) if(v ≠ f){
        dfs(v, u);
        for(int i = S[u];i ≥ 1;-- i)
            for(int j = S[v];j ≥ 1;-- j)
                F[u][i + j] = max(F[u][i + j], F[u][i] + F[v][j]);
        S[u] += S[v];
    }
}

int main(){
    int n, m;
    cin >> n >> m;
    for(int i = 1;i ≤ n;++ i){
        int f;
        cin >> f >> W[i];
        E[f].push_back(i);
    }
    dfs(0, 0);
    cout << F[0][m + 1] << endl;
    return 0;
}
```

1.3 动态动态规划 1

1.3.1 例题

给定一棵 n 个点的树，点有点权，求最大独立集。m 次修改，每次把 x 的权值修改成 y。

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i ≤ END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i ≥ END##i;-- i)
using namespace std;
typedef long long i64;
const int INF = 1e9;
const int MAXN = 1e5 + 3;

int W[MAXN];

struct Mat{ int M[2][2]; };
struct Vec{ int V[2];    };
Mat operator *(const Mat &a, const Mat &b){
    Mat c;
    c.M[0][0] = max(a.M[0][0] + b.M[0][0], a.M[0][1] + b.M[1][0]);
    c.M[0][1] = max(a.M[0][0] + b.M[0][1], a.M[0][1] + b.M[1][1]);
    c.M[1][0] = max(a.M[1][0] + b.M[0][0], a.M[1][1] + b.M[1][0]);
    c.M[1][1] = max(a.M[1][0] + b.M[0][1], a.M[1][1] + b.M[1][1]);
    return c;
}
Vec operator *(const Mat &a, const Vec &v){
    Vec r;
    r.V[0] = max(a.M[0][0] + v.V[0], a.M[0][1] + v.V[1]);
    r.V[1] = max(a.M[1][0] + v.V[0], a.M[1][1] + v.V[1]);
    return r;
}

namespace Gra{
    vector<int> E[MAXN];
    int G[MAXN], S[MAXN], D[MAXN], T[MAXN], F[MAXN];
    int X[MAXN], Y[MAXN];
    int H[MAXN][2];
    int K[MAXN][2];
    struct Mat M[MAXN];

    void dfs1(int u, int f){
        S[u] = 1;
        F[u] = f;
```

```
        for(auto &v : E[u]) if(v ≠ f){
            dfs1(v, u);
            S[u] += S[v];
            if(S[v] > S[G[u]]) G[u] = v;
        }
    }
    int o;
    void dfs2(int u, int f){
        if(u == G[f])
            X[u] = X[f];
        else
            X[u] = u;
        H[u][0] = H[u][1] = 0;
        K[u][0] = K[u][1] = 0;
        const int &g = G[u];
        D[u] = ++ o;
        T[o] = u;
        if(g){
            dfs2(g, u);
            Y[u] = Y[g];
            K[u][0] += max(K[g][0], K[g][1]);
            K[u][1] += K[g][0];
        } else {
            Y[u] = u;
        }
        for(auto &v : E[u]) if(v ≠ f && v ≠ g){
            dfs2(v, u);
            H[u][0] += max(K[v][0], K[v][1]);
            H[u][1] += K[v][0];
        }
        M[u].M[0][0] = H[u][0];
        M[u].M[0][1] = H[u][0];
        M[u].M[1][0] = H[u][1] + W[u];
        M[u].M[1][1] = -INF;
        K[u][0] += H[u][0];
        K[u][1] += H[u][1] + W[u];
    }
}

namespace Seg{
    const int SIZ = 4e5 + 3;
    struct Mat M[SIZ];
```

```
    #define lc(t) (t << 1)
    #define rc(t) (t << 1 | 1)
    void pushup(int t, int a, int b){
        M[t] = M[lc(t)] * M[rc(t)];
    }
    void build(int t, int a, int b){
        if(a == b){
            M[t] = Gra :: M[Gra :: T[a]];
        } else {
            int c = a + b >> 1;
            build(lc(t), a, c);
            build(rc(t), c + 1, b);
            pushup(t, a, b);
        }
    }
    void modify(int t, int a, int b, int p, const Mat &w){
        if(a == b){
            M[t] = w;
        } else {
            int c = a + b >> 1;
            if(p <= c) modify(lc(t), a, c, p, w);
                else   modify(rc(t), c + 1, b, p, w);
            pushup(t, a, b);
        }
    }
    Mat query(int t, int a, int b, int l, int r){
        if(l <= a && b <= r){
            return M[t];
        } else {
            int c = a + b >> 1;
            if(r <= c) return query(lc(t), a, c    , l, r); else
            if(l >  c) return query(rc(t), c + 1, b, l, r); else
                return query(lc(t), a, c    , l, r) *
                       query(rc(t), c + 1, b, l, r);
        }
    }
}
int qread();
int main(){
    int n = qread(), m = qread();
    up(1, n, i)
        W[i] = qread();
```

```
    up(2, n, i){
        int u = qread(), v = qread();
        Gra :: E[u].push_back(v);
        Gra :: E[v].push_back(u);
    }
    Gra :: dfs1(1, 0);
    Gra :: dfs2(1, 0);
    Seg :: build(1, 1, n);
    Vec v0;
    v0.V[0] = v0.V[1] = 0;
    up(1, m, i){
        using namespace Gra;
        int x = qread(), y = qread();
        W[x] = y;

        int u = x;
        while(u != 0){
            const int &v = X[u];
            const int &f = F[v];
            M[u].M[0][0] = H[u][0];
            M[u].M[0][1] = H[u][0];
            M[u].M[1][0] = H[u][1] + W[u];
            M[u].M[1][1] = -INF;
            const Vec p = Seg :: query(1, 1, n, D[v], D[Y[u]]) * v0;
            Seg :: modify(1, 1, n, D[u], M[u]);
            const Vec q = Seg :: query(1, 1, n, D[v], D[Y[u]]) * v0;
            if(f != 0){
                H[f][0] = H[f][0] - max(p.V[0], p.V[1]) + max(q.V[0], q
                    .V[1]);
                H[f][1] = H[f][1] - p.V[0] + q.V[0];
            }
            u = f;
        }
        Vec v1 = Seg :: query(1, 1, n, D[1], D[Y[1]]) * v0;
        printf("%d\n", max(v1.V[0], v1.V[1]));
    }
    return 0;
}
```

1.4 插头 dp

1.4.1 例题

给出 $n \times m$ 的方格，有些格子不能铺线，其它格子必须铺，形成一个闭合回路。问有多少种铺法?

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i ≤ END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i ≥ END##i;-- i)
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 20 + 3;
const int MAXM = 67108864 + 3;

namespace HashT{
    const int SIZ = 19999997;
    int H[SIZ], V[SIZ], N[SIZ], t;
    bool F[SIZ];
    i64  W[SIZ];
    void add(int u, int v, bool f, i64 w){
        V[++ t] = v, N[t] = H[u], F[t] = f, W[t] = w, H[u] = t;
    }
    i64& find(int u, bool f){
        for(int p = H[u % SIZ];p;p = N[p])
            if(V[p] == u && F[p] == f)
                return W[p];
        add(u % SIZ, u, f, 0);
        return W[t];
    }
}
char S[MAXN][MAXN];
int qread();
int n, m;
vector <pair<pair<int, bool>, i64> > M[2];
int getp(int s, int p){
    return (s >> (2 * p - 2)) & 3;
}
int setw(int s, int p, int w){
    return (s & ~(3 << (2 * p - 2))) | (w << (2 * p - 2));
```

```
}
int findr(int s, int p){
    int c = 0;
    for(int q = p;q ≤ m + 1;++ q){
        if(((s >> (2 * q - 2)) & 3) == 1) ++ c;
        if(((s >> (2 * q - 2)) & 3) == 2) -- c;
        if(c == 0)
            return q;
    }
    return -1;
}
int findl(int s, int p){
    int c = 0;
    for(int q = p;q ≥ 1;-- q){
        if(((s >> (2 * q - 2)) & 3) == 2) ++ c;
        if(((s >> (2 * q - 2)) & 3) == 1) -- c;
        if(c == 0)
            return q;
    }
    return -1;
}
void state(int s){
    return ;
    up(1, m + 1, i){
        switch(getp(s, i)){
            case 0 : putchar('#'); break;
            case 1 : putchar('('); break;
            case 2 : putchar(')'); break;
            case 3 : putchar('E');
        }
    }
    puts("");
}
int main(){
    n = qread(), m = qread();
    up(1, n, i)
        scanf("%s", S[i] + 1);
    int o = 0;
    #define X M[ o]
    #define Y M[!o]
    vector <pair<int, bool> > T;
    X.push_back({{0, 0}, 1});
```

```
    up(1, n, i){
        Y.clear();
        for(auto &u : X){
            auto [s0, c] = u;
            auto [s, f] = s0;
            if(getp(s, m + 1) == 0)
                Y.push_back({{s << 2, f}, c});
        }
        o ^= 1;
        up(1, m, j){
            int x = j, y = j + 1;
            for(auto &u : X){
                auto [s0, c] = u;
                auto [s, f] = s0;
                int a = getp(s, x);
                int b = getp(s, y);
                int t = setw(setw(s, x, 0), y, 0);
                #define update(t, c) HashT :: find(t, f) += c, T.
                    push_back({t, f})
                if(S[i][j] == '.'){     // 经过该格
                    if(a == 1 && b == 1){
                        t = setw(t, findr(s, y), 1),
                        update(t, c);
                    } else
                    if(a == 2 && b == 2){
                        t = setw(t, findl(s, x), 2),
                        update(t, c);
                    } else
                    if(a == 1 && b == 2){
                        if(f == false)  // 还没有闭合回路
                            f = true, update(t, c);
                    } else
                    if(a == 2 && b == 1){
                        update(t, c);
                    } else
                    if(a == 0 && b == 0){
                        t = setw(t, x, 1);
                        t = setw(t, y, 2);
                        update(t, c);
                    } else {    // a == 0 || b == 0
                        int t1 = setw(t, x, a | b);
                        int t2 = setw(t, y, a | b);
```

```
                        update(t1, c);
                        update(t2, c);
                    }
                }
                if(S[i][j] == '*'){ // 不经过该格
                    if(a == 0 && b == 0)
                        update(t, c);
                }
            }
            Y.clear();
            for(auto &u : T){
                auto [s, f] = u;
                if(HashT :: find(s, f) != 0){
                    Y.push_back({{s, f}, HashT :: find(s, f)});
                    HashT :: find(s, f) = 0;
                }
            }
            T.clear(), o ^= 1;
        }
    }
    i64 ans = 0;
    for(auto &u : X){
        auto [s0, c] = u;
        auto [s, f] = s0;
        bool g = true;
        up(1, m + 1, i)
            g &= getp(s, i) == 0;
        f &= g;
        if(f)
            ans = c;
    }
    printf("%lld\n", ans);
    return 0;
}
```

1.5 斜率优化

1.5.1 形式

考虑一个经典的 dp 转移方程如下:

$$f_i = \max_{j<i}\{f(j) + w(j,i)\}$$

我们将式子拆成三个部分：只跟 i 有关或者与 i,j 均不相关的部分 $a(i)$，只跟 j 有关的部分 $b(j)$，跟 i,j 均有关的部分 $c(i,j)$：

$$f_i = a(i) + \max_{j<i}\{b(j) + c(i,j)\}$$

斜率优化可被用来解决这样一个情形：$c(i,j) = ic_j$。此时 $b(j) + c(i,j)$ 可视作关于 j 的一次函数。如果 c_j 随着 j 的增大而单调，那么可用单调栈维护；否则可以考虑 CDQ 分治或者在凸包上二分。在凸包上可以使用二分查询最高/最低点。

1.5.2 例题

玩具装箱。原始转移方程为：

$$f_i = \max_{j<i}\{f_j + (s_i - s_j - L')^2\}$$

其中 $s_i = i + \sum_{j\le i} c_i, L' = L + 1$。将其分类得到：

$$\begin{aligned} f_i &= \max_{j<i}\{f_j + s_i^2 + s_j^2 + L'^2 - 2s_is_j + 2s_jL' - 2s_iL'\} \\ &= (s_i^2 - 2s_iL' + L'^2) + \max_{j<i}\{(f_j + s_j^2 + 2s_jL') - 2s_is_j\} \end{aligned}$$

在原始的玩具装箱中，s_j 单调增加，也就是斜率单调增加。因此可以直接使用单调栈维护凸包。同时 s_i 也单调增加，因此可以用指针维护。

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long   i64;
typedef long double f80;
const int INF = 2147483647;
const int MAXN= 5e4 + 3;
int n, L, p, e, C[MAXN], Q[MAXN];
f80 S[MAXN], F[MAXN];
f80 gtx(int x){ return S[x]; }
f80 gty(int x){ return F[x] + S[x] * S[x]; }
f80 gtw(int x){ return -2.0 * (L - S[x]); }
f80 gtk(int x,int y){ return (gty(y) - gty(x)) / (gtx(y) - gtx(x)); }
int main(){
    cin >> n >> L;
    for(int i = 1;i ≤ n;++ i){
        cin >> C[i];
        S[i] = S[i - 1] + C[i];
    }
    for(int i = 1;i ≤ n;++ i){
        S[i] += i;
    }
    e = p = 1, L ++, Q[p] = 0;
```

```
    for(int i = 1;i ≤ n;++ i){
        while(e < p && gtk(Q[e], Q[e + 1]) < gtw(i))
            ++ e;
        int j = Q[e];
        F[i] = F[j] + pow(S[i] - S[j] - L, 2);
        while(1 < p && gtk(Q[p - 1], Q[p]) > gtk(Q[p], i))
            e -= (e == p), -- p;
        Q[++ p] = i;
    }
    printf("%.0Lf\n", F[n]);
    return 0;
}
```

2 数据结构

2.1 平衡树

2.1.1 无旋 Treap

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

typedef unsigned int       u32;
typedef unsigned long long u64;
mt19937_64 MT(114514);
namespace Treap{
    const int SIZ = 1e6 + 1e5 + 3;
    int F[SIZ], C[SIZ], S[SIZ], W[SIZ], X[SIZ][2], sz;
    u64 H[SIZ];
    int  newnode(int w){
        W[++ sz] = w, C[sz] = S[sz] = 1; H[sz] = MT();
        return sz;
    }
    void pushup(int x){
        S[x] = C[x] + S[X[x][0]] + S[X[x][1]];
    }
    pair<int, int> split(int u, int x){
        if(u == 0)
            return make_pair(0, 0);
```

```
        if(W[u] > x){
            auto [a, b] = split(X[u][0], x);
            X[u][0] = b, pushup(u);
            return make_pair(a, u);
        } else {
            auto [a, b] = split(X[u][1], x);
            X[u][1] = a, pushup(u);
            return make_pair(u, b);
        }
    }
    int merge(int a, int b){
        if(a == 0 || b == 0)
            return a | b;
        if(H[a] < H[b]){
            X[a][1] = merge(X[a][1], b), pushup(a);
            return a;
        } else {
            X[b][0] = merge(a, X[b][0]), pushup(b);
            return b;
        }
    }
    void insert(int &root, int w){
        auto [p, q] = split(root, w    );
        auto [a, b] = split(   p, w - 1);
        if(b != 0){
            ++ S[b], ++ C[b];
        } else b = newnode(w);
        p    = merge(a, b);
        root = merge(p, q);
    }
    void erase(int &root, int w){
        auto [p, q] = split(root, w    );
        auto [a, b] = split(   p, w - 1);
        -- C[b], -- S[b];
        p    = C[b] == 0 ? a : merge(a, b);
        root = merge(p, q);
    }
    int  find_rank(int &root, int w){
        int x = root, o = x, a = 0;
        for(;x;){
            if(w <  W[x])
                o = x, x = X[x][0];
```

```
            else {
                a += S[X[x][0]];
                if(w == W[x]){
                    o = x; break;
                }
                a += C[x];
                o = x, x = X[x][1];
            }
        }
        return a + 1;
    }
    int  find_kth(int &root, int w){
        int x = root, o = x, a = 0;
        for(;x;){
            if(w <= S[X[x][0]])
                o = x, x = X[x][0];
            else {
                w -= S[X[x][0]];
                if(w <= C[x]){
                    o = x; break;
                }
                w -= C[x];
                o = x, x = X[x][1];
            }
        }
        return W[x];
    }
    int  find_pre(int &root, int w){
        return find_kth(root, find_rank(root, w) - 1);
    }
    int  find_suc(int &root, int w){
        return find_kth(root, find_rank(root, w + 1));
    }
}

// ==== TEST ====

int qread();
int main(){
    using namespace Treap;
    int n = qread(), m = qread(), root = 0;
    for(int i = 1;i <= n;++ i){
```

```
        int a = qread(); insert(root, a);
    }
    int last_ans = 0, ans = 0;
    for(int i = 1;i ≤ m;++ i){
        int op = qread(), x = qread() ^ last_ans;
        switch(op){
            case 1 : insert(root, x); break;
            case 2 : erase (root, x); break;
            case 3 : ans ^= (last_ans = find_rank(root, x)); break;
            case 4 : ans ^= (last_ans = find_kth (root, x)); break;
            case 5 : ans ^= (last_ans = find_pre (root, x)); break;
            case 6 : ans ^= (last_ans = find_suc (root, x)); break;
        }
    }
    printf("%d\n", ans);
    return 0;
}
```

2.1.2 Splay

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

namespace Splay{
    const int SIZ = 1e6 + 1e5 + 3;
    int F[SIZ], C[SIZ], S[SIZ], X[SIZ][2], size;
    bool T[SIZ];
    bool is_root(int x){ return   F[x]     == 0;}
    bool is_rson(int x){ return X[F[x]][1] == x;}
    void push_down(int x){
        if(!T[x]) return;
        int lc = X[x][0], rc = X[x][1];
        if(lc) T[lc] ^= 1, swap(X[lc][0], X[lc][1]);
        if(rc) T[rc] ^= 1, swap(X[rc][0], X[rc][1]);
        T[x] = 0;
    }
    void pushup(int x){
        S[x] = C[x] + S[X[x][0]] + S[X[x][1]];
```

```
    }
    void rotate(int x){
        int y = F[x], z = F[y];
        bool f = is_rson(x);
        bool g = is_rson(y);
        int &t = X[x][!f];
        if(z){ X[z][g] = x; }
        if(t){ F[t]    = y; }
        X[y][f] = t, t = y;
        F[y] = x, pushup(y);
        F[x] = z, pushup(x);
    }
    void splay(int &r, int x, int g = 0){
        for(int f;f = F[x], f ≠ g;rotate(x))
            if(F[f] ≠ g) rotate(is_rson(x) = is_rson(f) ? f : x);
        if(is_root(x)) r = x;
    }
    int  get_kth(int &r, int w){
        int x = r, o = x;
        for(;x;){
            push_down(x);
            if(w ≤ S[X[x][0]]) o = x, x = X[x][0]; else {
                w -= S[X[x][0]];
                if(C[x] && w ≤ C[x]){o = x; break;}
                w -= C[x], o = x, x = X[x][1];
            }
        }
        splay(r, o); return o;
    }
    int  build(int l, int r){
        if(l = r){
            C[l] = S[l] = 1; return l;
        }
        int c = l + r >> 1, a = 0, b = 0;
        if(l ≤ c - 1) a = build(l, c - 1), F[a] = c, X[c][0] = a;
        if(c + 1 ≤ r) b = build(c + 1, r), F[b] = c, X[c][1] = b;
        C[c] = 1, pushup(c); return c;
    }
    void output(int n, int &r){
        push_down(r);
        if(X[r][0]) output(n, X[r][0]);
        if(r ≠ 1 && r ≠ n + 2) printf("%d␣", r - 1);
```

```
        if(X[r][1]) output(n, X[r][1]);
    }
}
int qread();
int main(){
    using namespace Splay;
    int n = qread(), m = qread();
    int root = build(1, n + 2);
    for(int i = 1;i ≤ m;++ i){
        int l = qread() + 1, r = qread() + 1;
        int u = get_kth(root, r + 1);
        int v = get_kth(root, l - 1);
        splay(root, v, 0), splay(root, u, v);
        int t = X[u][0];
        T[t] ^= 1, swap(X[t][0], X[t][1]);
    }
    output(n, root);
    return 0;
}
```

2.1.3 Treap

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i ≤ END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i ≥ END##i;-- i)
using namespace std;
typedef long long i64;
const int INF = 2147483647;
typedef unsigned int       u32;
typedef unsigned long long u64;
mt19937_64 MT(114514);
namespace Treap{
    const int SIZ = 1e6 + 1e5 + 3;
    int F[SIZ], C[SIZ], S[SIZ], W[SIZ], X[SIZ][2], sz;
    u64 H[SIZ];
    bool is_root(int x){ return   F[x]     == 0;}
    bool is_rson(int x){ return X[F[x]][1] == x;}
    int  newnode(int w){
        W[++ sz] = w, C[sz] = S[sz] = 1; H[sz] = MT();
        return sz;
    }
    void pushup(int x){
```

```
        S[x] = C[x] + S[X[x][0]] + S[X[x][1]];
    }
    void rotate(int &root, int x){
        int y = F[x], z = F[y];
        bool f = is_rson(x);
        bool g = is_rson(y);
        int &t = X[x][!f];
        if(z){ X[z][g] = x; } else root = x;
        if(t){ F[t]    = y; }
        X[y][f] = t, t = y;
        F[y] = x, pushup(y);
        F[x] = z, pushup(x);
    }
    void insert(int &root, int w){
        if(root == 0) {root = newnode(w); return;}
        int x = root, o = x;
        for(;x;o = x, x = X[x][w > W[x]]){
            ++ S[x]; if(w == W[x]){ ++ C[x], o = x; break;}
        }
        if(W[o] != w){
            if(w < W[o]) X[o][0] = newnode(w), F[sz] = o, o = sz;
            else         X[o][1] = newnode(w), F[sz] = o, o = sz;
        }
        while(!is_root(o) && H[o] < H[F[o]])
            rotate(root, o);
    }
    void erase(int &root, int w){
        int x = root, o = x;
        for(;x;o = x, x = X[x][w > W[x]]){
            -- S[x]; if(w == W[x]){ -- C[x], o = x; break;}
        }
        if(C[o] == 0){
            while(X[o][0] || X[o][1]){
                u64 wl = X[o][0] ? H[X[o][0]] : ULLONG_MAX;
                u64 wr = X[o][1] ? H[X[o][1]] : ULLONG_MAX;
                if(wl < wr){
                    int p = X[o][0]; rotate(root, p);
                } else {
                    int p = X[o][1]; rotate(root, p);
                }
            }
            if(is_root(o)){
```

```
                root = 0;
            } else {
                X[F[o]][is_rson(o)] = 0;
            }
        }
    }
    int  find_rank(int &root, int w){
        int x = root, o = x, a = 0;
        for(;x;){
            if(w <  W[x])
                o = x, x = X[x][0];
            else {
                a += S[X[x][0]];
                if(w == W[x]){
                    o = x; break;
                }
                a += C[x];
                o = x, x = X[x][1];
            }
        }
        return a + 1;
    }
    int  find_kth(int &root, int w){
        int x = root, o = x, a = 0;
        for(;x;){
            if(w ≤ S[X[x][0]])
                o = x, x = X[x][0];
            else {
                w -= S[X[x][0]];
                if(w ≤ C[x]){
                    o = x; break;
                }
                w -= C[x];
                o = x, x = X[x][1];
            }
        }
        return W[x];
    }
    int  find_pre(int &root, int w){
        return find_kth(root, find_rank(root, w) - 1);
    }
    int  find_suc(int &root, int w){
```

```
        return find_kth(root, find_rank(root, w + 1));
    }
}
int qread();
int main(){
    using namespace Treap;
    int n = qread(), m = qread(), root = 0;
    up(1, n, i){
        int a = qread(); insert(root, a);
    }
    int last_ans = 0, ans = 0;
    up(1, m, i){
        int op = qread(), x = qread() ^ last_ans;
        switch(op){
            case 1 : insert(root, x); break;
            case 2 : erase (root, x); break;
            case 3 : ans ^= (last_ans = find_rank(root, x)); break;
            case 4 : ans ^= (last_ans = find_kth (root, x)); break;
            case 5 : ans ^= (last_ans = find_pre (root, x)); break;
            case 6 : ans ^= (last_ans = find_suc (root, x)); break;
        }
    }
    printf("%d\n", ans);
    return 0;
}
```

2.2 珂朵莉树

```
#include<bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;
const int MAXN = 1e6 + 3;

int power(int a, int b, int p){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % p;
        b >>= 1,  a = 1ll * a * a % p;
    }
    return r;
}
```

```cpp

namespace ODT {
    // <pos_type, value_type>
    map <int, long long> M;

    // 分裂为 [1, p) 和 [p, +inf), 返回后者迭代器
    auto split(int p) {
        auto it = prev(M.upper_bound(p));
        return M.insert(
            it,
            make_pair(p, it -> second)
        );
    }

    // 区间赋值
    void assign(int l, int r, int v) {
        auto it = split(l);
        split(r + 1);
        while (it -> first != r + 1) {
            it = M.erase(it);
        }
        M[l] = v;
    }

    // // 执行操作
    // void perform(int l, int r) {
    //     auto it = split(l);
    //     split(r + 1);
    //     while (it -> first != r + 1) {
    //         // Do something...
    //         it = next(it);
    //     }
    // }

    void modify1(int l, int r, int w) {
        auto it = split(l);
        split(r + 1);
        while(it -> first != r + 1) {
            it -> second += w;
            it = next(it);
        }
    }
```

```
    void modify2(int l, int r, int w) {
        assign(l, r, w);
    }
    long long query1(int l, int r, int k) {
        auto it = split(l);
        split(r + 1);

        map <long long, int> T;
        while(it -> first != r + 1) {
            T[it -> second] += next(it) -> first - it -> first;
            it = next(it);
        }
        for(auto &[w, c]: T){
            if(c >= k)
                return w;
            k -= c;
        }
        return -1;
    }
    long long query2(int l, int r, int x, int y) {
        auto it = split(l);
        split(r + 1);

        int ans = 0;
        while(it -> first != r + 1) {
            int c = next(it) -> first - it -> first;
            long long a = it -> second;
            ans = (ans + 1ll * c * power(a % y, x, y)) % y;
            it = next(it);
        }
        return ans;
    }

};

const int MOD = 1e9 + 7;

int read(int &seed){
    int ret = seed;
    seed = (seed * 7ll + 13) % MOD;
    return ret;
}
```

```

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, seed, vmax;
    cin >> n >> m >> seed >> vmax;

    ODT :: M[n + 1] = 0;
    for(int i = 1;i ≤ n;++ i){
        int a = read(seed) % vmax + 1;
        ODT :: M[i] = a;
    }
    for(int i = 1;i ≤ m;++ i){
        int op = read(seed) % 4 + 1;
        int l = read(seed) % n + 1;
        int r = read(seed) % n + 1;
        int x, y;
        if(l > r)
            swap(l, r);

        if(op == 3){
            x = (read(seed) % (r - l + 1)) + 1;
        } else
            x = read(seed) % vmax + 1;

        if(op == 4)
            y = read(seed) % vmax + 1;


        if(op == 1){
            ODT :: modify1(l, r, x);
        } else
        if(op == 2){
            ODT :: modify2(l, r, x);
        } else
        if(op == 3){
            cout << ODT :: query1(l, r, x) << "\n";
        } else
        if(op == 4){
            cout << ODT :: query2(l, r, x, y) << "\n";
        }
```

```
    }
    return 0;
}
```

2.3 可并堆

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

namespace LeftHeap{
    const int SIZ = 1e5 + 3;
    int W[SIZ], D[SIZ];
    int L[SIZ], R[SIZ];
    int F[SIZ], s;

    bool E[SIZ];

    int merge(int u, int v){
        if(u == 0 || v == 0)
            return u | v;
        if(W[u] > W[v] || (W[u] == W[v] && u > v))
            swap(u, v);
        int &lc = L[u];
        int &rc = R[u];
        rc = merge(rc, v);
        if(D[lc] < D[rc])
            swap(lc, rc);
        D[u] = min(D[lc], D[rc]) + 1;
        if(lc != 0) F[lc] = u;
        if(rc != 0) F[rc] = u;
        return u;
    }
    void pop(int &root){
        int root0 = merge(L[root], R[root]);
        F[root0] = root0;
        F[root ] = root0;
        E[root ] = true;
        root = root0;
```

```
    }
    int top(int &root){
        return W[root];
    }
    int getfa(int u){
        return u == F[u] ? u : F[u] = getfa(F[u]);
    }
    int newnode(int w){
        ++ s;
        W[s] = w;
        F[s] = s;
        D[s] = 1;
        return s;
    }
}

// ==== TEST ====

int qread();

const int MAXN = 1e5 + 3;
int A[MAXN], O[MAXN];

int main(){
    int n, m;
    cin >> n >> m;
    for(int i = 1;i <= n;++ i){
        cin >> A[i];
        O[i] = LeftHeap :: newnode(A[i]);
    }
    for(int i = 1;i <= m;++ i){
        int op;
        cin >> op;
        if(op == 1){
            int x, y;
            cin >> x >> y;
            if(LeftHeap :: E[O[x]])
                continue;
            if(LeftHeap :: E[O[y]])
                continue;
            int fx = LeftHeap :: getfa(O[x]);
            int fy = LeftHeap :: getfa(O[y]);
```

```
            if(fx ≠ fy){
                LeftHeap :: merge(fx, fy);
            }
        } else {
            int x;
            cin >> x;
            if(LeftHeap :: E[O[x]]){
                cout << -1 << endl;
                continue;
            }
            int fx = LeftHeap :: getfa(O[x]);
            cout << LeftHeap :: top(fx) << endl;
            LeftHeap :: pop(fx);
        }
    }
    return 0;
}
```

2.4 线性基

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

namespace LB{
    const int SIZ = 60 + 3;
    i64 W[SIZ], h = 60;
    void insert(i64 w){
        for(int i = h;i ≥ 0;-- i){
            if(w & (1ll << i)){
                if(!W[i]){
                    W[i] = w;
                    break;
                } else {
                    w ^= W[i];
                }
            }
        }
    }
```

```
    i64 query(i64 x){
        for(int i = h;i ≥ 0;-- i){
            if(W[i]){
                x = max(x, x ^ W[i]);
            }
        }
        return x;
    }
}

namespace realLB{
    const int SIZ = 500 + 3;
    long double W[SIZ][SIZ];
    int n = 0;
    void init(int n0){
        n = n0;
    }
    bool zero(long double w){
        return fabs(w) < 1e-9;
    }
    bool insert(long double X[]){
        for(int i = 1; i ≤ n;++ i){
            if(!zero(X[i])){
                if(zero(W[i][i])){
                    for(int j = 1;j ≤ n;++ j)
                        W[i][j] = X[j];
                    return true;
                } else {
                    long double t = X[i] / W[i][i];
                    for(int j = 1;j ≤ n;++ j)
                        X[j] -= t * W[i][j];
                }
            }
        }
        return false;
    }
}

// ═══ TEST ═══

int qread();

```

```
const int MAXN = 500 + 3;
long double X[MAXN][MAXN], C[MAXN];
int I[MAXN];
bool cmp(int a, int b){
    return C[a] < C[b];
}
int main(){
    int n, m;
    cin >> n >> m;
    realLB :: init(m);
    for(int i = 1;i ≤ n;++ i){
        for(int j = 1;j ≤ m;++ j){
            cin >> X[i][j];
        }
    }
    for(int i = 1;i ≤ n;++ i){
        cin >> C[i];
        I[i] = i;
    }
    sort(I + 1, I + 1 + n, cmp);
    int ans = 0, cnt = 0;
    for(int i = 1;i ≤ n;++ i){
        int x = I[i];
        if(realLB :: insert(X[x]))
            ans += C[x],
            cnt += 1;
    }
    cout << cnt << "␣" << ans << endl;
    return 0;
}
```

2.5 Link Cut 树

```
#include<bits/stdc++.h>
using namespace std;
typedef long long i64;

namespace LinkCutTree{
    const int SIZ = 1e5 + 3;
    int F[SIZ], C[SIZ], S[SIZ], W[SIZ], A[SIZ], X[SIZ][2], size;
    bool T[SIZ];
    bool is_root(int x){ return X[F[x]][0] ≠ x && X[F[x]][1] ≠ x;}
```

```
    bool is_rson(int x){ return X[F[x]][1] == x;}
    int  new_node(int w){
        ++ size;
        W[size] = w, C[size] = S[size] = 1;
        A[size] = w, F[size] = 0;
        X[size][0] = X[size][1] = 0;
        return size;
    }
    void push_up(int x){
        S[x] = C[x] + S[X[x][0]] + S[X[x][1]];
        A[x] = W[x] ^ A[X[x][0]] ^ A[X[x][1]];
    }
    void push_down(int x){
        if(!T[x]) return;
        int lc = X[x][0], rc = X[x][1];
        if(lc) T[lc] ^= 1, swap(X[lc][0], X[lc][1]);
        if(rc) T[rc] ^= 1, swap(X[rc][0], X[rc][1]);
        T[x] = false;
    }
    void update(int x){
        if(!is_root(x)) update(F[x]); push_down(x);
    }
    void rotate(int x){
        int y = F[x], z = F[y];
        bool f = is_rson(x);
        bool g = is_rson(y);
        if(is_root(y)){
            F[x] = z;
            F[y] = x;
            X[y][ f] = X[x][!f], F[X[x][!f]] = y;
            X[x][!f] = y;
        } else {
            F[x] = z;
            F[y] = x;
            X[z][ g] = x;
            X[y][ f] = X[x][!f], F[X[x][!f]] = y;
            X[x][!f] = y;
        }
        push_up(y), push_up(x);
    }
    void splay(int x){
        update(x);
```

```
        for(int f = F[x];f = F[x], !is_root(x);rotate(x))
            if(!is_root(f)) rotate(is_rson(x) == is_rson(f) ? f : x);
    }
    int  access(int x){
        int p;
        for(p = 0;x;p = x, x = F[x]){
            splay(x), X[x][1] = p, push_up(x);
        }
        return p;
    }
    void make_root(int x){
        x = access(x);
        T[x] ^= 1, swap(X[x][0], X[x][1]);
    }
    int find_root(int x){
        access(x), splay(x), push_down(x);
        while(X[x][0]) x = X[x][0], push_down(x);
        splay(x);
        return x;
    }
    void link(int x, int y){
        make_root(x), splay(x), F[x] = y;
    }
    void cut(int x, int p){
        make_root(x), access(p), splay(p), X[p][0] = F[x] = 0;
    }
    void modify(int x, int w){
        splay(x), W[x] = w, push_up(x);
    }
}
const int MAXN = 1e5 + 3;
map<pair<int, int>, bool> M;
int n, m;
int main(){
    cin >> n >> m;
    for(int i = 1;i <= n;++ i){
        int a; cin >> a;
        LinkCutTree :: new_node(a);
    }
    for(int i = 1;i <= m;++ i){
        int o; cin >> o;
        if(o == 0){
```

```
            int u, v; cin >> u >> v;
            LinkCutTree :: make_root(u);
            int p = LinkCutTree :: access(v);
            printf("%d\n", LinkCutTree :: A[p]);
        } else if(o == 1){
            int u, v; cin >> u >> v;
            int a = LinkCutTree :: find_root(u);
            int b = LinkCutTree :: find_root(v);
            if(a != b){
                LinkCutTree :: link(u, v);
                M[make_pair(min(u, v), max(u, v))] = true;
            }
        } else if(o == 2){
            int u, v; cin >> u >> v;
            if(M.count(make_pair(min(u, v), max(u, v)))){
                M.erase(make_pair(min(u, v), max(u, v)));
                LinkCutTree :: cut(u, v);
            }
        } else {
            int u, w; cin >> u >> w;
            LinkCutTree :: modify(u, w);
        }
    }
    return 0;
}
```

2.6 线段树

2.6.1 李超树

```
#include<bits/stdc++.h>
using namespace std;
typedef long long i64;

struct Line{ int id; double k, b; Line() = default;};
namespace LCSeg{
    const int SIZ = 2e5 + 3;
    struct Line T[SIZ];
    #define lc(t) (t << 1)
    #define rc(t) (t << 1 | 1)
    bool cmp(int p, Line x, Line y){
        double w1 = x.k * p + x.b;
        double w2 = y.k * p + y.b;
```

```
        double d = w1 - w2;
        if(fabs(d) < 1e-8) return x.id > y.id;
        return d < 0;
    }
    void merge(int t, int a, int b, Line x, Line y){
        int c = a + b >> 1;
        if(cmp(c, x, y)) swap(x, y);
        if(cmp(a, y, x)){
            T[t] = x; if(a ≠ b) merge(rc(t), c + 1, b, T[rc(t)], y);
        } else {
            T[t] = x; if(a ≠ b) merge(lc(t), a,     c, T[lc(t)], y);
        }
    }
    void modify(int t, int a, int b, int l, int r, Line x){
        if(l ≤ a && b ≤ r) merge(t, a, b, T[t], x);
        else {
            int c = a + b >> 1;
            if(l ≤ c) modify(lc(t), a,     c, l, r, x);
            if(r >  c) modify(rc(t), c + 1, b, l, r, x);
        }
    }
    void query(int t, int a, int b, int p, Line &x){
        if(cmp(p, x, T[t])) x = T[t];
        if(a ≠ b){
            int c = a + b >> 1;
            if(p ≤ c) query(lc(t), a,     c, p, x);
            if(p >  c) query(rc(t), c + 1, b, p, x);
        }
    }
}
const int MOD1 = 39989;
const int MOD2 = 1e9;
int qread();
int m = 39989, o;
int main(){
    int n = qread(), last_ans = 0;
    for(int i = 1;i ≤ n;++ i){
        int op = qread(); if(op == 0){
            int k = (qread() + last_ans - 1) % MOD1 + 1;
            Line x = {0, 0, 0}; LCSeg :: query(1, 1, m, k, x);
            printf("%d\n", last_ans = x.id);
        } else {
```

```
            int _x1 = (qread() + last_ans - 1) % MOD1 + 1;
            int _y1 = (qread() + last_ans - 1) % MOD2 + 1;
            int _x2 = (qread() + last_ans - 1) % MOD1 + 1;
            int _y2 = (qread() + last_ans - 1) % MOD2 + 1;
            if(_x1 > _x2) swap(_x1, _x2), swap(_y1, _y2);
            double k, b; int d = ++ o;
            if(_x1 == _x2) k = 0, b = max(_y1, _y2);
                else k = 1.0 * (_y2 - _y1) / (_x2 - _x1), b = _y1 - k *
                    _x1;
            Line x = {d, k, b}; LCSeg :: modify(1, 1, m, _x1, _x2, x);
        }
    }
    return 0;
}
```

2.6.2 线段树 3

```
#include<bits/stdc++.h>
using namespace std;
typedef long long i64;

const int INF = 2e9;
const int MAXN= 5e5 + 3;
int A[MAXN];
struct Node{
    i64 sum; int len, max1, max2, max_cnt, his_mx;
    Node():
        sum(0), max1(-INF), max2(-INF), max_cnt(0), his_mx(-INF), len
            (0) {}
    Node(int w):
        sum(w), max1(   w), max2(-INF), max_cnt(1), his_mx(   w), len
            (1) {}
    bool update(int w1, int w2, int h1, int h2){
        his_mx = max({his_mx, max1 + h1});
        max1 += w1, max2 += w2;
        sum += 1ll * w1 * max_cnt + 1ll * w2 * (len - max_cnt);
        return max1 > max2;
    }
};
struct Tag{
    int max_add, max_his_add, umx_add, umx_his_add; bool have;
    void update(int w1, int w2, int h1, int h2){
```

```
        max_his_add = max(max_his_add, max_add + h1);
        umx_his_add = max(umx_his_add, umx_add + h2);
        max_add += w1, umx_add += w2, have = true;
    }
    void clear(){
        max_add = max_his_add = umx_add = umx_his_add = have = 0;
    }
};
struct Node operator +(Node a, Node b){
    Node t;
    t.max1 = max(a.max1, b.max1);
    if(t.max1 ≠ a.max1){
        if(a.max1 > t.max2) t.max2 = a.max1;
    } else{
        if(a.max2 > t.max2) t.max2 = a.max2;
        t.max_cnt += a.max_cnt;
    }
    if(t.max1 ≠ b.max1){
        if(b.max1 > t.max2) t.max2 = b.max1;
    } else{
        if(b.max2 > t.max2) t.max2 = b.max2;
        t.max_cnt += b.max_cnt;
    }
    t.sum = a.sum + b.sum, t.len = a.len + b.len;
    t.his_mx = max(a.his_mx, b.his_mx);
    return t;
}
namespace Seg{
    const int SIZ = 2e6 + 3;
    struct Node W[SIZ]; struct Tag T[SIZ];
    #define lc(t) (t << 1)
    #define rc(t) (t << 1 | 1)
    void push_up(int t, int a, int b){
        W[t] = W[lc(t)] + W[rc(t)];
    }
    void push_down(int t, int a, int b){
        if(a == b) T[t].clear();
        if(T[t].have){
            int c = a + b >> 1, x = lc(t), y = rc(t);
            int w = max(W[x].max1, W[y].max1);
            int w1 = T[t].max_add, w2 = T[t].umx_add, w3 = T[t].
                max_his_add, w4 = T[t].umx_his_add;
```

```
            if(w == W[x].max1)
                W[x].update(w1, w2, w3, w4),
                T[x].update(w1, w2, w3, w4);
            else
                W[x].update(w2, w2, w4, w4),
                T[x].update(w2, w2, w4, w4);
            if(w == W[y].max1)
                W[y].update(w1, w2, w3, w4),
                T[y].update(w1, w2, w3, w4);
            else
                W[y].update(w2, w2, w4, w4),
                T[y].update(w2, w2, w4, w4);
            T[t].clear();
        }
    }
    void build(int t, int a, int b){
        if(a == b){W[t] = Node(A[a]), T[t].clear();} else {
            int c = a + b >> 1; T[t].clear();
            build(lc(t), a,     c);
            build(rc(t), c + 1, b);
            push_up(t, a, b);
        }
    }
    void modiadd(int t, int a, int b, int l, int r, int w){
        if(l ≤ a && b ≤ r){
            T[t].update(w, w, w, w);
            W[t].update(w, w, w, w);
        } else {
            int c = a + b >> 1; push_down(t, a, b);
            if(l ≤ c) modiadd(lc(t), a,     c, l, r, w);
            if(r >  c) modiadd(rc(t), c + 1, b, l, r, w);
            push_up(t, a, b);
        }
    }
    void modimin(int t, int a, int b, int l, int r, int w){
        if(l ≤ a && b ≤ r){
            if(w ≥ W[t].max1) return; else
            if(w >  W[t].max2){
                int k = w - W[t].max1;
                T[t].update(k, 0, k, 0);
                W[t].update(k, 0, k, 0);
            } else {
```

```
                int c = a + b >> 1;
                push_down(t, a, b);
                modimin(lc(t), a,     c, l, r, w);
                modimin(rc(t), c + 1, b, l, r, w);
                push_up(t, a, b);
            }
        } else {
            int c = a + b >> 1; push_down(t, a, b);
            if(l ≤ c) modimin(lc(t), a,     c, l, r, w);
            if(r >  c) modimin(rc(t), c + 1, b, l, r, w);
            push_up(t, a, b);
        }
    }
    Node query(int t, int a, int b, int l, int r){
        if(l ≤ a && b ≤ r) return W[t];
        int c = a + b >> 1; Node ret; push_down(t, a, b);
        if(l ≤ c) ret = ret + query(lc(t), a,     c, l, r);
        if(r >  c) ret = ret + query(rc(t), c + 1, b, l, r);
        return ret;
    }
}
int qread();
int main(){
    int n = qread(), m = qread();
    for(int i = 1;i ≤ n;++ i)
        A[i] = qread();
    Seg :: build(1, 1, n);
    for(int i = 1;i ≤ m;++ i){
        int op = qread();
        if(op == 1){
            int l = qread(), r = qread(), w = qread();
            Seg :: modiadd(1, 1, n, l, r, w);
        } else if(op == 2){
            int l = qread(), r = qread(), w = qread();
            Seg :: modimin(1, 1, n, l, r, w);
        } else if(op == 3){
            int l = qread(), r = qread();
            auto p = Seg :: query(1, 1, n, l, r);
            printf("%lld\n", p.sum);
        } else if(op == 4){
            int l = qread(), r = qread();
            auto p = Seg :: query(1, 1, n, l, r);
```

```
            printf("%d\n", p.max1);
        } else if(op == 5){
            int l = qread(), r = qread();
            auto p = Seg :: query(1, 1, n, l, r);
            printf("%d\n", p.his_mx);
        }
    }
    return 0;
}
```

2.6.3 扫描线

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

int qread(){
    int w = 1, c, ret;
    while((c = getchar()) >  '9' || c <  '0') w = (c == '-' ? -1 : 1);
        ret = c - '0';
    while((c = getchar()) ≥ '0' && c ≤ '9') ret = ret * 10 + c - '0';
    return ret * w;
}
const int MAXN = 1e5 + 3;
int X1[MAXN], Y1[MAXN];
int X2[MAXN], Y2[MAXN];
int n, h, H[MAXN * 2];
namespace Seg{
    #define lc(t) (t << 1)
    #define rc(t) (t << 1 | 1)
    const int SIZ = 8e5 + 3;
    int T[SIZ], S[SIZ], L[SIZ];
    void pushup(int t, int a, int b){
        S[t] = 0;
        if(a ≠ b){
            S[t] = S[lc(t)] + S[rc(t)];
            L[t] = L[lc(t)] + L[rc(t)];
        }
        if(T[t]) S[t] = L[t];
```

```
    }
    void modify(int t, int a, int b, int l, int r, int w){
        if(l ≤ a && b ≤ r){
            T[t] += w, pushup(t, a, b);
        } else {
            int c = a + b >> 1;
            if(l ≤ c) modify(lc(t), a, c, l, r, w);
            if(r >  c) modify(rc(t), c + 1, b, l, r, w);
            pushup(t, a, b);
        }
    }
    void build(int t, int a, int b){
        if(a == b){
            L[t] = H[a] - H[a - 1];
        } else {
            int c = a + b >> 1;
            build(lc(t), a, c);
            build(rc(t), c + 1, b);
            pushup(t, a, b);
        }
    }
    int query(int t){
        return S[t];
    }
}
tuple <int, int, int> P[MAXN];
tuple <int, int, int> Q[MAXN];
int main(){
    n = qread();
    for(int i = 1;i ≤ n;++ i){
        X1[i] = qread(), Y1[i] = qread();
        X2[i] = qread(), Y2[i] = qread();
        if(X1[i] > X2[i]) swap(X1[i], X2[i]);
        if(Y1[i] > Y2[i]) swap(Y1[i], Y2[i]);
        H[++ h] = Y1[i];
        H[++ h] = Y2[i];
        P[i] = make_tuple(X1[i], Y1[i], Y2[i]);
        Q[i] = make_tuple(X2[i], Y1[i], Y2[i]);
    }
    sort(H + 1, H + 1 + h);
    sort(P + 1, P + 1 + n);
    sort(Q + 1, Q + 1 + n);
```

```
    int o = unique(H + 1, H + 1 + h) - H - 1;
    Seg :: build(1, 1, o);
    i64 ans = 0, last = -1;
    int p = 1, q = 1;
    while(p ≤ n || q ≤ n){
        int x = INF;
        if(p ≤ n) x = min(x, get<0>(P[p]));
        if(q ≤ n) x = min(x, get<0>(Q[q]));

        if(last ≠ -1){
            ans += 1ll * Seg :: query(1) * (x - last);
        }
        last = x;

        while(q ≤ n && get<0>(Q[q]) == x){
            auto [x, l, r] = Q[q]; ++ q;
            l = lower_bound(H + 1, H + 1 + o, l) - H + 1;
            r = lower_bound(H + 1, H + 1 + o, r) - H;
            Seg :: modify(1, 1, o, l, r, 1);
        }
        while(p ≤ n && get<0>(P[p]) == x){
            auto [x, l, r] = P[p]; ++ p;
            l = lower_bound(H + 1, H + 1 + o, l) - H + 1;
            r = lower_bound(H + 1, H + 1 + o, r) - H;
            Seg :: modify(1, 1, o, l, r, -1);
        }
    }
    printf("%lld\n", ans);
    return 0;
}
```

2.7 根号数据结构

2.7.1 块状链表

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

namespace BLOCK{
```

```
    const int SIZ = 1e6 + 1e5 + 3;
    const int BSZ = 2000;

    list <vector<int> > block;

    void build(int n, const int A[]){
        for(int l = 0, r = 0;r ≠ n;){
            l = r;
            r = min(l + BSZ / 2, n);
            vector <int> V0(A + l, A + r);

            block.emplace_back(V0);
        }
    }
    int get_kth(int k){
        for(auto it = block.begin();it ≠ block.end();++ it){
            if(it → size() < k)
                k -= it → size();
            else return it → at(k - 1);
        }
        return -1;
    }
    int get_rank(int w){
        int ans = 0;
        for(auto it = block.begin();it ≠ block.end();++ it){
            if(it → back() < w)
                ans += it → size();
            else {
                ans += lower_bound(it → begin(), it → end(), w) - it
                    → begin();
                break;
            }
        }
        return ans + 1;
    }
    // 插入到第 k 个位置
    void insert(int k, int w){
        for(auto it = block.begin();it ≠ block.end();++ it){
            if(it → size() < k)
                k -= it → size();
            else{
                it → insert(it → begin() + k - 1, w);
```

```
                if(it -> size() > BSZ){
                    vector <int> V1(it -> begin(), it -> begin() + BSZ / 2);
                    vector <int> V2(it -> begin() + BSZ / 2, it -> end());

                    *it = V2;
                    block.insert(it, V1);
                }
                return;
            }
        }
    }
    // 删除第 k 个数
    void erase(int k){
        for(auto it = block.begin();it != block.end();++ it){
            if(it -> size() < k)
                k -= it -> size();
            else{
                it -> erase(it -> begin() + k - 1);
                if(it -> empty())
                    block.erase(it);
                return;
            }
        }
    }
}

int qread();

const int MAXN = 1e5 + 3;
int A[MAXN];

// ==== TEST ====

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;

    cin >> n >> m;
```

```
    for(int i = 1;i ≤ n;++ i)
        cin >> A[i];
    sort(A + 1, A + 1 + n);
    A[n + 1] = INT_MAX;

    BLOCK :: build(n + 1, A + 1);

    int last = 0;
    int ans = 0;

    for(int i = 1;i ≤ m;++ i){
        int op;
        cin >> op;
        if(op == 1){
            int x; cin >> x; x ^= last;
            int k = BLOCK :: get_rank(x);
            BLOCK :: insert(k, x);
        } else
        if(op == 2){
            int x; cin >> x; x ^= last;
            int k = BLOCK :: get_rank(x);
            BLOCK :: erase(k);
        } else
        if(op == 3){
            int x; cin >> x; x ^= last;
            int k = BLOCK :: get_rank(x);
            last = k, ans ^= last;
        } else
        if(op == 4){
            int x; cin >> x; x ^= last;
            int k = BLOCK :: get_kth (x);
            last = k, ans ^= last;
        } else
        if(op == 5){
            int x; cin >> x; x ^= last;
            int k = BLOCK :: get_rank(x);
            last = BLOCK :: get_kth (k - 1), ans ^= last;
        } else
        if(op == 6){
            int x; cin >> x; x ^= last;
            int k = BLOCK :: get_rank(x + 1);
            last = BLOCK :: get_kth (k), ans ^= last;
```

```
        }
    }
    cout << ans << endl;
    return 0;
}
```

2.7.2 莫队二次离线

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i <= END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i >= END##i;-- i)
using namespace std;
typedef long long i64;
const int INF = 2147483647;
const int MAXN= 1e5 + 3;
const int MAXM= (1 << 14) + 3;
int n, m, k, maxt = 16383, X[MAXM], C[MAXM], t;
const int BUF_SIZE = 1e6;
char *p1, *p2, BUF[BUF_SIZE];
inline char readc();
inline int qread();
int A[MAXN], bsize; i64 B[MAXN], R[MAXN];
struct Qry1{ int l, r, id; }O[MAXN];
struct Qry2{ int id, l, r; };
struct Qry3{ int id, l, r; };
bool cmp(Qry1 a, Qry1 b){
    return a.l / bsize == b.l / bsize ? a.r < b.r : a.l < b.l;
}
vector <Qry2> P[MAXN];
vector <Qry3> Q[MAXN];
int main(){
    n = qread(), m = qread(), k = qread(), bsize = sqrt(m + 1);
    up(1, n, i) A[i] = qread();
    up(1, m, i){
        int l = qread(), r = qread(); O[i] = {l, r, i};
    }
    sort(O + 1, O + 1 + m, cmp);
    int l = 1, r = 0;
    up(1, m, i){
        int p = O[i].l, q = O[i].r;
        if(r < q){
            P[r    ].push_back({ i, r + 1, q});
```

```
            Q[l - 1].push_back({-i, r + 1, q});
        }
        if(r > q){
            P[q    ].push_back({-i, q + 1, r});
            Q[l - 1].push_back({ i, q + 1, r});
        }
        r = q;
        if(l > p){
            P[p].push_back({-i, p, l - 1});
            Q[r].push_back({ i, p, l - 1});
        }
        if(l < p){
            P[l].push_back({ i, l, p - 1});
            Q[r].push_back({-i, l, p - 1});
        }
        l = p;
    }
    up(0, maxt, i) if(__builtin_popcount(i) == k) X[++ t] = i;
    up(0, n, i){
        up(1, t, j) ++ C[A[i] ^ X[j]];
        for(auto &o : P[i]){
            if(o.id > 0) R[ o.id] += C[A[o.l]];
            else         R[-o.id] -= C[A[o.l]];
            if(o.l < o.r)
                P[i + 1].push_back({o.id, o.l + 1, o.r});
        }
        for(auto &o : Q[i]){
            up(o.l, o.r, j){
                if(o.id > 0) R[ o.id] += C[A[j]];
                else         R[-o.id] -= C[A[j]];
            }
        }
        P[i].clear(), Q[i].clear();
        P[i].shrink_to_fit();
        Q[i].shrink_to_fit();
    }
    i64 ans = 0;
    up(1, m, i){ ans += R[i], B[O[i].id] = ans; }
    up(1, m, i) printf("%lld\n", B[i]);
    return 0;
}
```

3 树论

3.1 点分树

3.1.1 例题

给定 n 个点组成的树，点有点权 v_i。m 个操作，分为两种：

- `0 x k` 查询距离 x 不超过 k 的所有点的点权之和；
- `0 x y` 将点 x 的点权修改为 y。

```
#include<bits/stdc++.h>
#define endl "\n"
using namespace std;

const int MAXN = 1e5 + 3;
vector<int> E[MAXN];

namespace LCA{
    const int SIZ = 1e5 + 3;
    int D[SIZ], F[SIZ];
    int P[SIZ], Q[SIZ], o;
    void dfs(int u, int f){
        P[u] = ++ o;
        Q[o] = u;
        F[u] = f;
        D[u] = D[f] + 1;
        for(auto &v : E[u]) if(v != f){
            dfs(v, u);
        }
    }
    const int MAXH = 18 + 3;
    int h = 18;
    int ST[SIZ][MAXH];
    int cmp(int a, int b){
        return D[a] < D[b] ? a : b;
    }
    int T[SIZ], n;
    void init(int _n){
        n = _n;
        dfs(1, 0);
        for(int i = 1;i <= n;++ i)
            ST[i][0] = Q[i];
        for(int i = 2;i <= n;++ i)
            T[i] = T[i >> 1] + 1;
```

```
        for(int i = 1;i ≤ h;++ i){
            for(int j = 1;j ≤ n;++ j) if(j + (1 << i - 1) ≤ n){
                ST[j][i] = cmp(ST[j][i - 1], ST[j + (1 << i - 1)][i -
                    1]);
            }
        }
    }
    int lca(int a, int b){
        if(a == b)
            return a;
        int l = P[a];
        int r = P[b];
        if(l > r)
            swap(l, r);
        ++ l;
        int d = T[r - l + 1];
        return F[cmp(ST[l][d], ST[r - (1 << d) + 1][d])];
    }
    int dis(int a, int b){
        return D[a] + D[b] - 2 * D[lca(a, b)];
    }
}

namespace BIT{
    void modify(int D[], int n, int p, int w){
        ++ p;
        while(p ≤ n)
            D[p] += w, p += p & -p;
    }
    int query(int D[], int n, int p){
        if(p < 0) return 0;
        p = min(n, p + 1);
        int r = 0;
        while(p >  0)
            r += D[p], p -= p & -p;
        return r;
    }
}

namespace PTree{
    const int SIZ = 1e5 + 3;
    bool V[SIZ];
```

```
    int  S[SIZ], L[SIZ];
    vector<int> EE[MAXN];
    int *D1[MAXN];
    int *D2[MAXN];

    void dfs1(int s, int &g, int u, int f){
        S[u] = 1;
        int maxsize = 0;
        for(auto &v : E[u]) if(v ≠ f && !V[v]){
            dfs1(s, g, v, u);
            if(S[v] > maxsize)
                maxsize = S[v];
            S[u] += S[v];
        }
        maxsize = max(maxsize, s - S[u]);
        if(maxsize ≤ s / 2)
            g = u;
    }

    int n;
    void build(int s, int &g, int u, int f){
        dfs1(s, g, u, f);
        V[g] = true, L[g] = s;
        for(auto &u : E[g]) if(!V[u]){
            int h = 0;
            if(S[u] < S[g]) build(S[u], h, u, 0);
            else            build(s - S[g], h, u, 0);
            EE[g].push_back(h);
            EE[h].push_back(g);
        }
    }
    int F[SIZ];
    void dfs2(int u, int f){
        F[u] = f;
        for(auto &v : EE[u]) if(v ≠ f){
            dfs2(v, u);
        }
    }
    void build(int _n){
        n = _n;
        int s = n, g = 0;
        dfs1(s, g, 1, 0);
```

```
        V[g] = true, L[g] = s;
        for(auto &u : E[g]){
            int h = 0;
            if(S[u] < S[g]) build(S[u], h, u, 0);
            else            build(s - S[g], h, u, 0);
            EE[g].push_back(h);
            EE[h].push_back(g);
        }
        dfs2(g, 0);
        for(int i = 1;i ≤ n;++ i){
            L[i] += 2;
            D1[i] = new int[L[i] + 3];
            D2[i] = new int[L[i] + 3];
            for(int j = 0;j < L[i] + 3;++ j)
                D1[i][j] = D2[i][j] = 0;
        }
    }
    void modify(int x, int w){
        int u = x;
        while(1){
            BIT :: modify(D1[x], L[x], LCA :: dis(u, x), w);
            int y = F[x];
            if(y ≠ 0){
                int e = LCA :: dis(x, y);
                BIT :: modify(D2[x], L[x], LCA :: dis(u, y), w);
                x = y;
            } else break;
        }
    }
    int query(int x, int d){
        int ans = 0, u = x;
        while(1){
            ans += BIT :: query(D1[x], L[x], d - LCA :: dis(u, x));
            int y = F[x];
            if(y ≠ 0){
                int e = LCA :: dis(x, y);
                ans -= BIT :: query(D2[x], L[x], d - LCA :: dis(u, y));
                x = y;
            } else break;
        }
        return ans;
    }
```

```
}

int W[MAXN];

int main(){
    ios :: sync_with_stdio(false);
    int n, m;
    cin >> n >> m;
    for(int i = 1;i ≤ n;++ i){
        cin >> W[i];
    }
    for(int i = 2;i ≤ n;++ i){
        int u, v;
        cin >> u >> v;
        E[u].push_back(v);
        E[v].push_back(u);
    }
    LCA :: init(n);

    PTree :: build(n);

    for(int i = 1;i ≤ n;++ i)
        PTree :: modify(i, W[i]);

    int lastans = 0;
    for(int i = 1;i ≤ m;++ i){
        int op; cin >> op;
        if(op == 0){
            int x, d;
            cin >> x >> d;
            x ^= lastans;
            d ^= lastans;
            cout << (lastans = PTree :: query(x, d)) << endl;
        } else {
            int x, w;
            cin >> x >> w;
            x ^= lastans;
            w ^= lastans;
            PTree :: modify(x, -W[x]    );
            PTree :: modify(x,  W[x] = w);
        }
    }
```

```
    return 0;
}
```

3.2 长链剖分

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN= 5e5 + 3;
const int MAXM=  19 + 3;
vector <int> P[MAXN];
vector <int> Q[MAXN];
vector <int> E[MAXN];
int h = 19;
int L[MAXN], F[MAXN], G[MAXN], D[MAXN], S[MAXM][MAXN];
void dfs1(int u, int f){
    L[u] = 1, S[0][u] = f;
    F[u] = f, D[u] = D[f] + 1;
    for(int i = 1;i <= h;++ i)
        S[i][u] = S[i - 1][S[i - 1][u]];
    for(auto &v : E[u]) if(v != f){
        dfs1(v, u);
        if(L[v] > L[G[u]])
            G[u] = v;
        L[u] = max(L[u], L[v] + 1);
    }
}
int T[MAXN];
void dfs2(int u, int f){
    if(u == G[f]){
        T[u] = T[f];
        P[T[u]].push_back(u);
        Q[T[u]].push_back(F[Q[T[u]].back()]);
    } else {
        T[u] = u;
        P[u].push_back(u);
        Q[u].push_back(u);
    }
```

```
    if(G[u]) dfs2(G[u], u);
    for(auto &v : E[u]) if(v != f && v != G[u])
        dfs2(v, u);
}
typedef unsigned int       u32;
typedef unsigned long long u64;
int n, q; u32 s;
u32 get(u32 x) {
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return s = x;
}
int qread();
int H[MAXN];
int main(){
    scanf("%d%d%u", &n, &q, &s);
    int root = 0; H[0] = -1;
    for(int i = 1;i <= n;++ i){
        int f = qread();
        if(f == 0)
            root = i;
        else {
            E[f].push_back(i);
            E[i].push_back(f);
        }
        H[i] = H[i >> 1] + 1;
    }
    dfs1(root, 0);
    dfs2(root, 0);
    int lastans = 0;
    i64 realans = 0;
    for(int i = 1;i <= q;++ i){
        int x = (get(s) ^ lastans) % n + 1;
        int k = (get(s) ^ lastans) % D[x];

        if(k == 0){
            lastans = x;
        } else {
            int h = H[k];
            k -= 1 << h;
            x = S[h][x];
```

```
            int t = T[x];
            k -= D[x] - D[t];
            if(k > 0){
                x = Q[t][k];
            } else {
                x = P[t][-k];
            }
            lastans = x;
        }
        realans ^= 1ll * i * lastans;

    }
    printf("%lld\n", realans);
    return 0;
}
```

3.3 重链剖分

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN= 1e5 + 3;

int MOD;

int n, m, root;
int A[MAXN];
int qread();
vector <int> E[MAXN];
int S[MAXN], G[MAXN], D[MAXN], F[MAXN];
void dfs1(int u, int f){
    S[u] = 1, G[u] = 0, D[u] = D[f] + 1, F[u] = f;
    for(auto &v : E[u]) if(v != f){
        dfs1(v, u);
        S[u] += S[v];
        if(S[v] > S[G[u]])
            G[u] = v;
    }
```

```
}
int B[MAXN];
int P[MAXN], Q[MAXN], T[MAXN], L[MAXN], R[MAXN], cnt;
void dfs2(int u, int f){
    P[++ cnt] = u, B[cnt] = A[u], Q[u] = cnt;
    L[u] = cnt;
    if(u ≠ G[f]) T[u] = u;
        else      T[u] = T[f];
    if(G[u]) dfs2(G[u], u);
    for(auto &v : E[u]) if(v ≠ f && v ≠ G[u]){
        dfs2(v, u);
    }
    R[u] = cnt;
}

namespace Seg{
    #define lc(t) (t << 1)
    #define rc(t) (t << 1 | 1)
    const int SIZ = 4e5 + 3;
    i64 S[SIZ], T[SIZ];
    void pushup(int t, int a, int b){
        S[t] = (S[lc(t)] + S[rc(t)]) % MOD;
    }
    void pushdown(int t, int a, int b){
        if(T[t]){
            int c = a + b >> 1;
            T[lc(t)] = (T[lc(t)] + T[t]) % MOD;
            T[rc(t)] = (T[rc(t)] + T[t]) % MOD;
            S[lc(t)] = (S[lc(t)] + 1ull * (c - a + 1) * T[t]) % MOD;
            S[rc(t)] = (S[rc(t)] + 1ull * (b - c    ) * T[t]) % MOD;
            T[t] = 0;
        }
    }
    void modify(int t, int a, int b, int l, int r, int w){
        if(l ≤ a && b ≤ r){
            S[t] = (S[t] + 1ll * w * (b - a + 1)) % MOD;
            T[t] = (T[t] + w) % MOD;
        } else {
            int c = a + b >> 1;
            pushdown(t, a, b);
            if(l ≤ c) modify(lc(t), a, c, l, r, w);
            if(r >  c) modify(rc(t), c + 1, b, l, r, w);
```

```
            pushup(t, a, b);
        }
    }
    i64 query(int t, int a, int b, int l, int r){
        if(l ≤ a && b ≤ r)
            return S[t];
        int c = a + b >> 1;
        i64 ans = 0;
        pushdown(t, a, b);
        if(l ≤ c) ans = (ans + query(lc(t), a, c, l, r)) % MOD;
        if(r >  c) ans = (ans + query(rc(t), c + 1, b, l, r)) % MOD;
        return ans;
    }
    void build(int t, int a, int b){
        if(a == b){
            S[t] = B[a] % MOD;
        } else {
            int c = a + b >> 1;
            build(lc(t), a, c);
            build(rc(t), c + 1, b);
            pushup(t, a, b);
        }
    }
}
int main(){
    n = qread(), m = qread(), root = qread(), MOD = qread();
    for(int i = 1;i ≤ n;++ i)
        A[i] = qread();
    for(int i = 2;i ≤ n;++ i){
        int u = qread(), v = qread();
        E[u].push_back(v);
        E[v].push_back(u);
    }
    dfs1(root, 0);
    dfs2(root, 0);
    Seg :: build(1, 1, n);
    for(int i = 1;i ≤ m;++ i){
        int op = qread();
        if(op == 1){
            int u = qread(), v = qread(), k = qread();
            while(T[u] ≠ T[v]){
                if(D[T[u]] < D[T[v]])
```

```
                    swap(u, v);
                Seg :: modify(1, 1, n, Q[T[u]], Q[u], k);
                u = F[T[u]];
            }
            if(D[u] < D[v]) swap(u, v);
            Seg :: modify(1, 1, n, Q[v], Q[u], k);
        } else if(op == 2){
            int u = qread(), v = qread();
            i64 ans = 0;
            while(T[u] != T[v]){
                if(D[T[u]] < D[T[v]])
                    swap(u, v);
                ans = (ans + Seg :: query(1, 1, n, Q[T[u]], Q[u])) %
                    MOD;
                u = F[T[u]];
            }
            if(D[u] < D[v]) swap(u, v);
            ans = (ans + Seg :: query(1, 1, n, Q[v], Q[u])) % MOD;
            printf("%lld\n", ans);
        } else if(op == 3){
            int x = qread(), w = qread();
            Seg :: modify(1, 1, n, L[x], R[x], w);
        } else {
            int x = qread();
            printf("%lld\n", Seg :: query(1, 1, n, L[x], R[x]));
        }
    }
    return 0;
}
```

3.4 树哈希

3.4.1 用法

给定大小为 n 的以 1 为根的树，计算 h_i 表示子树 i 的哈希值，计算有多少个本质不同的值。

```
#include<bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;
const int MAXN = 1e6 + 3;

u64 xor_shift(u64 x){
    x ^= x << 13;
```

```
    x ^= x >> 7;
    x ^= x << 17;
    return x;
}
u64 H[MAXN];

vector <int> E[MAXN];
void dfs(int u, int f){
    H[u] = 1;
    for(auto &v: E[u]) if(v != f){
        dfs(v, u);
        H[u] += H[v];
    }
    H[u] = xor_shift(H[u]); // !important
}

int main(){
    int n;
    cin >> n;
    for(int i = 2;i <= n;++ i){
        int u, v;
        cin >> u >> v;
        E[u].push_back(v);
        E[v].push_back(u);
    }
    dfs(1, 0);

    sort(H + 1, H + 1 + n);
    cout << (unique(H + 1, H + 1 + n) - H - 1) << endl;
    return 0;
}
```

3.5 Prufer 序列

3.5.1 用法

给定大小为 n 的以 1 为根的树，计算 h_i 表示子树 i 的哈希值，计算有多少个本质不同的值。

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 5e6 + 3;
int D[MAXN], F[MAXN], P[MAXN];
vector<int> tree2prufer(int n){
```

```
    vector <int> P(n);
    for(int i = 1, j = 1;i ≤ n - 2;++ i, ++ j){
        while(D[j]) ++ j;
        P[i] = F[j];
        while(i ≤ n - 2 && !--D[P[i]] && P[i] < j)
            P[i + 1] = F[P[i]], i ++;
    }
    return P;
}
vector<int> prufer2tree(int n){
    vector <int> F(n);
    for(int i = 1, j = 1;i ≤ n - 1;++ i, ++ j){
        while(D[j]) ++ j;
        F[j] = P[i];
        while(i ≤ n - 1 && !--D[P[i]] && P[i] < j)
            F[P[i]] = P[i + 1], i ++;
    }
    return F;
}

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector <int> ANS;
    if(m == 1){     // tree → prufer
        for(int i = 1;i ≤ n - 1;++ i){
            cin >> F[i], D[F[i]] ++;
        }
        ANS = tree2prufer(n);
    } else {        // prufer → tree
        for(int i = 1;i ≤ n - 2;++ i){
            cin >> P[i], D[P[i]] ++;
        }
        P[n - 1] = n;
        ANS = prufer2tree(n);
    }
    long long ans = 0, cnt = 0;
    for(int i = 1;i ≤ n - (m == 1 ? 2 : 1);++ i)
```

```
        ans ^= 1ll * ANS[i] * i;
    cout << ans << "\n";
    return 0;
}
```

3.6 虚树

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 5e5 + 3;
vector<pair<int, int> > E[MAXN];

namespace LCA{
    const int SIZ = 5e5 + 3;
    int D[SIZ], H[SIZ], F[SIZ];
    int P[SIZ], Q[SIZ], o;
    void dfs(int u, int f){
        P[u] = ++ o;
        Q[o] = u;
        F[u] = f;
        D[u] = D[f] + 1;
        for(auto &[v, w] : E[u]) if(v != f){
            H[v] = H[u] + w, dfs(v, u);
        }
    }
    const int MAXH = 18 + 3;
    int h = 18;
    int ST[SIZ][MAXH];
    int cmp(int a, int b){
        return D[a] < D[b] ? a : b;
    }
    int T[SIZ], n;
    void init(int _n, int root){
        n = _n;
        dfs(root, 0);
        for(int i = 1;i <= n;++ i)
            ST[i][0] = Q[i];
        for(int i = 2;i <= n;++ i)
            T[i] = T[i >> 1] + 1;
        for(int i = 1;i <= h;++ i){
            for(int j = 1;j <= n;++ j) if(j + (1 << i - 1) <= n){
```

```
                ST[j][i] = cmp(ST[j][i - 1], ST[j + (1 << i - 1)][i - 1]);
            }
        }
    }
    int lca(int a, int b){
        if(a == b)
            return a;
        int l = P[a];
        int r = P[b];
        if(l > r)
            swap(l, r);
        ++ l;
        int d = T[r - l + 1];
        return F[cmp(ST[l][d], ST[r - (1 << d) + 1][d])];
    }
    int dis(int a, int b){
        return H[a] + H[b] - 2 * H[lca(a, b)];
    }
}

bool cmp(int a, int b){
    return LCA :: P[a] < LCA :: P[b];
}

bool I[MAXN];

vector <int> E1[MAXN];
vector <int> V1;

void solve(vector <int> &V){
    using LCA :: lca;
    using LCA :: D;
    stack <int> S;
    sort(V.begin(), V.end(), cmp);
    S.push(1);
    int v, l;
    for(auto &u : V) I[u] = true;
    for(auto &u : V) if(u != 1){
        int f = lca(u, S.top());
        l = -1;
        while(D[v = S.top()] > D[f]){
```

```
            if(l ≠ -1)
                E1[v].push_back(l);
            V1.push_back(l = v), S.pop();
        }
        if(l ≠ -1)
            E1[f].push_back(l);
        if(f ≠ S.top())
            S.push(f);
        S.push(u);
    }
    l = -1;
    while(!S.empty()){
        v = S.top();
        if(l ≠ -1)
            E1[v].push_back(l);
        V1.push_back(l = v), S.pop();
    }

    // dfs(1, 0); // SOLVE HERE !!!

    for(auto &u : V1)
        E1[u].clear(), I[u] = false;
    V1.clear();
}
```

4 图论

4.1 仙人掌

4.1.1 例题

给定一个仙人掌，多组询问 u, v 之间最短路长度。

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i ≤ END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i ≥ END##i;-- i)
using namespace std;
typedef long long i64;
const int INF = 2147483647;
const int MAXN= 2e5 + 3;
const int MAXM= 2e5 + 3;
const int MAXD=  18 + 3;
struct edge{int u, v, w;};
vector <edge> V1[MAXN];
```

```
vector <edge> V2[MAXN];
vector <int> H[MAXN];
int n, D[MAXN], W[MAXN], F[MAXD][MAXN];
int o, X[MAXN], L[MAXN];
bool E[MAXN];
void dfs1(int u, int f){
    D[u] = D[f] + 1, F[0][u] = f;
    for(auto &e : V1[u]) if(e.v ≠ f){
        if(D[e.v] && D[e.v] < D[u]){
            int a = e.u;
            int b = e.v;
            int c = ++ o, t = c + n;
            H[c].push_back(a);
            L[c] = W[a] - W[b] + e.w;
            while(a ≠ b)
                E[a] = true, a = F[0][a], H[c].push_back(a);
            for(auto &x : H[c]){
                int w = min(W[x] - W[b], L[c] - W[x] + W[b]);
                V2[x].push_back(edge{x, t, w});
                V2[t].push_back(edge{t, x, w});
            }
        } else if(!D[e.v]){
            W[e.v] = W[u] + e.w, dfs1(e.v, u);
        }
    }
    for(auto &e : V1[u]) if(D[e.v] > D[u]){
        if(!E[e.v]){
            V2[e.u].push_back({e.u, e.v, e.w});
            V2[e.v].push_back({e.v, e.u, e.w});
        }
    }
}
int d = 18;
void dfs2(int u, int f){
    D[u] = D[f] + 1, F[0][u] = f;
    up(1, d, i) F[i][u] = F[i - 1][F[i - 1][u]];
    for(auto &e : V2[u]) if(e.v ≠ f){
        X[e.v] = X[e.u] + e.w;
        dfs2(e.v, u);
    }
}
int lca(int u, int v){
```

```
    if(D[u] < D[v]) swap(u, v);
    dn(d, 0, i) if(D[F[i][u]] ≥ D[v]) u = F[i][u];
    if(u == v) return u;
    dn(d, 0, i) if(F[i][u] ≠ F[i][v]) u = F[i][u], v = F[i][v];
    return F[0][u];
}
int jump(int u, int v){
    dn(d, 0, i) if(D[F[i][v]] >  D[u]) v = F[i][v];
    return v;
}
int dis(int x, int y){
    int t = lca(x, y);
    if(t > n){
        int u = jump(t, x);
        int v = jump(t, y);
        int w = abs(W[u] - W[v]);
        int l = min(w, L[t - n] - w);
        return X[x] - X[u] + X[y] - X[v] + l;
    } else {
        return X[x] + X[y] - 2 * X[t];
    }
}
int m, q;
int qread();
int main(){
    n = qread(), m = qread(), q = qread();
    up(1, m, i){
        int u = qread(), v = qread(), w = qread();
        V1[u].push_back(edge{u, v, w});
        V1[v].push_back(edge{v, u, w});
    }
    dfs1(1, 0);
    dfs2(1, 0);
    up(1, q, i){
        int u = qread(), v = qread();
        printf("%d\n", dis(u, v));
    }
    return 0;
}
```

4.2 三元环计数

4.2.1 三元环计数

无向图：考虑将所有点按度数从小往大排序，然后将每条边定向，由排在前面的指向排在后面的，得到一个有向图。然后考虑枚举一个点，再枚举一个点，暴力数，具体见代码。结论是，这样定向后，每个点的出度是 $O(\sqrt{m})$ 的。复杂度 $O(m\sqrt{m})$。**有向图**：不难发现，上述方法枚举了三个点，计算有向图三元环也就只需要处理下方向的事，这个由于算法够暴力，随便改改就能做了。

```

// 无向图
ll n, m; cin >> n >> m;
vector<pair<ll, ll>> Edges(m);
vector<vector<ll>> G(n + 2);
vector<ll> deg(n + 2);
for (auto &[i, j] : Edges) cin >> i >> j, ++deg[i], ++deg[j];
for (auto [i, j] : Edges) {
    if (deg[i] > deg[j] || (deg[i] == deg[j] && i > j)) swap(i, j);
    G[i].emplace_back(j);
}
vector<ll> val(n + 2);
ll ans = 0;
for (ll i = 1; i <= n; ++i) {
    for (auto j : G[i]) ++val[j];
    for (auto j : G[i]) for (auto k : G[j]) ans += val[k];
    for (auto j : G[i]) val[j] = 0;
}

// 有向图
ll n, m; cin >> n >> m;
vector<pair<ll, ll>> Edges(m);
vector<vector<pll>> G(n + 2);
vector<ll> deg(n + 2);
for (auto &[i, j] : Edges) cin >> i >> j, ++deg[i], ++deg[j];
for (auto [i, j] : Edges) {
    ll flg = 0;
    if (deg[i] > deg[j] || (deg[i] == deg[j] && i > j)) swap(i, j), flg
        = 1;
    G[i].emplace_back(j, flg);
}
vector<ll> in(n + 2), out(n + 2);
ll ans = 0;
for (ll i = 1; i <= n; ++i) {
    for (auto [j, w] : G[i]) w ? (++in[j]) : (++out[j]);
```

```
    for (auto [j, w1] : G[i]) for (auto [k, w2] : G[j]) {
        if (w1 == w2) ans += w1 ? in[k] : out[k];
    }
    for (auto [j, w] : G[i]) in[j] = out[j] = 0;
}
cout << ans << '\n';
```

4.3 四元环计数

4.3.1 四元环计数

From zpk

- **无向图**：类似，由于定向后出度结论过于强大，可以暴力。讨论了三种情况。
- **有向图**：缺少题目，但应当类似三元环计数有向形式记录定向边和原边的正反关系。因为此法最强的结论是定向后出度 $O(\sqrt{m})$，实际上方法很暴力，应当不难数有向形式的。

```

ll n, m; cin >> n >> m;
vector<pair<ll, ll>> Edges(m);
vector<vector<ll>> G(n + 2), iG(n + 2);
vector<ll> deg(n + 2);
for (auto &[i, j] : Edges) cin >> i >> j, ++deg[i], ++deg[j];
for (auto [i, j] : Edges) {
    if (deg[i] > deg[j] || (deg[i] == deg[j] && i > j)) swap(i, j);
    G[i].emplace_back(j), iG[j].emplace_back(i);
}
ll ans = 0;
vector<ll> v1(n + 2), v2(n + 2);
for (ll i = 1; i <= n; ++i) {
    for (auto j : G[i]) for (auto k : G[j]) ++v1[k];
    for (auto j : iG[i]) for (auto k : G[j]) ans += v1[k], ++v2[k];
    for (auto j : G[i]) for (auto k : G[j]) ans += v1[k] * (v1[k] - 1)
        / 2, v1[k] = 0;
    for (auto j : iG[i]) for (auto k : G[j]) {
        if (deg[k] > deg[i] || (deg[k] == deg[i] && k > i)) ans += v2[k
            ] * (v2[k] - 1) / 2;
        v2[k] = 0;
    }
}
cout << ans << '\n';
```

4.4 基环树

```cpp
#include<bits/stdc++.h>
using namespace std;

typedef long long i64;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 1e5 + 3;

using edge = tuple<int, int, int>;

vector <edge> E[MAXN];
vector <edge> W;
vector <int>  C;

edge F[MAXN];
bool V[MAXN];
int  I[MAXN], o;

void dfs0(int u, int e){
    V[u] = true;
    I[u] = ++ o;
    for(auto &[i, v, w] : E[u]) if(i != e){
        if(V[v]){
            if(I[v] < I[u]){
                for(int p = u;p != v;){
                    auto &[j, f, x] = F[p];
                    C.push_back(p);
                    W.push_back({j, p, x});
                    p = f;
                }
                C.push_back(v);
                W.push_back({i, v, w});
            }
        } else {
            F[v] = {i, u, w};
            dfs0(v, i);
        }
    }
}

```

```
namespace Problem2{
// ═══ 删除环上第 i 条边，求直径 ═══
    i64 H[MAXN], A1[MAXN], B1[MAXN], A2[MAXN], B2[MAXN], A3[MAXN], B3[
        MAXN];

    i64 L[MAXN];
    i64 dis = 0;
    void dfs1(int u, int e){
        for(auto &[i, v, w] : E[u]) if(i ≠ e){
            if(!V[v]){
                dfs1(v, i);
                dis = max(dis, L[u] + w + L[v]);
                L[u] = max(L[u], L[v] + w);
            }
        }
    }
    int main(){
        int n;
        cin >> n;
        for(int i = 1;i ≤ n;++ i){
            int u, v, w;
            cin >> u >> v >> w;
            E[u].push_back({i, v, w});
            E[v].push_back({i, u, w});
        }
        dfs0(1, 0);
        memset(V, 0, sizeof(V));
        for(auto &u : C)
            V[u] = true;
        for(auto &u : C){
            dfs1(u, 0);
        }
        int l = 0, r = C.size() - 1;
        for(int i = l;i ≤ r;++ i){
            int x = C[i];
            if(i > 0)
                H[i] = H[i - 1] + get<2>(W[i - 1]);
            A1[i] = L[x] + H[i];
            B1[i] = L[x] - H[i];
            A2[i] = L[x] - H[i];
            B2[i] = L[x] + H[i];
        }
```

```
        i64 h = H[r] + get<2>(W.back());
        for(int i = l;i ≤ r;++ i)
            A1[i] = max(i == l ? -INFL : A1[i - 1], L[C[i]] + H[i]),
            A2[i] = max(i == l ? -INFL : A2[i - 1], L[C[i]] - H[i]);
        for(int i = r;i ≥ l;-- i)
            B1[i] = max(i == r ? -INFL : B1[i + 1], L[C[i]] - H[i]),
            B2[i] = max(i == r ? -INFL : B2[i + 1], L[C[i]] + H[i]);
        A3[l] = -INFL, B3[r] = -INFL;
        for(int i = l + 1;i ≤ r;++ i){
            int x = C[i];
            i64 w = A2[i - 1] + L[x] + H[i];
            A3[i] = max(A3[i - 1], w);
        }
        for(int i = r - 1;i ≥ l;-- i){
            int x = C[i];
            i64 w = B2[i + 1] + L[x] - H[i];
            B3[i] = max(B3[i + 1], w);
        }
        i64 t =  INFL;
        for(int i = l;i < r;++ i){
            i64 d = A1[i] + B1[i + 1] + h;
            i64 g = A2[i] + B2[i + 1] + 0;
            d = max({d, dis, A3[i], B3[i + 1]});
            t = min(t, d);
        }
        t = min(t, max(A3[r], dis));

        if(t % 2 == 0)
            cout << t / 2 << ".0" << endl;
        if(t % 2 == 1)
            cout << t / 2 << ".5" << endl;
        return 0;
    }
}

namespace Problem3{
// ═══ 求最大点权独立集 ═══

    int A[MAXN];
    i64 X[MAXN], Y[MAXN];
    i64 P[MAXN][2], Q[MAXN][2];
    void dfs1(int u, int e){
```

```
        for(auto &[i, v, w] : E[u]) if(i ≠ e){
            if(!V[v]){
                dfs1(v, i);
                Y[u] += max(X[v], Y[v]);
                X[u] += Y[v];
            }
        }
        X[u] += A[u];
    }

    int main(){
        int n;
        cin >> n;
        for(int i = 1;i ≤ n;++ i){
            cin >> A[i];
        }
        for(int i = 1;i ≤ n;++ i){
            int u, v;
            cin >> u >> v;
            ++ u, ++ v;
            E[u].push_back({i, v, 0});
            E[v].push_back({i, u, 0});
        }
        double p;
        cin >> p;
        dfs0(1, 0);
        memset(V, 0, sizeof(V));
        for(auto &u : C)
            V[u] = true;
        for(auto &u : C){
            dfs1(u, 0);
        }
        int l = 0, r = C.size() - 1;
        P[0][1] = X[C[0]];
        P[0][0] = -INFL;
        Q[0][0] = Y[C[0]];
        Q[0][1] = -INFL;
        for(int i = l + 1;i ≤ r;++ i){
            int x = C[i];
            P[i][1] = X[x] + P[i - 1][0];
            P[i][0] = Y[x] + max(P[i - 1][0], P[i - 1][1]);
            Q[i][1] = X[x] + Q[i - 1][0];
```

```
            Q[i][0] = Y[x] + max(Q[i - 1][0], Q[i - 1][1]);
        }
        i64 ans = max({P[r][0], Q[r][0], Q[r][1]});
        cout << fixed << setprecision(1) << ans * p << endl;
        return 0;
    }
}

int main(){
    return Problem3 :: main();
}
```

4.5 2-SAT

4.5.1 例题

n 个变量 m 个条件，形如若 $x_i = a$ 则 $y_j = b$，找到任意一组可行解或者报告无解。

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;
namespace SCC{
    const int MAXN= 2e6 + 3;

    vector <int> V[MAXN];
    stack  <int> S;
    int D[MAXN], L[MAXN], C[MAXN], o, s;
    bool F[MAXN], I[MAXN];
    void add(int u, int v){ V[u].push_back(v); }
    void dfs(int u){
        L[u] = D[u] = ++ o, S.push(u), I[u] = F[u] = true;
        for(auto &v : V[u]){
            if(F[v]){
                if(I[v]) L[u] = min(L[u], D[v]);
            } else {
                dfs(v),  L[u] = min(L[u], L[v]);
            }
        }
        if(L[u] == D[u]){
            int c = ++ s;
            while(S.top() != u){
```

```
                int v = S.top(); S.pop();
                I[v] = false;
                C[v] = c;
            }
            S.pop(), I[u] = false, C[u] = c;
        }
    }
}
const int MAXN = 1e6 + 3;
int X[MAXN][2], o;
int main(){
    ios :: sync_with_stdio(false);
    int n, m;
    cin >> n >> m;

    for(int i = 1;i ≤ n;++ i)
        X[i][0] = ++ o;
    for(int i = 1;i ≤ n;++ i)
        X[i][1] = ++ o;
    for(int i = 1;i ≤ m;++ i){
        int a, x, b, y;
        cin >> a >> x >> b >> y;
        SCC :: add(X[a][!x], X[b][y]);
        SCC :: add(X[b][!y], X[a][x]);
    }
    for(int i = 1;i ≤ o;++ i)
        if(!SCC :: F[i])
            SCC :: dfs(i);
    bool ok = true;
    for(int i = 1;i ≤ n;++ i){
        if(SCC :: C[X[i][0]] == SCC :: C[X[i][1]])
            ok = false;
    }
    if(ok){
        cout << "POSSIBLE" << endl;
        for(int i = 1;i ≤ n;++ i){
            int a = SCC :: C[X[i][0]];
            int b = SCC :: C[X[i][1]];
            if(a < b)
                cout << 0 << "␣";
            else
                cout << 1 << "␣";
```

```
        }
        cout << endl;
    } else {
        cout << "IMPOSSIBLE" << endl;
    }
    return 0;
}
```

4.6 割点

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN= 2e4 + 3;
const int MAXM= 1e5 + 3;
vector<int> V[MAXN];

int n, m, o, D[MAXN], L[MAXN];
bool F[MAXN], C[MAXN];
void dfs(int u, int g){
    L[u] = D[u] = ++ o, F[u] = true; int s = 0;
    for(auto &v : V[u]){
        if(!F[v]){
            dfs(v, g), ++ s;
            L[u] = min(L[u], L[v]);
            if(u != g && L[v] >= D[u]) C[u] = true;
        } else {
            L[u] = min(L[u], D[v]);
        }
    }
    if(u == g && s > 1) C[u] = true;
}
int main(){
    cin >> n >> m;
    for(int i = 1;i <= m;++ i){
        int u, v;
        cin >> u >> v;
        V[u].push_back(v);
        V[v].push_back(u);
    }
    for(int i = 1;i <= n;++ i)
        if(!F[i]) dfs(i, i);
```

```
    vector <int> ANS;
    for(int i = 1;i ≤ n;++ i)
        if(C[i]) ANS.push_back(i);
    cout << ANS.size() << endl;
    for(auto &u : ANS)
        cout << u << "␣";
    return 0;
}
```

4.7 边双连通分量

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN= 5e5 + 3;
vector <vector<int>> A;
vector <pair<int, int>> V[MAXN];
stack  <int> S;
int D[MAXN], L[MAXN], o;
bool I[MAXN];
void dfs(int u, int l){
    D[u] = L[u] = ++ o; I[u] = true, S.push(u); int s = 0;
    for(auto &p : V[u]) {
        int v = p.first, id = p.second;
        if(id ≠ l){
            if(D[v]){
                if(I[v])    L[u] = min(L[u], D[v]);
            } else {
                dfs(v, id), L[u] = min(L[u], L[v]), ++ s;
            }
        }
    }
    if(D[u] == L[u]){
        vector <int> T;
        while(S.top() ≠ u){
            int v = S.top(); S.pop();
            T.push_back(v), I[v] = false;
        }
```

```
        T.push_back(u), S.pop(), I[u] = false;
        A.push_back(T);
    }
}
```

4.8 点双连通分量

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN= 5e5 + 3;
vector <vector<int>> A;
vector <int> V[MAXN];
stack  <int> S;
int D[MAXN], L[MAXN], o; bool I[MAXN];
void dfs(int u, int f){
    D[u] = L[u] = ++ o; I[u] = true, S.push(u); int s = 0;
    for(auto &v : V[u]) if(v != f){
        if(D[v]){
            if(I[v])   L[u] = min(L[u], D[v]);
        } else {
            dfs(v, u), L[u] = min(L[u], L[v]), ++ s;
            if(L[v] >= D[u]){
                vector <int> T;
                while(S.top() != v){
                    int t = S.top(); S.pop();
                    T.push_back(t), I[t] = false;
                }
                T.push_back(v), S.pop(), I[v] = false;
                T.push_back(u);
                A.push_back(T);
            }
        }
    }
    if(f == 0 && s == 0){
        A.push_back({u});
    }
}
```

4.9 强连通分量

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN= 5e5 + 3;

vector <int> V[MAXN];
stack  <int> S;
int D[MAXN], L[MAXN], C[MAXN], o, s;
bool F[MAXN], I[MAXN];
void add(int u, int v){ V[u].push_back(v); }
void dfs(int u){
    L[u] = D[u] = ++ o, S.push(u), I[u] = F[u] = true;
    for(auto &v : V[u]){
        if(F[v]){
            if(I[v]) L[u] = min(L[u], D[v]);
        } else {
            dfs(v),  L[u] = min(L[u], L[v]);
        }
    }
    if(L[u] == D[u]){
        int c = ++ s;
        while(S.top() != u){
            int v = S.top(); S.pop();
            I[v] = false;
            C[v] = c;
        }
        S.pop(), I[u] = false, C[u] = c;
    }
}

vector <int> ANS[MAXN];
int main(){

    int n, m;
    cin >> n >> m;
    for(int i = 1;i <= m;++ i){
        int u, v;
```

```
        cin >> u >> v;
        V[u].push_back(v);
    }
    for(int i = 1;i ≤ n;++ i)
        if(!F[i])
            dfs(i);
    for(int i = 1;i ≤ n;++ i){
        ANS[C[i]].push_back(i);
    }
    cout << s << endl;
    for(int i = 1;i ≤ n;++ i) if(F[i]){
        int c = C[i];
        sort(ANS[c].begin(), ANS[c].end());
        for(auto &u : ANS[c])
            cout << u << "␣", F[u] = false;
        cout << endl;
    }
    return 0;
}
```

5 网络流

5.1 费用流

```
#include<bits/stdc++.h>
using namespace std;
namespace MCMF{
    using i64 = long long;
    const i64 INF  = 1e18;
    const int MAXN = 1e5 + 3;
    const int MAXM = 2e5 + 3;
    int H[MAXN], V[MAXM], N[MAXM], W[MAXM], F[MAXM], o = 1, n;
    void add(int u, int v, int f, int c){
        V[++ o] = v, N[o] = H[u], H[u] = o, F[o] = f, W[o] =  c;
        V[++ o] = u, N[o] = H[v], H[v] = o, F[o] = 0, W[o] = -c;
        n = max(n, u);
        n = max(n, v);
    }
    void clear(){
        for(int i = 1;i ≤ n;++ i)
            H[i] = 0;
        n = 0, o = 1;
```

```
    }
    bool I[MAXN];
    i64 D[MAXN];
    bool spfa(int s, int t){
        queue <int> Q;
        Q.push(s), I[s] = true;
        for(int i = 1;i ≤ n;++ i)
            D[i] = INF;
        D[s] = 0;
        while(!Q.empty()){
            int u = Q.front(); Q.pop(), I[u] = false;
            for(int i = H[u];i;i = N[i]){
                const int &v = V[i];
                const int &f = F[i];
                const int &w = W[i];
                if(f && D[u] + w < D[v]){
                    D[v] = D[u] + w;
                    if(!I[v]) Q.push(v), I[v] = true;
                }
            }
        }
        return D[t] ≠ INF;
    }
    int C[MAXN]; bool T[MAXN];
    pair<i64, i64> dfs(int s, int t, int u, i64 maxf){
        if(u == t)
            return make_pair(maxf, 0);
        i64 totf = 0;
        i64 totc = 0;
        T[u] = true;
        for(int &i = C[u];i;i = N[i]){
            const int &v = V[i];
            const int &f = F[i];
            const int &w = W[i];
            if(f && D[v] == D[u] + w && !T[v]){
                auto p = dfs(s, t, v, min(1ll * F[i], maxf));
                i64 f = p.first;
                i64 c = p.second;
                F[i     ] -= f;
                F[i ^ 1] += f;
                totf += f;
                totc += 1ll * f * W[i] + c;
```

```
                maxf -= f;
                if(maxf == 0){
                    T[u] = false;
                    return make_pair(totf, totc);
                }
            }
        }
        T[u] = false;
        return make_pair(totf, totc);
    }
    pair<i64, i64> mcmf(int s, int t){
        i64 ans1 = 0;
        i64 ans2 = 0;
        pair<i64, i64> r;
        while(spfa(s, t)){
            memcpy(C, H, sizeof(H));
            r = dfs(s, t, s, INF);
            ans1 += r.first;
            ans2 += r.second;
        }
        return make_pair(ans1, ans2);
    }
}
int qread();
int main(){
    int n = qread(), m = qread(), s = qread(), t = qread();
    for(int i = 1;i <= m;++ i){
        int u = qread(), v = qread(), f = qread(), c = qread();
        MCMF :: add(u, v, f, c);
    }
    pair<long long, long long> ans = MCMF :: mcmf(s, t);
    printf("%lld %lld\n", ans.first, ans.second);
    return 0;
}
```

5.2 最小割树

5.2.1 用法

给定无向图求出最小割树，点 u 和 v 作为起点终点的最小割为树上 u 到 v 路径上边权的最小值。

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp

int qread();
namespace Dinic{
    const long long INF = 1e18;
    const int SIZ = 1e5 + 3;
    int n, m;
    int H[SIZ], V[SIZ], N[SIZ], F[SIZ], t = 1;
    int add(int u, int v, int f){
        V[++ t] = v, N[t] = H[u], F[t] = f, H[u] = t;
        V[++ t] = u, N[t] = H[v], F[t] = 0, H[v] = t;
        n = max(n, u);
        n = max(n, v);
        return t - 1;
    }
    void clear(){
        for(int i = 1;i ≤ n;++ i)
            H[i] = 0;
        n = m = 0, t = 1;
    }
    int D[SIZ];
    bool bfs(int s, int t){
        queue <int> Q;
        for(int i = 1;i ≤ n;++ i)
            D[i] = 0;
        Q.push(s), D[s] = 1;
        while(!Q.empty()){
            int u = Q.front(); Q.pop();
            for(int i = H[u];i;i = N[i]){
                const int &v = V[i];
                const int &f = F[i];
                if(f ≠ 0 && !D[v]){
                    D[v] = D[u] + 1;
                    Q.push(v);
                }
            }
        }
        return D[t] ≠ 0;
    }
    int C[SIZ];
    long long dfs(int s, int t, int u, long long maxf){
        if(u = t)
            return maxf;
```

```
        long long totf = 0;
        for(int &i = C[u];i;i = N[i]){
            const int &v = V[i];
            const int &f = F[i];
            if(D[v] == D[u] + 1){
                long long resf = dfs(s, t, v, min(maxf, 1ll * f));
                totf += resf;
                maxf -= resf;
                F[i    ] -= resf;
                F[i ^ 1] += resf;
                if(maxf == 0)
                    return totf;
            }
        }
        return totf;
    }
    long long dinic(int s, int t){
        long long ans = 0;
        while(bfs(s, t)){
            memcpy(C, H, sizeof(H));
            ans += dfs(s, t, s, INF);
        }
        return ans;
    }
}

namespace GHTree{
    const int MAXN =  500 + 5;
    const int MAXM = 1500 + 5;
    const int INF  = 1e9;
    int n, m, U[MAXM], V[MAXM], W[MAXM], A[MAXM], B[MAXM];
    void add(int u, int v, int w){
        ++ m;
        U[m] = u;
        V[m] = v;
        W[m] = w;
        A[m] = Dinic :: add(u, v, w);
        B[m] = Dinic :: add(v, u, w);
        n = max(n, u);
        n = max(n, v);
    }
    vector <pair<int, int> > E[MAXN];
```

```
    void build(vector <int> N){
        int s = N.front();
        int t = N.back();
        if(s == t) return;
        for(int i = 1;i <= m;++ i){
            int a = A[i]; Dinic :: F[a] = W[i], Dinic :: F[a ^ 1] = 0;
            int b = B[i]; Dinic :: F[b] = W[i], Dinic :: F[b ^ 1] = 0;
        }
        int w = Dinic :: dinic(s, t);
        E[s].push_back(make_pair(t, w));
        E[t].push_back(make_pair(s, w));

        vector <int> P;
        vector <int> Q;
        for(auto &u : N){
            if(Dinic :: D[u] != 0)
                P.push_back(u);
            else
                Q.push_back(u);
        }
        build(P), build(Q);
    }
    int D[MAXN];
    int cut(int s, int t){
        queue <int> Q; Q.push(s);
        for(int i = 1;i <= n;++ i)
            D[i] = -1;
        D[s] = INF;
        while(!Q.empty()){
            int u = Q.front(); Q.pop();
            for(auto &e : E[u]){
                int v = e.first;
                int w = e.second;
                if(D[v] == -1){
                    D[v] = min(D[u], w);
                    Q.push(v);
                }
            }
        }
        return D[t];
    }
}
```

5.3 最大流

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

namespace Dinic{
    const i64 INF = 1e18;
    const int SIZ = 5e5 + 3;
    int n, m;
    int H[SIZ], V[SIZ], N[SIZ], F[SIZ], t = 1;
    void add(int u, int v, int f){
        V[++ t] = v, N[t] = H[u], F[t] = f, H[u] = t;
        V[++ t] = u, N[t] = H[v], F[t] = 0, H[v] = t;
        n = max(n, u);
        n = max(n, v);
    }
    void clear(){
        for(int i = 1;i ≤ n;++ i)
            H[i] = 0;
        n = m = 0, t = 1;
    }
    int D[SIZ];
    bool bfs(int s, int t){
        queue <int> Q;
        for(int i = 1;i ≤ n;++ i)
            D[i] = 0;
        Q.push(s), D[s] = 1;
        while(!Q.empty()){
            int u = Q.front(); Q.pop();
            for(int i = H[u];i;i = N[i]){
                const int &v = V[i];
                const int &f = F[i];
                if(f ≠ 0 && !D[v]){
                    D[v] = D[u] + 1;
                    Q.push(v);
                }
            }
        }
        return D[t] ≠ 0;
```

```
    }
    int C[SIZ];
    i64 dfs(int s, int t, int u, i64 maxf){
        if(u == t)
            return maxf;
        i64 totf = 0;
        for(int &i = C[u];i;i = N[i]){
            const int &v = V[i];
            const int &f = F[i];
            if(D[v] == D[u] + 1){
                i64 resf = dfs(s, t, v, min(maxf, 1ll * f));
                totf += resf;
                maxf -= resf;
                F[i    ] -= resf;
                F[i ^ 1] += resf;
                if(maxf == 0)
                    return totf;
            }
        }
        return totf;
    }
    i64 dinic(int s, int t){
        i64 ans = 0;
        while(bfs(s, t)){
            memcpy(C, H, sizeof(H));
            ans += dfs(s, t, s, INF);
        }
        return ans;
    }
}

// ==== TEST ====

int qread();

int main(){
    int n = qread(), m = qread(), s = qread(), t = qread();
    for(int i = 1;i <= m;++ i){
        int u = qread(), v = qread(), f = qread();
        Dinic :: add(u, v, f);
    }
    printf("%lld\n", Dinic :: dinic(s, t));
```

```
    return 0;
}
```

5.4 上下界费用流

5.4.1 用法

- add(u, v, l, r, c): 连一条容量在 $[l,r]$ 的从 u 到 v 的费用为 c 的边;
- solve(): 计算无源汇最小费用可行流;
- solve(s, t): 计算有源汇最小费用最大流。

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  = 1e9;
const i64 INFL = 1e18;

namespace MCMF{
    const int MAXN = 1e5 + 3;
    const int MAXM = 2e5 + 3;
    int H[MAXN], V[MAXM], N[MAXM], W[MAXM], F[MAXM], o = 1, n;
    void add0(int u, int v, int f, int c){
        V[++ o] = v, N[o] = H[u], H[u] = o, F[o] = f, W[o] =  c;
        V[++ o] = u, N[o] = H[v], H[v] = o, F[o] = 0, W[o] = -c;
        n = max(n, u);
        n = max(n, v);
    }
    bool I[MAXN];
    i64 D[MAXN];
    bool spfa(int s, int t){
        queue <int> Q;
        Q.push(s), I[s] = true;
        for(int i = 1;i ≤ n;++ i)
            D[i] = INFL;
        D[s] = 0;
        while(!Q.empty()){
            int u = Q.front(); Q.pop(), I[u] = false;
            for(int i = H[u];i;i = N[i]){
                const int &v = V[i];
                const int &f = F[i];
                const int &w = W[i];
                if(f && D[u] + w < D[v]){
```

```
                    D[v] = D[u] + w;
                    if(!I[v]) Q.push(v), I[v] = true;
                }
            }
        }
        return D[t] ≠ INFL;
    }
    int C[MAXN]; bool T[MAXN];
    pair<i64, i64> dfs(int s, int t, int u, i64 maxf){
        if(u == t)
            return make_pair(maxf, 0);
        i64 totf = 0;
        i64 totc = 0;
        T[u] = true;
        for(int &i = C[u];i;i = N[i]){
            const int &v = V[i];
            const int &f = F[i];
            const int &w = W[i];
            if(f && D[v] == D[u] + w && !T[v]){
                auto p = dfs(s, t, v, min(1ll * F[i], maxf));
                i64 f = p.first;
                i64 c = p.second;
                F[i     ] -= f;
                F[i ^ 1] += f;
                totf += f;
                totc += 1ll * f * W[i] + c;
                maxf -= f;
                if(maxf == 0){
                    T[u] = false;
                    return make_pair(totf, totc);
                }
            }
        }
        T[u] = false;
        return make_pair(totf, totc);
    }
    pair<i64, i64> mcmf(int s, int t){
        i64 ans1 = 0;
        i64 ans2 = 0;
        pair<i64, i64> r;
        while(spfa(s, t)){
            memcpy(C, H, sizeof(H));
```

```
            r = dfs(s, t, s, INFL);
            ans1 += r.first;
            ans2 += r.second;
        }
        return make_pair(ans1, ans2);
    }
    i64 cost0;
    int G[MAXN];
    void add(int u, int v, int l, int r, int c){
        G[v] += l;
        G[u] -= l;
        cost0 += 1ll * l * c;
        add0(u, v, r - l, c);
    }
    i64 solve(){
        int s = ++ n;
        int t = ++ n;
        i64 sum = 0;
        for(int i = 1;i <= n - 2;++ i){
            if(G[i] < 0)
                add0(i, t, -G[i], 0);
            else
                add0(s, i,  G[i], 0), sum += G[i];
        }
        auto res = mcmf(s, t);
        if(res.first != sum)
            return -1;
        return res.second + cost0;
    }
    i64 solve(int s0, int t0){
        add0(t0, s0, INF, 0);
        int s = ++ n;
        int t = ++ n;
        i64 sum = 0;
        for(int i = 1;i <= n - 2;++ i){
            if(G[i] < 0)
                add0(i, t, -G[i], 0);
            else
                add0(s, i,  G[i], 0), sum += G[i];
        }
        auto res = mcmf(s, t);
        if(res.first != sum)
```

```
            return -1;
        return res.second + cost0;
    }
}

// ==== TEST ====

int qread();
int main(){

    return 0;
}
```

5.5 上下界最大流

5.5.1 用法

- add(u, v, l, r, c): 连一条容量在 $[l,r]$ 的从 u 到 v 的边;
- solve(): 检查是否存在无源汇可行流;
- solve(s, t): 计算有源汇最大流。

```
#include <bits/stdc++.h>
using namespace std;

int qread();
using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

namespace MCMF{
    const int MAXN = 1e5 + 3;
    const int MAXM = 2e5 + 3;
    int H[MAXN], V[MAXM], N[MAXM], F[MAXM], o = 1, n;
    void add0(int u, int v, int f){
        V[++ o] = v, N[o] = H[u], H[u] = o, F[o] = f;
        V[++ o] = u, N[o] = H[v], H[v] = o, F[o] = 0;
        n = max(n, u);
        n = max(n, v);
    }
    i64 D[MAXN];
    bool bfs(int s, int t){
        queue <int> Q;
        for(int i = 1;i ≤ n;++ i)
```

```
            D[i] = 0;
        Q.push(s), D[s] = 1;
        while(!Q.empty()){
            int u = Q.front(); Q.pop();
            for(int i = H[u];i;i = N[i]){
                const int &v = V[i];
                const int &f = F[i];
                if(f != 0 && !D[v]){
                    D[v] = D[u] + 1;
                    Q.push(v);
                }
            }
        }
        return D[t] != 0;
    }
    int C[MAXN];
    i64 dfs(int s, int t, int u, i64 maxf){
        if(u == t)
            return maxf;
        i64 totf = 0;
        for(int &i = C[u];i;i = N[i]){
            const int &v = V[i];
            const int &f = F[i];
            if(f && D[v] == D[u] + 1){
                i64 f = dfs(s, t, v, min(1ll * F[i], maxf));
                F[i     ] -= f;
                F[i ^ 1] += f;
                totf += f;
                maxf -= f;
                if(maxf == 0){
                    return totf;
                }
            }
        }
        return totf;
    }
    i64 mcmf(int s, int t){
        i64 ans = 0;
        while(bfs(s, t)){
            memcpy(C, H, sizeof(H));
            ans += dfs(s, t, s, INFL);
        }
```

```
        return ans;
    }
    int G[MAXN];
    void add(int u, int v, int l, int r){
        G[v] += l;
        G[u] -= l;
        add0(u, v, r - l);
    }
    void clear(){
        for(int i = 1;i ≤ o;++ i){
            N[i] = F[i] = V[i] = 0;
        }
        for(int i = 1;i ≤ n;++ i){
            H[i] = G[i] = C[i] = 0;
        }
        o = 1, n = 0;
    }
    bool solve(){
        int s = ++ n;
        int t = ++ n;
        i64 sum = 0;
        for(int i = 1;i ≤ n - 2;++ i){
            if(G[i] < 0)
                add0(i, t, -G[i]);
            else
                add0(s, i,  G[i]), sum += G[i];
        }
        auto res = mcmf(s, t);
        if(res ≠ sum)
            return true;
        return false;
    }
    i64 solve(int s0, int t0){
        add0(t0, s0, INF);
        int s = ++ n;
        int t = ++ n;
        i64 sum = 0;
        for(int i = 1;i ≤ n - 2;++ i){
            if(G[i] < 0)
                add0(i, t, -G[i]);
            else
                add0(s, i,  G[i]), sum += G[i];
```

```
        }
        auto res = mcmf(s, t);
        if(res ≠ sum)
            return -1;
        return mcmf(s0, t0);
    }
}

const int MAXN = 1e3 + 3;
const int MAXM = 365 + 3;

int G[MAXN], A[MAXN], B[MAXM];

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, o = 0;
    while(cin >> n >> m){

        int s = ++ o;
        int t = ++ o;

        for(int i = 1;i ≤ m;++ i){
            cin >> G[i];
            A[i] = ++ o;
            MCMF :: add(A[i], t, G[i], INF);
        }
        for(int i = 1;i ≤ n;++ i){
            B[i] = ++ o;

            int c, d;
            cin >> c >> d;

            MCMF :: add(s, B[i], 0, d);

            for(int j = 1;j ≤ c;++ j){
                int t, l, r;
                cin >> t >> l >> r;
                t ++;
                MCMF :: add(B[i], A[t], l, r);
            }
```

```
        }
        cout << MCMF :: solve(s, t) << "\n\n";

        MCMF :: clear();
    }

    return 0;
}
```

6 数学

6.1 线性代数

6.1.1 行列式

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 600 + 3;
int MOD;

struct Mat{
    int n, m;
    int W[MAXN][MAXN];
    Mat(int _n = 0, int _m = 0){
        n = _n;
        m = _m;
        for(int i = 1;i ≤ n;++ i)
            for(int j = 1;j ≤ m;++ j)
                W[i][j] = 0;
    }
};

int mat_det(Mat a){
    int ans = 1;

    const int &n = a.n;
    for(int i = 1;i ≤ n;++ i){
        int f = -1;
```

```
        for(int j = i;j ≤ n;++ j) if(a.W[j][i] ≠ 0){
            f = j;
            break;
        }
        if(f == -1){
            return 0;
        }
        if(f ≠ i){
            for(int j = 1;j ≤ n;++ j)
                swap(a.W[i][j], a.W[f][j]);
            ans = MOD - ans;
        }
        for(int j = i + 1;j ≤ n;++ j) if(a.W[j][i]){
            while(a.W[j][i]){
                int u = a.W[i][i];
                int v = a.W[j][i];
                if(u > v){
                    for(int k = 1;k ≤ n;++ k)
                        swap(a.W[i][k], a.W[j][k]);
                    ans = MOD - ans;
                    swap(u, v);
                }
                int rate = v / u;
                for(int k = 1;k ≤ n;++ k){
                    a.W[j][k] = (a.W[j][k] - 1ll * rate * a.W[i][k] %
                        MOD + MOD) % MOD;
                }
            }
        }
    }
    for(int i = 1;i ≤ n;++ i)
        ans = 1ll * ans * a.W[i][i] % MOD;
    return ans;
}

int main(){
    int n;
    cin >> n >> MOD;
    Mat A(n, n);
    for(int i = 1;i ≤ n;++ i)
        for(int j = 1;j ≤ n;++ j)
            cin >> A.W[i][j], A.W[i][j] %= MOD;
```

```
    cout << mat_det(A) << endl;
    return 0;
}
```

6.1.2 高斯消元与求秩（实数）

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 100 + 3;

const double EPS = 1e-9;

struct Mat{
    int n, m;
    double W[MAXN][MAXN];
    Mat(int _n = 0, int _m = 0){
        n = _n;
        m = _m;
        for(int i = 1;i ≤ n;++ i)
            for(int j = 1;j ≤ m;++ j)
                W[i][j] = 0;
    }
};

bool zero(double f){
    return fabs(f) < EPS;
}

int mat_rank(Mat &a){
    const int &n = a.n;
    const int &m = a.m;
    int cnt = 0;
    for(int i = 1;i ≤ m;++ i){
        int p = cnt + 1;
        int f = -1;
        for(int j = p;j ≤ n;++ j){
            if(!zero(a.W[j][i])){
```

```
                f = j;
                break;
            }
        }
        if(f == -1)
            continue;
        if(f != p){
            for(int j = 1;j <= m;++ j)
                swap(a.W[p][j], a.W[f][j]);
        }
        ++ cnt;
        for(int j = p + 1;j <= n;++ j){
            double rate = a.W[j][i] / a.W[p][i];
            for(int k = 1;k <= m;++ k){
                a.W[j][k] -= rate * a.W[p][k];
            }
        }
    }
    return cnt;
}

double X[MAXN];

int main(){
    int n;
    cin >> n;
    Mat A(n, n);
    Mat T(n, n + 1);
    for(int i = 1;i <= n;++ i){
        for(int j = 1;j <= n;++ j)
            cin >> A.W[i][j];
        for(int j = 1;j <= n;++ j)
            T.W[i][j] = A.W[i][j];
        cin >> T.W[i][n + 1];
    }
    int res1 = mat_rank(A);
    int res2 = mat_rank(T);
    if(res1 != res2)
        cout << -1 << endl;
    else
    if(res2 <  n)
        cout << 0 << endl;
```

```
    else {
        for(int i = n;i ≥ 1;-- i){
            X[i] = T.W[i][n + 1] / T.W[i][i];
            for(int j = i - 1;j ≥ 1;-- j){
                double rate = T.W[j][i] / T.W[i][i];
                T.W[j][    i] -= rate * T.W[i][    i];
                T.W[j][n + 1] -= rate * T.W[i][n + 1];
            }
        }
        for(int i = 1;i ≤ n;++ i)
            cout << "x" << i << "=" << fixed << setprecision(2) << X[i]
                << endl;
    }

    return 0;
}
```

6.1.3 高斯消元与求秩（整数）

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 100 + 3;

const int MOD  = 998244353;

struct Mat{
    int n, m;
    int W[MAXN][MAXN];
    Mat(int _n = 0, int _m = 0){
        n = _n;
        m = _m;
        for(int i = 1;i ≤ n;++ i)
            for(int j = 1;j ≤ m;++ j)
                W[i][j] = 0;
    }
};

```

```
int power(int a, int b){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % MOD;
        b >>= 1,  a = 1ll * a * a % MOD;
    }
    return r;
}

int inv(int x){
    return power(x, MOD - 2);
}

int mat_rank(Mat &a){
    const int &n = a.n;
    const int &m = a.m;
    int cnt = 0;
    for(int i = 1;i ≤ m;++ i){
        int p = cnt + 1;
        int f = -1;
        for(int j = p;j ≤ n;++ j){
            if(a.W[j][i] ≠ 0){
                f = j;
                break;
            }
        }
        if(f == -1)
            continue;
        if(f ≠ p){
            for(int j = 1;j ≤ m;++ j)
                swap(a.W[p][j], a.W[f][j]);
        }
        ++ cnt;
        int invp = inv(a.W[p][i]);
        for(int j = p + 1;j ≤ n;++ j){
            int rate = 1ll * a.W[j][i] * invp % MOD;
            for(int k = 1;k ≤ m;++ k){
                a.W[j][k] = (a.W[j][k] - 1ll * rate * a.W[p][k] % MOD +
                    MOD) % MOD;
            }
        }
    }
```

```
    return cnt;
}

int X[MAXN];

int main(){
    int n;
    cin >> n;
    Mat A(n, n);
    Mat T(n, n + 1);
    for(int i = 1;i ≤ n;++ i){
        for(int j = 1;j ≤ n;++ j)
            cin >> A.W[i][j];
        for(int j = 1;j ≤ n;++ j)
            T.W[i][j] = A.W[i][j];
        cin >> T.W[i][n + 1];
    }
    int res1 = mat_rank(A);
    int res2 = mat_rank(T);
    if(res1 ≠ res2)
        cout << -1 << endl;
    else
    if(res2 <  n)
        cout << 0 << endl;
    else {
        for(int i = n;i ≥ 1;-- i){
            int invp = inv(T.W[i][i]);
            X[i] = 1ll * T.W[i][n + 1] * invp % MOD;
            for(int j = i - 1;j ≥ 1;-- j){
                int rate = 1ll * T.W[j][i] * invp % MOD;
                T.W[j][    i] = (T.W[j][    i] - 1ll * rate * T.W[i][
                      i] % MOD + MOD) % MOD;
                T.W[j][n + 1] = (T.W[j][n + 1] - 1ll * rate * T.W[i][n
                   + 1] % MOD + MOD) % MOD;
            }
        }
        for(int i = 1;i ≤ n;++ i)
            cout << "x" << i << "=" << X[i] << endl;
    }

    return 0;
}
```

6.1.4 矩阵求逆

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 400 + 3;
const int MOD  = 1e9 + 7;

struct Mat{
    int n, m;
    int W[MAXN][MAXN];
    Mat(int _n = 0, int _m = 0){
        n = _n;
        m = _m;
        for(int i = 1;i ≤ n;++ i)
            for(int j = 1;j ≤ m;++ j)
                W[i][j] = 0;
    }
};

int power(int a, int b){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % MOD;
        b >>= 1,  a = 1ll * a * a % MOD;
    }
    return r;
}

int inv(int x){
    return power(x, MOD - 2);
}

bool mat_inv(Mat &a){
    const int &n = a.n;
    Mat b(n, n);
    for(int i = 1;i ≤ n;++ i)
        b.W[i][i] = 1;
    for(int i = 1;i ≤ n;++ i){
```

```
        int f = -1;
        for(int j = i;j ≤ n;++ j) if(a.W[j][i] ≠ 0){
            f = j;
            break;
        }
        if(f == -1){
            return false;
        }
        if(f ≠ i){
            for(int j = 1;j ≤ n;++ j)
                swap(a.W[i][j], a.W[f][j]),
                swap(b.W[i][j], b.W[f][j]);
        }
        int invp = inv(a.W[i][i]);
        for(int j = i + 1;j ≤ n;++ j){
            int rate = 1ll * a.W[j][i] * invp % MOD;
            for(int k = 1;k ≤ n;++ k){
                a.W[j][k] = (a.W[j][k] - 1ll * rate * a.W[i][k] % MOD +
                    MOD) % MOD;
                b.W[j][k] = (b.W[j][k] - 1ll * rate * b.W[i][k] % MOD +
                    MOD) % MOD;
            }
        }
    }
    for(int i = n;i ≥ 1;-- i){
        int invp = inv(a.W[i][i]);
        for(int j = 1;j ≤ n;++ j){
            a.W[i][j] = 1ll * a.W[i][j] * invp % MOD;
            b.W[i][j] = 1ll * b.W[i][j] * invp % MOD;
        }
        for(int j = i - 1;j ≥ 1;-- j){
            int rate = 1ll * a.W[j][i] % MOD;
            for(int k = 1;k ≤ n;++ k){
                a.W[j][k] = (a.W[j][k] - 1ll * rate * a.W[i][k] % MOD +
                    MOD) % MOD;
                b.W[j][k] = (b.W[j][k] - 1ll * rate * b.W[i][k] % MOD +
                    MOD) % MOD;
            }
        }
    }
    for(int i = 1;i ≤ n;++ i)
        for(int j = 1;j ≤ n;++ j)
```

```
            a.W[i][j] = b.W[i][j];
    return true;
}

int X[MAXN];

int main(){
    int n;
    cin >> n;
    Mat A(n, n);
    for(int i = 1;i ≤ n;++ i)
        for(int j = 1;j ≤ n;++ j)
            cin >> A.W[i][j];
    bool res = mat_inv(A);
    if(res == false){
        cout << "No Solution" << endl;
    } else {
        for(int i = 1;i ≤ n;++ i)
            for(int j = 1;j ≤ n;++ j)
                cout << A.W[i][j] << " \n"[j == n];
    }
    return 0;
}
```

6.1.5 矩阵树

LGV 定理叙述 设 G 是一张有向无环图，边带权，每个点的度数有限。给定起点集合 $A = \{a_1, a_2, \cdots, a_n\}$，终点集合 $B = \{b_1, b_2, \cdots, b_n\}$。

- 一段路径 $p : v_0 \to^{w_1} v_1 \to^{w_2} v_2 \to \cdots \to^{w_k} v_k$ 的边权被定义为 $\omega(p) = \prod w_i$。
- 一对顶点 (a, b) 的权值被定义为 $e(a, b) = \sum_{p:a\to b} \omega(p)$。

设矩阵 M 如下：

$$M = \begin{pmatrix} e(a_1, b_1) & e(a_1, b_2) & \cdots & e(a_1, b_n) \\ e(a_2, b_1) & e(a_2, b_2) & \cdots & e(a_2, b_n) \\ \vdots & \vdots & \ddots & \vdots \\ e(a_n, b_1) & e(a_n, b_2) & \cdots & e(a_n, b_n) \end{pmatrix}$$

从 A 到 B 得到一个**不相交**的路径组 $p = (p_1, p_2, \cdots, p_n)$，其中从 a_i 到达 b_{π_i}，π 是一个排列。定义 $\sigma(\pi)$ 是 π 逆序对的数量。

给出 LGV 的叙述如下：

$$\det(M) = \sum_{p:A\to B} (-1)^{\sigma(\pi)} \prod_{i=1}^{n} \omega(p_i)$$

可以将边权视作边的重数，那么 $e(a,b)$ 就可以视为从 a 到 b 的不同路径方案数。

矩阵树定理 对于无向图，

- 定义度数矩阵 $D_{i,j} = [i=j]\deg(i)$；
- 定义邻接矩阵 $E_{i,j} = E_{j,i}$ 是从 i 到 j 的边数个数；
- 定义拉普拉斯矩阵 $L = D - E$。

对于无向图的矩阵树定理叙述如下：

$$t(G) = \det(L_i) = \frac{1}{n}\lambda_1\lambda_2\cdots\lambda_{n-1}$$

其中 L_i 是将 L 删去第 i 行和第 i 列得到的子式。

对于有向图，类似于无向图定义入度矩阵、出度矩阵、邻接矩阵 $D^{\text{in}}, D^{\text{out}}, E$，同时定义拉普拉斯矩阵 $L^{\text{in}} = D^{\text{in}} - E, L^{\text{out}} - E$。

$$t^{\text{leaf}}(G,k) = \det(L_k^{\text{in}})$$
$$t^{\text{root}}(G,k) = \det(L_k^{\text{out}})$$

其中 $t^{\text{leaf}}(G,k)$ 表示以 k 为根的叶向树，$t^{\text{root}}(G,k)$ 表示以 k 为根的根向树。

BEST 定理 对于一个有向欧拉图 G，记点 i 的出度为 out_i，同时 G 的根向生成树个数为 T。T 可以任意选取根。则 G 的本质不同的欧拉回路个数为：

$$T\prod_i(\text{out}_i - 1)!$$

```cpp
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 300 + 3;
const int MOD  = 1e9 + 7;

struct Mat{
    int n, m;
    int W[MAXN][MAXN];
    Mat(int _n = 0, int _m = 0){
        n = _n;
        m = _m;
        for(int i = 1;i ≤ n;++ i)
            for(int j = 1;j ≤ m;++ j)
                W[i][j] = 0;
```

```
    }
};

int mat_det(Mat a){
    int ans = 1;

    const int &n = a.n;
    for(int i = 1;i ≤ n;++ i){
        int f = -1;
        for(int j = i;j ≤ n;++ j) if(a.W[j][i] ≠ 0){
            f = j;
            break;
        }
        if(f == -1){
            return 0;
        }
        if(f ≠ i){
            for(int j = 1;j ≤ n;++ j)
                swap(a.W[i][j], a.W[f][j]);
            ans = MOD - ans;
        }
        for(int j = i + 1;j ≤ n;++ j) if(a.W[j][i]){
            while(a.W[j][i]){
                int u = a.W[i][i];
                int v = a.W[j][i];
                if(u > v){
                    for(int k = 1;k ≤ n;++ k)
                        swap(a.W[i][k], a.W[j][k]);
                    ans = MOD - ans;
                    swap(u, v);
                }
                int rate = v / u;
                for(int k = 1;k ≤ n;++ k){
                    a.W[j][k] = (a.W[j][k] - 1ll * rate * a.W[i][k] %
                        MOD + MOD) % MOD;
                }
            }
        }
    }
    for(int i = 1;i ≤ n;++ i)
        ans = 1ll * ans * a.W[i][i] % MOD;
    return ans;
```

```
}

int D[MAXN];
int W[MAXN][MAXN];

int main(){
    int n, m, t;
    cin >> n >> m >> t;
    for(int i = 1;i ≤ m;++ i){
        int u, v, w;
        cin >> u >> v >> w;
        if(u ≠ v){
            if(t = 0){ // 无向图
                D[u] = (D[u] + w) % MOD;
                D[v] = (D[v] + w) % MOD;
                W[u][v] = (W[u][v] + w) % MOD;
                W[v][u] = (W[v][u] + w) % MOD;
            } else {    // 叶向树
                D[v] = (D[v] + w) % MOD;
                W[u][v] = (W[u][v] + w) % MOD;
            }
        }
    }
    Mat A(n - 1, n - 1);
    for(int i = 2;i ≤ n;++ i)
        for(int j = 2;j ≤ n;++ j)  // 以 1 为根的叶向树
            A.W[i - 1][j - 1] = MOD - W[i][j];
    for(int i = 2;i ≤ n;++ i)
        A.W[i - 1][i - 1] = (D[i] + A.W[i - 1][i - 1]) % MOD;
    cout << mat_det(A) << endl;
    return 0;
}
```

6.2 大步小步

6.2.1 用法

给定 a, p 求出 x 使得 $a^x = y \pmod p$，其中 p 为质数。

```
#include<bits/stdc++.h>
using namespace std;

int power(int a, int b, int  p){
    int r = 1;
```

```
    while(b){
        if(b & 1) r = 1ll * r * a % p;
        b >>= 1,  a = 1ll * a * a % p;
    }
    return r;
}
namespace BSGS {
    unordered_map <int, int> M;
    int solve(int a, int y, int p){    // a ^ x = y (mod p)
        M.clear();
        int B = sqrt(p);
        int w1 = y, u1 = power(a, p - 2, p);
        int w2 = 1, u2 = power(a, B, p);
        for(int i = 0;i < B;++ i){
            M[w1] = i;
            w1 = 1ll * w1 * u1 % p;
        }
        for(int i = 0;i < p / B;++ i){
            if(M.count(w2)){
                return i * B + M[w2];
            }
            w2 = 1ll * w2 * u2 % p;
        }
        return -1;
    }
}

int main(){
    int p, b, n;
    cin >> p >> b >> n;

    int ans = BSGS :: solve(b, n, p);
    if(ans == -1){
        cout << "no solution\n";
    } else {
        cout << ans << "\n";
    }
    return 0;
}
```

6.3 中国剩余定理

6.3.1 定理

对于线性方程:

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \dots \\ x \equiv a_n \pmod{m_n} \end{cases}$$

如果 a_i 两两互质，可以得到 x 的解 $x \equiv L \pmod{M}$，其中 $M = \prod m_i$，而 L 由下式给出:

$$L = \left(\sum a_i m_i \times ((M/m_i)^{-1} \bmod m_i)\right) \bmod M$$

```cpp
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 100 + 3;
long long A[MAXN], B[MAXN], M = 1;

long long exgcd(long long a, long long b, long long &x, long long &y){
    if(a == 0){
        x = 0, y = 1; return b;
    } else {
        long long x0 = 0, y0 = 0;
        long long d = exgcd(b % a, a, x0, y0);
        x = y0 - (b / a) * x0;
        y = x0;
        return d;
    }
}

int main(){
    int n;
    cin >> n;
    for(int i = 1;i <= n;++ i){
        cin >> B[i] >> A[i];
        M = M * B[i];
    }
    long long L = 0;
    for(int i = 1;i <= n;++ i){
        long long m = M / B[i], b, k;
        exgcd(m, B[i], b, k);
```

```
        L = (L + (__int128)A[i] * m * b) % M;
    }
    L = (L % M + M) % M;
    cout << L << endl;
    return 0;
}
```

6.4 狄利克雷前缀和

6.4.1 用法

计算:

$$s(i) = \sum_{d|i} f_d$$

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 2e7 + 3;
unsigned A[MAXN];

int p, P[MAXN]; bool V[MAXN];
void solve(int n){
    for(int i = 2;i <= n;++ i){
        if(!V[i]){
            P[++ p] = i;
            for(int j = 1;j <= n / i;++ j){ // 前缀和
                A[j * i] += A[j];
            }
        }
        for(int j = 1;j <= p && P[j] <= n / i;++ j){
            V[i * P[j]] = true;
            if(i % P[j] == 0)
                break;
        }
    }

}

unsigned seed;
inline unsigned read(){
    seed ^= seed << 13;
    seed ^= seed >> 17;
```

```
    seed ^= seed << 5;
    return seed;
}


int main(){
    int n;
    cin >> n >> seed;
    for(int i = 1;i ≤ n;++ i){
        A[i] = read();
    }
    solve(n);
    unsigned ans = 0;
    for(int i = 1;i ≤ n;++ i){
        ans ^= A[i];
    }
    cout << ans << endl;

    return 0;
}
```

6.5 万能欧几里得

6.5.1 类欧几里得（万能欧几里得）

From zpk

一种神奇递归，对 $y = \left\lfloor \frac{Ax+B}{C} \right\rfloor$ 向右和向上走的每步进行压缩，做到 $O(\log V)$ 复杂度。其中 $A \geq C$ 就是直接压缩，向右之后必有至少 $\lfloor A/C \rfloor$ 步向上。$A < C$ 实际上切换 x, y 轴后，相当于压缩了一个上取整折线，而上取整下取整可以互化，便又可以递归。

代码中从 $(0,0)$ 走到 $(n, \lfloor (An+B)/C \rfloor)$，假设了 $A, B, C \geq 0, C \neq 0$（类欧基本都作此假设），U, R 矩阵是从右往左乘的，对列向量进行优化，和实际操作顺序恰好相反。快速幂的 log 据说可以被递归过程均摊掉，实际上并不会导致变成两个 log。

```
Matrix solve(ll n, ll A, ll B, ll C, Matrix R, Matrix U) {  // (0, 0) 走到 (n, (An+B)/C)
    if (A ≥ C) return solve(n, A % C, B, C, U.qpow(A / C) * R, U);
    ll l = B / C, r = (A * n + B) / C;
    if (l == r) return R.qpow(n) * U.qpow(l);   // l = r → l = r or A = 0 or n = 0.
    ll p = (C * r - B - 1) / A + 1;
    return R.qpow(n - p) * U * solve(r - l - 1, C, C - B % C + A - 1, A, U, R) * U.qpow(l);
}
```

6.6 扩展欧几里得

6.6.1 内容

给定 a, b，求出 $ax + by = \gcd(a, b)$ 的一组 x, y。

```
int exgcd(int a, int b, int &x, int &y){
    if(a == 0){
        x = 0, y = 1; return b;
    } else {
        int x0 = 0, y0 = 0;
        int d = exgcd(b % a, a, x0, y0);
        x = y0 - (b / a) * x0;
        y = x0;
        return d;
    }
}
```

6.7 快速离散对数

6.7.1 用法

给定原根 g 以及模数 mod，T 次询问 x 的离散对数。

复杂度 $\mathcal{O}(\mathrm{mod}^{2/3} + T \log \mathrm{mod})$。

```
#include<bits/stdc++.h>
using namespace std;

int power(int a, int b, int  p){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % p;
        b >>= 1,  a = 1ll * a * a % p;
    }
    return r;
}
namespace BSGS {
    unordered_map <int, int> M;
    int B, U, P, g;
    void init(int g, int P0, int B0){
        M.clear();
        B = B0;
        P = P0;
        U = power(power(g, B, P), P - 2, P);
        int w = 1;
        for(int i = 0;i < B;++ i){
```

```
            M[w] = i;
            w = 1ll * w * g % P;
        }
    }
    int solve(int y){
        int w = y;
        for(int i = 0;i < P / B;++ i){
            if(M.count(w)){
                return i * B + M[w];
            }
            w = 1ll * w * U % P;
        }
        return -1;
    }
}

const int MAXN = 1e5 + 3;
int H[MAXN], P[MAXN], H0, p, h, g, mod;
bool V[MAXN];

int solve(int x){
    if(x ≤ h){
        return H[x];
    }
    int v = mod / x, r = mod % x;
    if(r < x - r){
        return ((H0 + solve(r)) % (mod - 1) - H[v] + mod - 1) % (mod -
            1);
    } else {
        return (solve(x - r) - H[v + 1] + mod - 1) % (mod - 1);
    }
}

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> g >> mod;
    h = sqrt(mod) + 1;

    BSGS :: init(g, mod, sqrt(1ll * mod * sqrt(mod) / log10(mod)));
```

```
    H0 = BSGS :: solve(mod - 1);

    H[1] = 0;
    for(int i = 2;i ≤ h;++ i){
        if(!V[i]){
            P[++ p] = i;
            H[i] = BSGS :: solve(i);
        }
        for(int j = 1;j ≤ p && P[j] ≤ h / i;++ j){
            int &p = P[j];
            H[i * p] = (H[i] + H[p]) % (mod - 1);
            V[i * p] = true;
            if(i % p == 0)
                break;
        }
    }

    cin >> T;
    while(T --){
        int x, tmp = 0;
        cin >> x;
        cout << solve(x) << "\n";
    }
    return 0;
}
```

6.8 快速最大公约数

6.8.1 用法

已知小值域 m 以及 n 次询问，$\mathcal{O}(m)$ 预处理，$\mathcal{O}(1)$ 单次查询 x, y 的最大公约数。

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i ≤ END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i ≥ END##i;-- i)
using namespace std;
typedef long long i64;
const int INF = 2147483647;
const int MAXN= 5e3 + 3;
const int MAXT= 1e6 + 3;
const int MAXM= 1e3 + 3;
int G[MAXM][MAXM];
int T[MAXT][3];
int A[MAXN], B[MAXN], o = 1e6, h = 1e3, V[MAXT];
```

```
int tgcd(int a, int b){
    if(a ≤ h && b ≤ h) return G[a][b];
    return a == b ? a : 1;
}
int qgcd(int a, int b){
    int ans = 1;
    up(0, 2, i){
        if(T[b][i] > h){
            if(a % T[b][i] == 0) a /= T[b][i], ans *= T[b][i];
        } else {
            int d = G[a % T[b][i]][T[b][i]];
            a /= d, ans *= d;
        }
    }
    return ans;
}
const int MOD = 998244353;
int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    up(1, h, i) G[0][i] = G[i][0] = i;
    up(1, h, i) up(1, h, j){
        if(i ≥ j) G[i][j] = G[i - j][j];
            else   G[i][j] = G[i][j - i];
    }
    up(2, o, i) if(!V[i]){
        V[i] = i;
        for(int j = 2;i * j ≤ o;++ j)
            if(!V[i * j]) V[i * j] = i;
    }
    T[1][0] = T[1][1] = T[1][2] = 1;
    up(2, o, i){
        int p = V[i];
        int a = T[i / p][0];
        int b = T[i / p][1];
        int c = T[i / p][2];
        int x, y, z;
        if(p ≥ h){
            x = 1, y = i / p, z = p;
        } else {
            if(c * p ≤ h){
```

```
                x = a, y = b, z = c * p;
            }
            else if(b * p ≤ h){
                x = a, y = b * p, z = c;
                if(y > z) swap(y, z);
            }
            else if(a * p ≤ h){
                x = a * p, y = b, z = c;
                if(x > y) swap(x, y);
                if(y > z) swap(y, z);
            } else {
                x = a * b, y = c, z = p;
                if(x > y) swap(x, y);
                if(y > z) swap(y, z);
                if(x > z) swap(x, z);
            }
        }
        T[i][0] = x;
        T[i][1] = y;
        T[i][2] = z;
    }
    int n;
    cin >> n;
    up(1, n, i) cin >> A[i];
    up(1, n, i) cin >> B[i];
    up(1, n, i){
        int s = 0, u = 1;
        up(1, n, j){
            int d = qgcd(A[i], B[j]);
            u = 1ll * u * i % MOD;
            s = (s + 1ll * d * u) % MOD;
        }
        printf("%d\n", s);
    }
    return 0;
}
```

6.9 原根

6.9.1 用法

计算 P 的最小原根。

6.9.2 附注

原根表，其中 $P = r \times 2^k$，对应原根为 g。

Prime	r	k	g	Prime	r	k	g
5	1	2	2	3221225473	3	30	5
17	1	4	3	75161927681	35	31	3
97	3	5	5	77309411329	9	33	7
193	3	6	5	206158430209	3	36	22
257	1	8	3	2061584302081	15	37	7
7681	15	9	17	2748779069441	5	39	3
12289	3	12	11	6597069766657	3	41	5
40961	5	13	3	39582418599937	9	42	5
65537	1	16	3	79164837199873	9	43	5
786433	3	18	10	263882790666241	15	44	7
5767169	11	19	3	1231453023109121	35	45	3
7340033	7	20	3	1337006139375617	19	46	3
23068673	11	21	3	3799912185593857	27	47	5
104857601	25	22	3	4222124650659841	15	48	19
167772161	5	25	3	7881299347898369	7	50	6
469762049	7	26	3	31525197391593473	7	52	3
1004535809	479	21	3	180143985094819841	5	55	6
2013265921	15	27	31	1945555039024054273	27	56	5
2281701377	17	27	3	4179340454199820289	29	57	3

```cpp
#include<bits/stdc++.h>
using namespace std;

int power(int a, int b, int p){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % p;
        b >>= 1,  a = 1ll * a * a % p;
    }
```

```
    return r;
}

int getphi(int x){
    int t = x, r = x;
    for(int i = 2;i ≤ x / i;++ i){
        if(t % i == 0){
            r = r / i * (i - 1);
            while(t % i == 0)
                t /= i;
        }
    }
    if(t ≠ 1){
        r = r / t * (t - 1);
    }
    return r;
}
vector <int> getprime(int x){
    vector <int> p;
    int t = x;
    for(int i = 2;i ≤ x / i;++ i){
        if(t % i == 0){
            p.push_back(i);
            while(t % i == 0)
                t /= i;
        }
    }
    if(t ≠ 1)
        p.push_back(x);
    return p;
}

bool test(int g, int m, int mm, vector<int> &P){
    for(auto &p: P){
        if(power(g, mm / p, m) == 1)
            return false;
    }
    return true;
}

int get_genshin(int m){
    int mm = getphi(m);
```

```
    vector <int> P = getprime(mm);
    for(int i = 1;;++ i){
        if(test(i, m, mm, P))
            return i;
    }
}

int main(){
    cout << get_genshin(998244353) << endl;
    return 0;
}
```

6.10 快速乘法逆元（离线）

6.10.1 用法

离线计算 $x = [x_1, x_2, \cdots, x_n]$ 在模 p 意义下的逆元。

```
#include<bits/stdc++.h>
using namespace std;

int power(int a, int b, int p){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % p;
        b >>= 1,  a = 1ll * a * a % p;
    }
    return r;
}

const int MAXN = 5e6 + 3;
int A[MAXN], B[MAXN];
int P[MAXN], Q[MAXN];

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    int n, p, K, S = 1;
    cin >> n >> p >> K;
    P[0] = 1;
    for(int i = 1;i ≤ n;++ i){
        cin >> A[i];
        P[i] = 1ll * P[i - 1] * A[i] % p;
```

```
    }
    Q[n] = power(P[n], p - 2, p);
    for(int i = n;i ≥ 1;-- i){
        Q[i - 1] = 1ll * Q[i] * A[i] % p;
        B[i] = 1ll * Q[i] * P[i - 1] % p;
    }
    int ans = 0;
    for(int i = 1;i ≤ n;++ i){
        S = 1ll * S * K % p;
        ans = (ans + 1ll * S * B[i]) % p;
    }
    cout << ans << "\n";

    return 0;
}
```

6.11 快速乘法逆元（在线）

6.11.1 用法

在线计算 $x = [x_1, x_2, \cdots, x_n]$ 在模 p 意义下的逆元。

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 1e7 + 3;
pair<int, int> F[MAXN], G[MAXN];
int I[MAXN];

using u32 = uint32_t;

u32 read(u32 &seed){
    seed ^= seed << 13;
    seed ^= seed >> 17;
    seed ^= seed << 5;
    return seed;
}

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    u32 seed;
    int n, p;
```

```
    cin >> n >> p >> seed;
    int m = pow(p, 1.0 / 3.0);
    I[1] = 1;

    for(int i = 2;i ≤ p / m;++ i){
        I[i] = 1ll * (p / i) * (p - I[p % i]) % p;
    }
    for(int i = 1;i < m;++ i){
        for(int j = i + 1;j ≤ m;++ j){
            if(!F[i * m * m / j].second){
                F[i * m * m / j] = { i, j };
                G[i * m * m / j] = { i, j };
            }
        }
    }
    F[    0] = G[    0] = { 0, 1 };
    F[m * m] = G[m * m] = { 1, 1 };

    for(int i = 1;i <      m * m;++ i) if(!F[i].second)
        F[i] = F[i - 1];
    for(int i = m * m - 1;i ≥ 1;-- i) if(!G[i].second)
        G[i] = G[i + 1];

    int lastans = 0;
    for(int i = 1;i ≤ n;++ i){
        int a, inv;
        a = (read(seed) ^ lastans) % (p - 1) + 1;
        int w = 1ll * a * m * m / p;
        auto &yy1 = F[w].second;    // *avoid y1 in <cmath>
        if(1ll * a * yy1 % p ≤ p / m){
            inv = 1ll * I[1ll * a * yy1 % p] * yy1 % p;
        } else {
            auto &yy2 = G[w].second;
            inv = 1ll * I[1ll * a * (p - yy2) % p] * (p - yy2) % p;
        }
        lastans = inv;
    }
    cout << lastans << "\n";
    return 0;
}
```

6.12 拉格朗日插值

6.12.1 定理

给定 n 个横坐标不同的点 (x_i, y_i)，可以唯一确定一个 $n-1$ 阶多项式如下：

$$f(x) = \sum_{i=1}^{n} \frac{\prod_{j \neq i}(x - x_j)}{\prod_{j \neq i}(x_i - x_j)} \cdot y_i$$

下面代码先求出了多项式再计算 $f(k)$，也可以直接带入计算。

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 2e3 + 3;
const int MOD  = 998244353;
int X[MAXN], Y[MAXN], F[MAXN], G[MAXN], H[MAXN], A[MAXN];
int power(int a, int b){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % MOD;
        b >>= 1,  a = 1ll * a * a % MOD;
    }
    return r;
}

int main(){
    int n, k;
    cin >> n >> k;
    for(int i = 1;i ≤ n;++ i){
        cin >> X[i] >> Y[i];
    }
    F[0] = 1;
    for(int i = 1;i ≤ n;++ i){     // 计算 prod(x - x_i)
        for(int j = 0;j ≤ n;++ j){
            G[j] = ((j == 0 ? 0 : F[j - 1]) - 1ll * F[j] * X[i] % MOD +
                MOD) % MOD;
        }
        for(int j = 0;j ≤ n;++ j){
            F[j] = G[j];
        }
    }
    for(int i = 1;i ≤ n;++ i){
        for(int j = 0;j ≤ n;++ j){
            G[j] = F[j];
        }
```

```
        for(int j = n;j ≥ 0;-- j){ // 计算 prod(x - x_j) / (x - x_i)
            H[j] = G[j + 1];
            G[j] = (G[j] + 1ll * H[j] * X[i]) % MOD;
        }
        int w = 1;                  // 计算 inv(prod(x_i - x_j))
        for(int j = 1;j ≤ n;++ j) if(j ≠ i)
            w = 1ll * w * (X[i] - X[j] + MOD) % MOD;
        w = 1ll * power(w, MOD - 2) * Y[i] % MOD;
        for(int j = 0;j ≤ n;++ j)
            A[j] = (A[j] + 1ll * w * H[j]) % MOD;
    }
    int t = 1, ans = 0;
    for(int i = 0;i ≤ n - 1;++ i){
        ans = (ans + 1ll * A[i] * t) % MOD;
        t = 1ll * t * k % MOD;
    }
    cout << ans << endl;
}
```

6.13 min-max 容斥

6.13.1 定理

$$\max_{i\in S}\{x_i\} = \sum_{T\subseteq S}(-1)^{|T|-1}\min_{j\in T}\{x_j\}$$
$$\min_{i\in S}\{x_i\} = \sum_{T\subseteq S}(-1)^{|T|-1}\max_{j\in T}\{x_j\}$$

期望意义下上式依然成立。

另外设 $\max^k$ 表示第 k 大的元素，可以推广为如下式子：

$$\max_{i\in S}^{k}\{x_i\} = \sum_{T\subseteq S}(-1)^{|T|-k}\binom{|T-1|}{k-1}\min_{j\in T}\{x_j\}$$

此外在数论上可以得到：

$$\operatorname{lcm}_{i\in S}\{x_i\} = \prod_{T\subseteq S}\left(\gcd_{j\in T}\{x_j\}\right)^{(-1)^{|T|-1}}$$

6.13.2 应用

对于计算“n 个属性都出现的期望时间”问题，设第 i 个属性第一次出现的时间是 t_i，所求即为 $\max(t_i)$，使用 min-max 容斥转为计算 $\min(t_i)$。

比如 n 个独立物品，每次抽中物品 i 的概率是 p_i，问期望抽多少次抽中所有物品。那么就可以计算 $\min_S$ 表示第一次抽中物品集合 S 内物品的时间，可以得到：

$$\max_U = \sum_{S\subseteq U}(-1)^{|S|-1}\min_S = \sum_{S\subseteq U}(-1)^{|S|-1}\cdot\frac{1}{\sum_{x\in S}p_x}$$

6.14 Barrett 取模

6.14.1 用法

调用 init 计算出 S 和 X，得到计算 $x \bmod P = (x \times X)/2^{60} + S$。

```
#include<bits/stdc++.h>
using namespace std;

long long S = 0, X = 0;

void init(int MOD){
    while((1 << (S + 1)) < MOD)
        S ++;
    X = ((__int128)1 << 60 + S) / MOD + !!(((__int128)1 << 60 + S) %
       MOD);

    cerr << S << " " << X << endl;
}

int power(long long x, int y, int MOD){
    long long r = 1;
    while(y){
        if(y & 1){
            r = r * x;
            r = r - MOD * ((__int128)r * X >> 60 + S);
        }
        x = x * x;
        x = x - MOD * ((__int128)x * X >> 60 + S);
        y >>= 1;
    }
    return r;
}

int main(){
    init(998244353);

    cout << power(2, 10, 998244353) << endl;
    cout << power(2, 20, 998244353) << endl;
    cout << power(2, 30, 998244353) << endl;
```

```
    cout << power(2, 40, 998244353) << endl;

    return 0;
}
```

6.15 Pollard's Rho

6.15.1 用法

- 调用 test(n) 判断 n 是否是质数;
- 调用 rho(n) 计算 n 分解质因数后的结果，不保证结果有序。

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

i64 step(i64 a, i64 c, i64 m){
    return ((__int128)a * a + c) % m;
}
i64 multi(i64 a, i64 b, i64 m){
    return (__int128) a * b % m;
}
i64 power(i64 a, i64 b, i64 m){
    i64 r = 1;
    while(b){
        if(b & 1) r = multi(r, a, m);
        b >>= 1,  a = multi(a, a, m);
    }
    return r;
}
mt19937_64 MT;
bool test(i64 n){
    if(n < 3 || n % 2 == 0)
        return n == 2;
    i64 u = n - 1, t = 0;
    while(u % 2 == 0)
        u /= 2,
        t += 1;
    int test_time = 20;
    for(int i = 1; i <= test_time;++ i){
```

```
        i64 a = MT() % (n - 2) + 2;
        i64 v = power(a, u, n);
        if(v == 1){
            continue;
        }
        int s;
        for(s = 0;s < t;++ s){
            if(v == n - 1)
                break;
            v = multi(v, v, n);
        }
        if(s == t)
            return false;
    }
    return true;
}
basic_string<i64> rho(i64 n){
    if(n == 1)
        return {};
    if(test(n)){
        return {n};
    }
    i64 a  = MT() % (n - 1) + 1;
    i64 x1 = MT() % (n - 1);
    i64 x2 = x1;
    for(int i = 1;;i <<= 1){
        i64 tot = 1;
        for(int j = 1;j ≤ i;++ j){
            x2 = step(x2, a, n);
            tot = multi(tot, llabs(x1 - x2), n);
            if(j % 127 == 0){
                i64 d = __gcd(tot, n);
                if(d > 1)
                    return rho(d) + rho(n / d);
            }
        }
        i64 d = __gcd(tot, n);
        if(d > 1)
            return rho(d) + rho(n / d);
        x1 = x2;
    }
}
```

```cpp

// ==== TEST ====

int main(){
    int T;
    cin >> T;
    for(int _ = 1;_ ≤ T;++ _){
        i64 n, p = 0;
        cin >> n;
        auto res = rho(n);
        for(auto &u : res)
            p = max(p, u);
        if(res.size() == 1)
            cout << "Prime" << endl;
        else
            cout << p << endl;
    }
    return 0;
}
```

6.16 polya 定理

6.16.1 Burnside 引理

记所有染色方案的集合为 X，其中单个染色方案为 x。一种对称操作 $g \in X$ 作用于染色方案 $x \in X$ 上可以得到另外一种染色 x'。

将所有对称操作作为集合 G，那么 $Gx = \{gx \mid g \in G\}$ 是与 x 本质相同的染色方案的集合，形式化地称为 x 的轨道。统计本质不同染色方案数，就是统计不同轨道个数。

Burnside 引理说明如下：

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

其中 X^g 表示在 $g \in G$ 的作用下，不动点的集合。不动点被定义为 $x = gx$ 的 x。

6.16.2 Polya 定理

对于通常的染色问题，X 可以看作一个长度为 n 的序列，每个元素是 1 到 m 的整数。可以将 n 看作面数、m 看作颜色数。Polya 定理叙述如下：

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} \sum_{g \in G} m^{c(g)}$$

其中 $c(g)$ 表示对一个序列做轮换操作 g 可以分解成多少个置换环。

然而，增加了限制（比如要求某种颜色必须要多少个），就无法直接应用 Polya 定理，需要利用 Burnside 引理进行具体问题具体分析。

6.16.3 应用

给定 n 个点 n 条边的环，现在有 n 种颜色，给每个顶点染色，询问有多少种本质不同的染色方案。

显然 X 是全体元素在 1 到 n 之间长度为 n 的序列，G 是所有可能的单次旋转方案，共有 n 种，第 i 种方案会把 1 置换到 i。于是：

$$\begin{aligned}
\text{ans} &= \frac{1}{|G|}\sum_{i=1}^{n} m^{c(g_i)} \\
&= \frac{1}{n}\sum_{i=1}^{n} n^{\gcd(i,n)} \\
&= \frac{1}{n}\sum_{d|n}^{n} n^{d}\sum_{i=1}^{n}[\gcd(i,n)=d] \\
&= \frac{1}{n}\sum_{d|n}^{n} n^{d}\varphi(n/d)
\end{aligned}$$

```cpp
#include<bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
int power(int a, int b){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % MOD;
        b >>= 1,  a = 1ll * a * a % MOD;
    }
    return r;
}

vector <tuple<int, int> > P;
void solve(int step, int n, int d, int f, int &ans){
    if(step == P.size()){
        ans = (ans + 1ll * power(n, n / d) * f) % MOD;
    } else {
        auto [w, c] = P[step];
        int dd = 1, ff = 1;
        for(int i = 0;i <= c;++ i){
            solve(step + 1, n, d * dd, f * ff, ans);
            ff = ff * (w - (i == 0));
            dd = dd * w;
        }
    }
}

```

```
int main(){
    int T;
    cin >> T;
    while(T --){
        int n, t;
        cin >> n;
        t = n;
        for(int i = 2;i * i ≤ n;++ i) if(n % i == 0){
            int w = i, c = 0;
            while(t % i == 0){
                t /= i, c ++;
            }
            P.push_back({ w, c });
        }
        if(t ≠ 1){
            P.push_back({ t, 1 });
        }
        int ans = 0;
        solve(0, n, 1, 1, ans);
        ans = 1ll * ans * power(n, MOD - 2) % MOD;
        cout << ans << endl;
        P.clear();
    }
    return 0;
}
```

6.17 min25 筛

设有一个积性函数 $f(n)$，满足 $f(p^k)$ 可以快速求，考虑搞一个在质数位置和 $f(n)$ 相等的 $g(n)$，满足它有完全积性，并且单点和前缀和都可以快速求，然后通过第一部分筛出 g 在质数位置的前缀和，从而相当于得到 f 在质数位置的前缀和，然后利用它，做第二部分，求出 f 的前缀和。

第一部分：$G_k(n) = \sum_{i=1}^{n}[\text{mindiv}(i) > p_k \text{ or isprime}(i)]g(i)$ $(p_0 = 1)$，则有 $G_k(n) = G_{k-1}(n) - g(p_k)(G_{k-1}(n/p_k) - G_{k-1}(p_{k-1}))$，复杂度 $O(n^{3/4}/\log n)$。

第二部分：$F_k(n) = \sum_{i=1}^{n}[\text{mindiv}(i) \geq p_k]f(i)$，$F_k(n) = \sum_{\substack{h\geq k \\ p_h^2\leq n}} \sum_{\substack{c\geq 1 \\ p_h^{c+1}\leq n}} (f(p_h^c)F_{h+1}(n/p_h^c) + f(p_h^{c+1})) + F_{\text{prime}}(n) - F_{\text{prime}}(p_{k-1})$，在 $n \leq 10^{13}$ 可以证明复杂度 $O(n^{3/4}/\log n)$。

常见细节问题：

- 由于 n 通常是 10^{10} 到 10^{11} 的数，导致 n 会爆 int，n^2 会爆 long long，而且往往会用自然数幂和，更容易爆，所以要小心。
- 记 $s = \lfloor\sqrt{n}\rfloor$，由于 F 递归时会去找 F_{h+1}，会访问到 s 以内最大的质数往后的一个质数，而已经证明对于所有 $n \in \mathbb{N}^+$，$[n+1, 2n]$ 中有至少一个质数，所以只需要筛到 $2s$ 即可。
- 注意补回 $f(1)$。

```cpp

// 预处理，$1$ 所在的块也算进去了
namespace init {
    ll init_n, sqrt_n;
    vector<ll> np, p, id1, id2, val;
    ll cnt;

    void main(ll n) {
        init_n = n, sqrt_n = sqrt(n);
        ll M = sqrt_n * 2; // 筛出一个 > floor(sqrt(n)) 的质数，避免后续讨论边界

        np.resize(M + 1), p.resize(M + 1);
        for (ll i = 2; i <= M; ++i) {
            if (!np[i]) p[++p[0]] = i;
            for (ll j = 1; j <= p[0]; ++j) {
                if (i * p[j] > M) break;
                np[i * p[j]] = 1;
                if (i % p[j] == 0) break;
            }
        }
        p[0] = 1;

        id1.resize(sqrt_n + 1), id2.resize(sqrt_n + 1);
        val.resize(1);
        for (ll l = 1, r, v; l <= n; l = r + 1) {
            v = n / l, r = n / v;
            if (v <= sqrt_n) id1[v] = ++cnt;
            else id2[init_n / v] = ++cnt;
            val.emplace_back(v);
        }
    }

    ll id(ll n) {
        if (n <= sqrt_n) return id1[n];
        else return id2[init_n / n];
    }
}
using namespace init;

// 计算 $G_k$，两个参数分别是 $g$ 从 $2$ 开始的前缀和和 $g$
auto calcG = [&] (auto&& sum, auto&& g) -> vector<ll> {
```

```
    vector<ll> G(cnt + 1);
    for (int i = 1; i ≤ cnt; ++i) G[i] = sum(val[i]);
    ll pre = 0;
    for (int i = 1; p[i] * p[i] ≤ n; ++i) {
        for (int j = 1; j ≤ cnt; ++j) {
            if (p[i] * p[i] > val[j]) break;
            ll tmp = id(val[j] / p[i]);
            G[j] = (G[j] - g(p[i]) * (G[tmp] - pre)) % MD;
        }
        pre = (pre + g(p[i])) % MD;
    }
    for (int i = 1; i ≤ cnt; ++i) G[i] = (G[i] % MD + MD) % MD;
    return G;
};

// 计算 $F_k$，直接搜，不用记忆化。`fp` 是 $F_{\text{prime}}$，`pc` 是 $p^c$，其中 `f(p[h] ^ c)` 要替换掉。
function<ll(ll, int)> calcF = [&] (ll m, int k) {
    if (p[k] > m) return 0;
    ll ans = (fp[id(m)] - fp[id(p[k - 1])]) % MD;
    for (int h = k; p[h] * p[h] ≤ m; ++h) {
        ll pc = p[h], c = 1;
        while (pc * p[h] ≤ m) {
            ans = (ans + calcF(m / pc, h + 1) * f(p[h] ^ c)) % MD;
            ++c, pc = pc * p[h], ans = (ans + f(p[h] ^ c)) % MD;
        }
    }
    return ans;
};
```

6.18 杜教筛

6.18.1 用法

对于积性函数 f，找到易求前缀和的积性函数 g, h 使得 $h = f * g$，根据递推式计算 $S(n) = \sum_{i=1}^{n} f(i)$：

$$S(n) = H(n) - \sum_{d=1}^{n} g(d) \times S(\left\lfloor \frac{n}{d} \right\rfloor)$$

6.18.2 例题

- 对于 $f = \varphi$，寻找 $g = 1, h = \mathrm{id}$；
- 对于 $f = \mu$，寻找 $g = 1, h = \varepsilon$。

```cpp
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 1e7 + 3;
const int H = 1e7;

int P[MAXN], p; bool V[MAXN];

long long ph[MAXN], sph[MAXN];
long long mu[MAXN], smu[MAXN];
long long tp[MAXN];

long long solve_ph(long long N){
    for(int d = N / H;d ≥ 1;-- d){
        long long n = N / d;
        long long wh = 1ll * n * (n + 1) / 2;
        tp[d] = wh;
        for(long long l = 2, r;l ≤ n;l = r + 1){
            r = n / (n / l);
            long long wg = r - l + 1;
            long long ws = n / l ≤ H ? sph[n / l] : tp[N / (n / l)];
            tp[d] -= wg * ws;
        }
    }
    return N ≤ H ? sph[N] : tp[1];
}
long long solve_mu(long long N){
    for(int d = N / H;d ≥ 1;-- d){
        long long n = N / d;
        long long wh = 1;
        tp[d] = wh;
        for(long long l = 2, r;l ≤ n;l = r + 1){
            r = n / (n / l);
            long long wg = r - l + 1;
            long long ws = n / l ≤ H ? smu[n / l] : tp[N / (n / l)];
            tp[d] -= wg * ws;
        }
    }
    return N ≤ H ? smu[N] : tp[1];
}

int main(){
```

```
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    ph[1] = 1;
    mu[1] = 1;
    for(int i = 2;i ≤ H;++ i){
        if(!V[i]){
            P[++ p] = i;
            ph[i] = i - 1;
            mu[i] = -1;
        }
        for(int j = 1;j ≤ p && P[j] ≤ H / i;++ j){
            int &p = P[j];
            V[i * p] = true;
            if(i % p == 0){
                ph[i * p] = ph[i] * p;
                mu[i * p] = 0;
                break;
            } else {
                ph[i * p] = ph[i] * (p - 1);
                mu[i * p] = -mu[i];
            }
        }
    }
    for(int i = 1;i ≤ H;++ i){
        sph[i] = sph[i - 1] + ph[i];
        smu[i] = smu[i - 1] + mu[i];
    }
    int T;
    cin >> T;
    while(T --> 0){
        int n;
        cin >> n;

        cout << solve_ph(n) << "␣" << solve_mu(n) << "\n";
    }
    return 0;
}
```

6.19 PN 筛

6.19.1 用法

对于积性函数 $f(x)$，寻找积性函数 $g(x)$ 使得 $g(p) = f(p)$，且 g 易求前缀和 G。

令 $h = f * g^{-1}$，可以证明只有 PN 处 h 的函数值非 0，PN 指每个素因子幂次都不小于 2 的数。同时可以证明 n 以内的 PN 只有 $\mathcal{O}(\sqrt{n})$ 个，且可以暴力枚举质因子幂次得到所有 PN。

可利用下面公式计算 $h(p^c)$：

$$h(p^c) = f(p^c) - \sum_{i=1}^{c} g(p^i) \times h(p^{c-i})$$

6.19.2 例题

定义积性函数 $f(x)$ 满足 $f(p^k) = p^k(p^k - 1)$，计算 $\sum f(i)$。

取 $g(p) = \mathrm{id}(p)\varphi(p) = f(p)$，根据 $g * \mathrm{id} = \mathrm{id}_2$ 利用杜教筛求解。$h(p^c)$ 的值利用递推式进行计算。

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 1e7 + 3;
const int MAXM = 1e5 + 3;
const int H = 1e7;

const int MOD = 1e9 + 7;
const int DIV2 = 500000004;
const int DIV6 = 166666668;

int P[MAXN], p; bool V[MAXN];

int g[MAXN], le[MAXN], ge[MAXN];
int s1(long long n){    // 1^1 + 2^1 + ... + n^1
    n %= MOD;
    return 1ll * n * (n + 1) % MOD * DIV2 % MOD;
}
int s2(long long n){    // 1^2 + 2^2 + ... + n^2
    n %= MOD;
    return 1ll * n * (n + 1) % MOD * (2 * n + 1) % MOD * DIV6 % MOD;
}
int sg(long long n, long long N){
    return n <= H ? le[n] : ge[N / n];
}
int sieve_du(long long N){
    for(int d = N / H;d >= 1;-- d){
```

```
        long long n = N / d;
        int wh = s2(n);
        for(long long l = 2, r;l ≤ n;l = r + 1){
            r = n / (n / l);
            int wg = (s1(r) - s1(l - 1) + MOD) % MOD;
            int ws = sg(n / l, N);
            ge[d] = (ge[d] + 1ll * wg * ws) % MOD;
        }
        ge[d] = (wh - ge[d] + MOD) % MOD;
    }
    return N ≤ H ? le[N] : ge[1];
}
vector <int> hc[MAXM], gc[MAXM];

int ANS;
void sieve_pn(int last, long long x, int h, long long N){
    ANS = (ANS + 1ll * h * sg(N / x, N)) % MOD;
    for(long long i = last + 1;x ≤ N / P[i] / P[i];++ i){
        int c = 2;
        for(long long t = x * P[i] * P[i];t ≤ N;t *= P[i], c ++){
            int hh = 1ll * h * hc[i][c] % MOD;
            sieve_pn(i, t, hh, N);
        }
    }
}
int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    g[1] = 1;
    for(int i = 2;i ≤ H;++ i){
        if(!V[i]){
            P[++ p] = i, g[i] = 1ll * i * (i - 1) % MOD;
        }
        for(int j = 1;j ≤ p && P[j] ≤ H / i;++ j){
            int &p = P[j];
            V[i * p] = true;
            if(i % p == 0){
                g[i * p] = 1ll * g[i] * p % MOD * p % MOD;
                break;
            } else {
                g[i * p] = 1ll * g[i] * p % MOD * (p - 1) % MOD;
```

```
            }
        }
    }
    for(int i = 1;i ≤ H;++ i){
        le[i] = (le[i - 1] + g[i]) % MOD;
    }
    long long N;
    cin >> N;
    for(int i = 1;i ≤ p && 1ll * P[i] * P[i] ≤ N;i ++){
        int &p = P[i];
        hc[i].push_back(1);
        gc[i].push_back(1);
        for(long long c = 1, t = p;t ≤ N;t = t * p, ++ c){
            if(c == 1){
                gc[i].push_back(1ll * p * (p - 1) % MOD);
            } else {
                gc[i].push_back(1ll * gc[i].back() * p % MOD * p % MOD);
            }
            int w = 1ll * (t % MOD) * ((t - 1) % MOD) % MOD;
            int s = 0;
            for(int j = 1;j ≤ c;++ j){
                s = (s + 1ll * gc[i][j] * hc[i][c - j]) % MOD;
            }
            hc[i].push_back((w - s + MOD) % MOD);
        }
    }
    sieve_du(N);
    sieve_pn(0, 1, 1, N);

    cout << ANS << "\n";

    return 0;
}
```

6.20 常用数表

6.20.1 分拆数表

n	10	20	30	40	50	60	70	80	90	100
$p(n)$	42	627	5604	37338	204226	966467	4087968	15796476	56634173	190569292

6.20.2 因数个数表

N	10^1	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\max d(n)$	4	12	32	64	128	240	448	768	1344
$\max \omega(n)$	2	3	4	5	6	7	8	8	9
N	10^{10}	10^{11}	10^{12}	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$\max d(n)$	2304	4032	6720	10752	17280	26880	41472	64512	103680
$\max \omega(n)$	10	10	11	12	12	13	13	14	15

6.20.3 大质数

10^{18} 级别:

- $P = 10^{18} + 3$，好记。
- $P = 2924438830427668481$，可以进行 NTT，$P = 174310137655 \times 2^24 + 1$，原根为 3。

6.21 二次剩余

6.21.1 用法

多次询问，每次询问给定奇素数 p 以及 y，在 $\mathcal{O}(\log p)$ 复杂度计算 x 使得 $x^2 \equiv 0 \pmod p$ 或者无解。

```

#include<bits/stdc++.h>
using namespace std;

int power(int a, int b, int p){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % p;
        b >>= 1,  a = 1ll * a * a % p;
    }
    return r;
}

bool check(int x, int p){
    return power(x, (p - 1) / 2, p) == 1;
}

```

```
struct Node {
    int real, imag;
};
Node mul(const Node a, const Node b, int p, int v){
    int nreal = (1ll * a.real * b.real + 1ll * a.imag * b.imag % p * v)
        % p;
    int nimag = (1ll * a.real * b.imag + 1ll * a.imag * b.real) % p;
    return { (nreal), nimag };
}
Node power(Node a, int b, int p, int v){
    Node r = { 1, 0 };
    while(b){
        if(b & 1) r = mul(r, a, p, v);
        b >>= 1,  a = mul(a, a, p, v);
    }
    return r;
}

mt19937 MT;
void solve(int n, int p, int &x1, int &x2){
    if(n == 0){
        x1 = x2 = 0;
        return;
    }
    if(!check(n, p)){
        x1 = x2 = -1;
        return;
    }
    int a, t;
    do {
        a = MT() % p;
    }while(check(t = (1ll * a * a - n + p) % p, p));
    Node u = { a, 1 };
    x1 = power(u, (p + 1) / 2, p, t).real;
    x2 = (p - x1) % p;

    if(x1 > x2)
        swap(x1, x2);
}

int main(){
    ios :: sync_with_stdio(false);
```

```
    cin.tie(nullptr);

    int T;
    cin >> T;
    while(T --){
        int n, p, x1, x2;
        cin >> n >> p;
        solve(n, p, x1, x2);
        if(x1 == -1){
            cout << "Hola!\n";
        } else {
            if(x1 == x2){
                cout << x1 << "\n";
            } else {
                cout << x1 << "␣" << x2 << "\n";
            }
        }
    }
    return 0;
}
```

6.22 单位根反演

6.22.1 定理

给出单位根反演如下：

$$[d \mid n] = \frac{1}{d}\sum_{i=0}^{d-1}\omega_d^{ni}$$

因为题太难了不会做，所以没有例题，咕着。

7 多项式

7.1 NTT 全家桶

7.1.1 用法

多项式全家桶。

- 包含基础多项式算法：快速傅里叶变换（FFT）及其逆变换（IFFT）、快速数论变换（NTT）及其逆变换（INTT）；
- 包含基于 NTT 的扩展多项式算法：多项式乘法（MUL）、多项式乘法逆元（INV）、多项式微分（DIF）、多项式积分（INT）、多项式对数（LN）、多项式指数（EXP）、多项式开根（SQT）、多项式平移（即计算 $G(x) = F(x + c)$，SHF）。

```cpp
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i <= END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i >= END##i;-- i)
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MOD = 998244353;

int power(int a, int b){
    int r = 1;
    while(b){
        if(b & 1) r = 1ll * r * a % MOD;
        b >>= 1,  a = 1ll * a * a % MOD;
    }
    return r;
}

int inv(int x){
    return power(x, MOD - 2);
}

const int MAX_ = (1 << 19) + 3;
struct cplx{
    double a, b; cplx(double _a = 0, double _b = 0) :a(_a), b(_b){}
    cplx operator +(cplx t){ return cplx(a + t.a, b + t.b); }
    cplx operator -(cplx t){ return cplx(a - t.a, b - t.b); }
    cplx operator *(cplx t){ return cplx(a * t.a - b * t.b, a * t.b + b
        * t.a); }
    cplx operator *(int t) { return cplx(a * t, b * t); }
};

const long double pi = acos(-1);
namespace Poly{
    void FFT(int n, cplx Z[]){
        static int W[MAX_];
        int l = 1; W[0] = 0;
        while (n >>= 1)
            up(0, l - 1, i)
                W[l++] = W[i] << 1 | 1, W[i] <<= 1;
```

```
        up(0, l - 1, i)
            if(W[i] > i) swap(Z[i], Z[W[i]]);
        for (n = l >> 1, l = 1;n;n >>= 1, l <<= 1){
            cplx* S = Z, o(cos(pi / l), sin(pi / l));
            up(0, n - 1, i){
                cplx s(1, 0);
                up(0, l - 1, j){
                    cplx x = S[j] + s * S[j + l];
                    cplx y = S[j] - s * S[j + l];
                    S[j] = x, S[j + l] = y, s = s * o;
                }
                S += l << 1;
            }
        }
    }
    void IFFT(int n, cplx Z[]){
        FFT(n, Z); reverse(Z + 1, Z + n);
        up(0, n - 1, i)
            Z[i].a /= 1.0 * n, Z[i].b /= 1.0 * n;
    }
    void NTT(int n, int Z[]){
        static int W[MAX_];
        int g = 3, l = 1;
        W[0] = 0;
        while (n >>= 1)
            up(0, l - 1, i)
                W[l++] = W[i] << 1 | 1, W[i] <<= 1;
        up(0, l - 1, i)
            if (W[i] > i)swap(Z[i], Z[W[i]]);
        for (n = l >> 1, l = 1;n;n >>= 1, l <<= 1){
            int* S = Z, o = power(g, (MOD - 1) / l / 2);
            up(0, n - 1, i){
                int s = 1;
                up(0, l - 1, j){
                    int x = (S[j] + 1ll * s * S[j + l] % MOD      ) %
                        MOD;
                    int y = (S[j] - 1ll * s * S[j + l] % MOD + MOD) %
                        MOD;
                    S[j] = x, S[j + l] = y;
                    s = 1ll * s * o % MOD;
                }
                S += l << 1;
```

```
            }
        }
    }
    void INTT(int n, int Z[]){
        NTT(n, Z); reverse(Z + 1, Z + n);
        int o = inv(n);
        up(0, n - 1, i)
            Z[i] = 1ll * Z[i] * o % MOD;
    }
    void MUL(int n, int A[], int B[]){          // 乘法
        NTT(n, A), NTT(n, B);
        up(0, n - 1, i)
            A[i] = 1ll * A[i] * B[i] % MOD;
        INTT(n, A);
    }
    void INV(int n, int Z[], int T[]){          // 乘法逆
        static int A[MAX_];
        up(0, n - 1, i)
            T[i] = 0;
        T[0] = power(Z[0], MOD - 2);
        for (int l = 1;l < n;l <<= 1){
            up(    0, 2 * l - 1, i) A[i] = Z[i];
            up(2 * l, 4 * l - 1, i) A[i] = 0;
            NTT(4 * l, A), NTT(4 * l, T);
            up(0, 4 * l - 1, i)
                T[i] = (2ll * T[i] - 1ll * A[i] * T[i] % MOD * T[i] %
                    MOD + MOD) % MOD;
            INTT(4 * l, T);
            up(2 * l, 4 * l - 1, i)
                T[i] = 0;
        }
    }
    void DIF(int n, int Z[], int T[]){          // 微分
        up(0, n - 2, i)
            T[i] = 1ll * Z[i + 1] * (i + 1) % MOD;
        T[n - 1] = 0;
    }
    void INT(int n, int c, int Z[], int T[]){   // 积分
        up(1, n - 1, i)
            T[i] = 1ll * Z[i - 1] * inv(i) % MOD;
        T[0] = c;
    }
```

```
    void LN(int n, int* Z, int* T){            // 求对数
        static int A[MAX_];
        static int B[MAX_];
        up(0, 2 * n - 1, i)
            A[i] = B[i] = 0;
        DIF(n, Z, A);
        INV(n, Z, B);
        MUL(2 * n, A, B);
        INT(n, 0, A, T);
    }
    void EXP(int n, int* Z, int* T){           // 求指数
        static int A[MAX_];
        static int B[MAX_];
        up(1, 2 * n - 1, i) T[i] = 0;
        T[0] = 1;
        for (int l = 1;l < n;l <<= 1){
            LN (2 * l, T, A);
            up(    0, 2 * l - 1, i)
                B[i] = (-A[i] + Z[i] + MOD) % MOD;
            B[0] = (B[0] + 1) % MOD;
            up(2 * l, 4 * l - 1, i)
                T[i] = B[i] = 0;
            MUL(4 * l, T, B);
        }
    }
    void SQT(int n, int* Z, int* T){           // 开根
        static int A[MAX_];
        static int B[MAX_];
        up(1, 2 * n - 1, i) T[i] = 0;
        T[0] = 1;
        int o = inv(2);
        for (int l = 1;l < n;l <<= 1){
            INV(2 * l, T, A);
            up(0, 2 * l - 1, i)
                B[i] = Z[i];
            up(2 * l, 4 * l - 1, i)
                A[i] = B[i] = 0;
            MUL(4 * l, A, B);
            up(0, 2 * l - 1, i)
                T[i] = 1ll * (T[i] + A[i]) * o % MOD;
        }
    }
```

```
    void SHF(int n, int c, int* Z, int* T){   // 平移
        static int A[MAX_];
        static int B[MAX_];
        static int F[MAX_];
        static int G[MAX_];
        int o = 1;
        up(1, n - 1, i)
            F[i] = 1ll * F[i - 1] *     i  % MOD,
            G[i] = 1ll * G[i - 1] * inv(i) % MOD;
        up(0, n - 1, i)
            A[i] = 1ll * Z[n - 1 - i] * F[n - 1 - i] % MOD;
        up(0, n - 1, i){
            B[i] = 1ll * G[i] * o % MOD;
            o = 1ll * o * c % MOD;
        }
        int l = 1; while (l < 2 * n - 1) l <<= 1;
        up(n, l - 1, i)
            A[i] = B[i] = 0;
        MUL(l, A, B);
        up(0, n - 1, i)
            T[n - 1 - i] = 1ll * G[n - 1 - i] * A[i] % MOD;
    }
}
```

7.2 FWT 全家桶

7.2.1 用法

沃尔什全家桶。

包含与卷积、或卷积、异或卷积，定义分别为二进制与、或、异或带入下式：

$$b_k = \sum_{i \otimes j = k} a_i \times b_j$$

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i <= END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i >= END##i;-- i)
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MOD = 998244353;
```

```cpp

namespace Solve1{   // and 卷积
    void FWT(int n, int *A){
        for(int l = 1 << n, u = 2, v = 1;u ≤ l;u <<= 1, v <<= 1)
            for(int j = 0;j < l;j += u)
                for(int k = 0;k < v;++ k)
                    A[j + v + k] = (A[j + v + k] + A[j + k]) % MOD;
    }
    void IFWT(int n, int *A){
        for(int l = 1 << n, u = l, v = l / 2;u > 1;u >>= 1, v >>= 1)
            for(int j = 0;j < l;j += u)
                for(int k = 0;k < v;++ k)
                    A[j + v + k] = (A[j + v + k] - A[j + k] + MOD) %
                        MOD;
    }
}
namespace Solve2{   // or  卷积
    void FWT(int n, int *A){
        for(int l = 1 << n, u = 2, v = 1;u ≤ l;u <<= 1, v <<= 1)
            for(int j = 0;j < l;j += u)
                for(int k = 0;k < v;++ k)
                    A[j + k] = (A[j + k] + A[j + v + k]) % MOD;
    }
    void IFWT(int n, int *A){
        for(int l = 1 << n, u = l, v = l / 2;u > 1;u >>= 1, v >>= 1)
            for(int j = 0;j < l;j += u)
                for(int k = 0;k < v;++ k)
                    A[j + k] = (A[j + k] - A[j + v + k] + MOD) % MOD;
    }
}
namespace Solve3{   // xor 卷积
    void FWT(int n, int *A){
        for(int l = 1 << n, u = 2, v = 1;u ≤ l;u <<= 1, v <<= 1)
            for(int j = 0;j < l;j += u)
                for(int k = 0;k < v;++ k){
                    int a = A[j + k];
                    int b = A[j + v + k];
                    A[j + k    ] = (a + b + MOD) % MOD;
                    A[j + v + k] = (a - b + MOD) % MOD;
                }
    }
    void IFWT(int n, int *A){
```

```
        int div2 = (MOD + 1) / 2;
        for(int l = 1 << n, u = l, v = l / 2;u > 1;u >>= 1, v >>= 1)
            for(int j = 0;j < l;j += u)
                for(int k = 0;k < v;++ k){
                    int a = A[j + k];
                    int b = A[j + v + k];
                    A[j + k    ] = 1ll * (a + b + MOD) * div2 % MOD;
                    A[j + v + k] = 1ll * (a - b + MOD) * div2 % MOD;
                }
    }
}
```

7.3 任意模数 NTT

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i ≤ END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i ≥ END##i;-- i)
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAX_ = (1 << 19) + 3;

template <typename T>
struct cplx0{
    T a, b; cplx0(T _a = 0, T _b = 0) :a(_a), b(_b){}
    cplx0 operator +(cplx0 t){ return cplx0(a + t.a, b + t.b); }
    cplx0 operator -(cplx0 t){ return cplx0(a - t.a, b - t.b); }
    cplx0 operator *(cplx0 t){ return cplx0(a * t.a - b * t.b, a * t.b
        + b * t.a); }
    cplx0 operator *(int t) { return cplx0(a * t, b * t); }
};
using cplx = cplx0<double>;

const long double pi = acos(-1);
namespace Poly{
    void FFT(int n, cplx Z[]){
        static int W[MAX_];
        int l = 1; W[0] = 0;
        while (n >>= 1)
```

```
            up(0, l - 1, i)
                W[l++] = W[i] << 1 | 1, W[i] <<= 1;
        up(0, l - 1, i)
            if(W[i] > i) swap(Z[i], Z[W[i]]);
        for (n = l >> 1, l = 1;n;n >>= 1, l <<= 1){
            cplx* S = Z;
            cplx0<long double> o(cosl(pi / l), sinl(pi / l));
            up(0, n - 1, i){
                cplx0<long double> s(1, 0);
                up(0, l - 1, j){
                    cplx x = S[j] + cplx(s.a, s.b) * S[j + l];
                    cplx y = S[j] - cplx(s.a, s.b) * S[j + l];
                    S[j] = x, S[j + l] = y, s = s * o;
                }
                S += l << 1;
            }
        }
    }
    void IFFT(int n, cplx Z[]){
        FFT(n, Z); reverse(Z + 1, Z + n);
        up(0, n - 1, i)
            Z[i].a /= 1.0 * n, Z[i].b /= 1.0 * n;
    }
}

const int MAXN  = (1 << 19) + 3;
const int BLOCK = 32768;
cplx A1[MAXN], A2[MAXN], B1[MAXN], B2[MAXN];
int n, m, L, mod;

cplx P[MAXN], Q[MAXN];

void FFTFFT(int L, cplx X[], cplx Y[]){
    for(int i = 0;i < L;++ i){
        P[i].a = X[i].a;
        P[i].b = Y[i].a;
    }
    Poly :: FFT(L, P);
    for(int i = 0;i < L;++ i){
        Q[i] = (i == 0 ? P[0] : P[L - i]);
        Q[i].b = -Q[i].b;
    }
```

```
    for(int i = 0;i < L;++ i){
        X[i] = (P[i] + Q[i]);
        Y[i] = (Q[i] - P[i]) * cplx(0, 1);
        X[i].a /= 2.0, X[i].b /= 2.0;
        Y[i].a /= 2.0, Y[i].b /= 2.0;
    }
}

int main(){
    ios :: sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m >> mod;
    for(int i = 0;i ≤ n;++ i){
        int a; cin >> a;
        a %= mod;
        A1[i].a = a / BLOCK;
        A2[i].a = a % BLOCK;
    }
    for(int i = 0;i ≤ m;++ i){
        int a; cin >> a;
        a %= mod;
        B1[i].a = a / BLOCK;
        B2[i].a = a % BLOCK;
    }
    for(L = 1;L ≤ n + m;L <<= 1);
    FFTFFT(L, A1, A2);
    FFTFFT(L, B1, B2);
    for(int i = 0;i < L;++ i){
        P[i] = A1[i] * B1[i] + cplx(0, 1) * A2[i] * B1[i];
        Q[i] = A1[i] * B2[i] + cplx(0, 1) * A2[i] * B2[i];
    }
    Poly :: IFFT(L, P);
    Poly :: IFFT(L, Q);
    for(int i = 0;i < L;++ i){
        long long a1b1 = P[i].a + 0.5;
        long long a2b1 = P[i].b + 0.5;
        long long a1b2 = Q[i].a + 0.5;
        long long a2b2 = Q[i].b + 0.5;

        long long w = ((a1b1 % mod * (BLOCK * BLOCK % mod)) + ((a2b1 +
            a1b2) % mod) * BLOCK + a2b2) % mod;
```

```

        if(i ≤ n + m)
            cout << w << "␣";
    }

    return 0;
}
```

8 字符串

8.1 AC 自动机

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;

const int MOD = 1e9 + 7;

namespace ACAM{
    const int MAXN =1e6 + 3;
    const int MAXM = 26 + 3;
    int C[MAXN][MAXM], o;

    void insert(char *S){
        int p = 0, len = 0;
        for(int i = 0;S[i];++ i){
            int e = S[i] - 'a';
            if(C[p][e]){
                p = C[p][e];
            } else {
                p = C[p][e] = ++ o;
            }
            ++ len;
        }
    }
    int F[MAXN];
    void build(){
        queue <int> Q; Q.push(0);
        while(!Q.empty()){
            int u = Q.front(); Q.pop();
            for(int i = 0;i < 26;++ i){
```

```
                int v = C[u][i];
                if(v == 0)
                    continue;
                int p = F[u];
                while(!C[p][i] && p != 0)
                    p = F[p];
                if(C[p][i] && C[p][i] != v)
                    F[v] = C[p][i];
                Q.push(v);
            }
        }
    }
}
```

8.2 扩展 KMP

8.2.1 定义

$$z_i^{(1)} = |\mathrm{lcp}(b, \mathrm{suffix}(b, i))|$$
$$z_i^{(2)} = |\mathrm{lcp}(b, \mathrm{suffix}(a, i))|$$

```
#include<bits/stdc++.h>
using namespace std;
typedef long long i64;

const int MAXN = 2e7 + 3;
char A[MAXN], B[MAXN * 2];
int n, m, l, r, Z[MAXN * 2];
i64 ans1, ans2;

int main(){
    scanf("%s%s", A + 1, B + 1);
    n = strlen(A + 1);
    m = strlen(B + 1);
    l = 0, r = 0; Z[1] = 0, ans1 = m + 1;
    for(int i = 2;i <= m;++ i){
        if(i <= r) Z[i] = min(r - i + 1, Z[i - l + 1]);
        else       Z[i] = 0;
        while(B[Z[i] + 1] == B[i + Z[i]])
            ++ Z[i];
        if(i + Z[i] - 1 > r)
            r = i + Z[i] - 1, l = i;
        ans1 ^= 1ll * i * (Z[i] + 1);
```

```
    }
    l = 0, r = 0;
    Z[1] = 0, B[m + 1] = '#', strcat(B + 1, A + 1);
    for(int i = 2;i ≤ n + m + 1;++ i){
        if(i ≤ r) Z[i] = min(r - i + 1, Z[i - l + 1]);
        else       Z[i] = 0;
        while(B[Z[i] + 1] == B[i + Z[i]])
            ++ Z[i];
        if(i + Z[i] - 1 > r)
            r = i + Z[i] - 1, l = i;
    }
    for(int i = m + 2;i ≤ n + m + 1;++ i){
        ans2 ^= 1ll * (i - m - 1) * (Z[i] + 1);
    }
    printf("%lld\n%lld\n", ans1, ans2);
    return 0;
}
```

8.3 Manacher

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN= 2.2e7 + 11;
char S[MAXN], T[MAXN]; int n, R[MAXN];
int main(){
    scanf("%s", S + 1);
    n = strlen(S + 1);
    for(int i = 1;i ≤ n;++ i){
        T[2 * i - 1] = S[i];
        T[2 * i    ] =  '#';
    }
    T[0] = '#';
    n = 2 * n;
    int p = 0, x = 0, ans = 0;
    for(int i = 1;i ≤ n;++ i){
        if(i ≤ p) R[i] = min(R[2 * x - i], p - i);
        while(i - R[i] - 1 ≥ 0 && T[i + R[i] + 1] == T[i - R[i] - 1])
```

```
            ++ R[i];
        if(i + R[i] > p){
            p = i + R[i];
            x = i;
        }
        ans = max(ans, R[i]);
    }
    printf("%d\n", ans);
    return 0;
}
```

8.4 回文自动机

```
#include<bits/stdc++.h>
using namespace std;

const int MAXM = 26 + 3;
namespace PAM{
    const int SIZ = 5e5 + 3;
    int n, s, F[SIZ], L[SIZ], D[SIZ];
    int M[SIZ][MAXM];
    char S[SIZ];
    void init(){
        S[0] = '$', n = 1;
        F[s = 0] = -1, L[0] = -1, D[0] = 0;
        F[s = 1] =  0, L[1] =  0, D[1] = 0;
    }
    void extend(int &last, char c){
        S[++ n] = c;
        int e = c - 'a';
        int a = last;
        while(c != S[n - 1 - L[a]])
            a = F[a];
        if(M[a][e]){
            last = M[a][e];
        } else {
            int cur = M[a][e] = ++ s;
            L[cur] = L[a] + 2;
            if(a == 0){
                F[cur] = 1;
            } else {
                int b = F[a];
```

```
                while(c ≠ S[n - 1 - L[b]])
                    b = F[b];
                F[cur] = M[b][e];
            }
            D[cur] = D[F[cur]] + 1;
            last = cur;
        }
    }
}

const int MAXN = 5e5 + 3;
char T[MAXN];
int main(){
    PAM :: init();
    int m = 0, last = 0, lastans = 0;
    for(char c = getchar();isalpha(c);c = getchar()){
        char d = (c - 97 + lastans) % 26 + 97;
        PAM :: extend(last, d);
        cout << (lastans = PAM :: D[last]) << " ";
    }
    return 0;
}
/*
azzzyyzyyx
1 2 1 2 3 2 2 2 3 3
*/
```

8.5 后缀平衡树

8.5.1 本代码尚未完成

8.6 后缀数组（倍增）

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXN = 1e6 + 3;
```

```
int n, m;
int A[MAXN], B[MAXN];
int C[MAXN], R[MAXN], P[MAXN], Q[MAXN];
char S[MAXN];
int main(){
    scanf("%s", S), n = strlen(S), m = 256;
    for(int i = 0;i < n;++ i) R[i] = S[i];
    for (int k = 1;k ≤ n;k <<= 1){
        for(int i = 0;i < n;++ i){
            Q[i] = ((i + k > n - 1) ? 0 : R[i + k]);
            P[i] = R[i];
            m = max(m, R[i]);
        }
#define fun(a, b, c) \
        memset(C, 0, sizeof(int) * (m + 1));            \
        for(int i = 0;i <  n;++ i) C[a] +=        1;    \
        for(int i = 1;i ≤ m;++ i) C[i] += C[i - 1];    \
        for(int i = n - 1;i ≥ 0;-- i) c[-- C[a]] = b;
        fun(Q[  i ],   i , B)
        fun(P[B[i]], B[i], A)
#undef fun
        int p = 1; R[A[0]] = 1;
        for(int i = 1;i ≤ n - 1;++ i){
            bool f1 = P[A[i]] == P[A[i - 1]];
            bool f2 = Q[A[i]] == Q[A[i - 1]];
            R[A[i]] = f1 && f2 ? R[A[i - 1]] : ++ p;
        }
        if (m == n) break;
    }
    for(int i = 0;i < n;++ i)
        printf("%u␣", A[i] + 1);
    return 0;
}
```

8.7 后缀数组（SAIS）

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;
```

```cpp

const int MAXN = 1e6 + 3;
const int MAXM = 256 + 3;

#define LTYPE 0
#define STYPE 1

void induce_sort(int n, int S[], int T[], int m, int LM[], int SA[], int C[]){
    vector <int> BL(n);
    vector <int> BS(n);
    vector <int> BM(n);
    fill(SA, SA + n, -1);
    for(int i = 0;i < n;++ i){      // 预处理桶
        BM[i] = BS[i] = C[i] - 1;
        BL[i] = i == 0 ? 0 : C[i - 1];
    }
    for(int i = m - 1;i >= 0;-- i)  // 放置 LMS 后缀
        SA[BM[S[LM[i]]] --] = LM[i];
    for(int i =     0, p;i  < n;++ i)   // 计算 L 类型后缀的位置
        if(SA[i] > 0 && T[p = SA[i] - 1] == LTYPE)
            SA[BL[S[p]] ++] = p;
    for(int i = n - 1, p;i >= 0;-- i)   // 计算 S 类型后缀的位置
        if(SA[i] > 0 && T[p = SA[i] - 1] == STYPE)
            SA[BS[S[p]] --] = p;
}

// 长度 n, 字符集 [0, n), 要求最后一个元素为 0
// 例如输入 ababa 传入 n = 6, S = [1 2 1 2 1 0]
void sais(int n, int S[], int SA[]){
    vector <int> T(n);
    vector <int> C(n);
    vector <int> I(n, -1);
    T[n - 1] = STYPE;
    for(int i = n - 2;i >= 0;-- i){     // 递推类型
        T[i] = S[i] == S[i + 1] ? T[i + 1] : (S[i] < S[i + 1] ? STYPE : LTYPE);
    }
    for(int i = 0;i < n;++ i){     // 统计个数
        C[S[i]] ++;
    }
    for(int i = 1;i < n;++ i){     // 前缀累加
```

```
        C[i] += C[i - 1];
    }
    vector <int> P;
    for(int i = 0;i < n;++ i){      // 统计 LMS 后缀
        if(T[i] == STYPE && (i == 0 || T[i - 1] == LTYPE)){
            I[i] = P.size(), P.push_back(i);
        }
    }
    int m = P.size(), tot = 0, cnt = 0;
    induce_sort(n, S, T.data(), m, P.data(), SA, C.data());
    vector <int> S0(m), SA0(m);
    for(int i = 0, x, y = -1;i < n;++ i){
        if((x = I[SA[i]]) != -1){
            if(tot == 0 || P[x + 1] - P[x] != P[y + 1] - P[y])
                tot ++;
            else for(int p1 = P[x], p2 = P[y];p2 <= P[y + 1];++ p1, ++
                p2){
                if((S[p1] << 1 | T[p1]) != (S[p2] << 1 | T[p2])){
                    tot ++; break;
                }
            }
            S0[y = x] = tot - 1;
        }
    }
    if(tot == m){
        for(int i = 0;i < m;++ i)
            SA0[S0[i]] = i;
    } else {
        sais(m, S0.data(), SA0.data());
    }
    for(int i = 0;i < m;++ i)
        S0[i] = P[SA0[i]];
    induce_sort(n, S, T.data(), m, S0.data(), SA, C.data());
}

int S[MAXN], SA[MAXN], H[MAXM], G[MAXM];
int main(){
    int n = 0, t = 0, m = 256;
    for(char c = cin.get();isgraph(c);c = cin.get()){
        S[n ++] = c;
        H[c] ++;
    }
```

```
    for(int i = 0;i < m;++ i){
        t += !!H[i], G[i] = t;
    }
    for(int i = 0;i < n;++ i){
        S[i] = G[S[i]];
    }
    sais(n + 1, S, SA);
    for(int i = 1;i ≤ n;++ i){
        cout << SA[i] + 1 << "␣";
    }
    return 0;
}
```

8.8 广义后缀自动机（离线）

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXM= 26 + 3;
namespace SAM{
    const int SIZ = 2e6 + 3;
    int M[SIZ][MAXM];
    int L[SIZ], F[SIZ], S[SIZ];
    int s = 0, h = 25;
    void init(){
        F[0] = -1, s = 0;
    }
    void extend(int &last, char c){
        int e = c - 'a';
        int cur = ++ s;
        L[cur] = L[last] + 1;
        int p = last;
        while(p ≠ -1 && !M[p][e])
            M[p][e] = cur, p = F[p];
        if(p == -1){
            F[cur] = 0;
        } else {
            int q = M[p][e];
```

```
            if(L[p] + 1 == L[q]){
                F[cur] = q;
            } else {
                int clone = ++ s;
                L[clone] = L[p] + 1;
                F[clone] = F[q];
                for(int i = 0;i <= h;++ i)
                    M[clone][i] = M[q][i];
                while(p != -1 && M[p][e] == q)
                    M[p][e] = clone, p = F[p];
                F[cur] = F[q] = clone;
            }
        }
        last = cur;
    }
    void solve(){
        i64 ans = 0;
        for(int i = 1;i <= s;++ i)
            ans += L[i] - L[F[i]];
        cout << ans << endl;
    }
}

namespace Trie{
    const int SIZ = 1e6 + 3;
    int M[SIZ][MAXM], s, h = 25;

    void insert(char *S){
        int p = 0;
        for(int i = 0;S[i];++ i){
            int e = S[i] - 'a';
            if(M[p][e]){
                p = M[p][e];
            } else
                p = M[p][e] = ++ s;
        }
    }
    int O[SIZ];

    void build_sam(){
        queue <int> Q;
        Q.push(0);
```

```
        while(!Q.empty()){
            int u = Q.front(); Q.pop();
            for(int i = 0;i ≤ h;++ i){
                char c = i + 'a';
                if(M[u][i]){
                    int v = M[u][i];
                    O[v] = O[u];
                    SAM :: extend(O[v], c);
                    Q.push(v);
                }
            }
        }
    }
}
const int MAXN = 1e6 + 3;
char S[MAXN];
int main(){
    SAM :: init();
    int n, last = 0;
    cin >> n;
    for(int i = 1;i ≤ n;++ i){
        scanf("%s", S);
        Trie :: insert(S);
    }
    Trie :: build_sam();
    SAM :: solve();
    cout << SAM :: s + 1 << endl;
    return 0;
}
```

8.9 广义后缀自动机（在线）

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXM= 26 + 3;
namespace SAM{
    const int SIZ = 2e6 + 3;
```

```
    int M[SIZ][MAXM];
    int L[SIZ], F[SIZ], S[SIZ];
    int s = 0, h = 25;
    void init(){
        F[0] = -1, s = 0;
    }
    void extend(int &last, char c){
        int e = c - 'a';
        if(M[last][e]){
            int p = last;
            int q = M[last][e];
            if(L[q] == L[last] + 1){
                last = q;
            } else {
                int clone = ++ s;
                L[clone] = L[p] + 1;
                F[clone] = F[q];
                for(int i = 0;i <= h;++ i)
                    M[clone][i] = M[q][i];
                while(p != -1 && M[p][e] == q)
                    M[p][e] = clone, p = F[p];
                F[q] = clone;
                last = clone;
            }
        } else {
            int cur = ++ s;
            L[cur] = L[last] + 1;
            int p = last;
            while(p != -1 && !M[p][e])
                M[p][e] = cur, p = F[p];
            if(p == -1){
                F[cur] = 0;
            } else {
                int q = M[p][e];
                if(L[p] + 1 == L[q]){
                    F[cur] = q;
                } else {
                    int clone = ++ s;
                    L[clone] = L[p] + 1;
                    F[clone] = F[q];
                    for(int i = 0;i <= h;++ i)
                        M[clone][i] = M[q][i];
```

```
                    while(p ≠ -1 && M[p][e] == q)
                        M[p][e] = clone, p = F[p];
                    F[cur] = F[q] = clone;
                }
            }
            last = cur;
        }
    }
    void solve(){
        i64 ans = 0;
        for(int i = 1;i ≤ s;++ i)
            ans += L[i] - L[F[i]];
        cout << ans << endl;
    }
}
const int MAXN = 1e6 + 3;
char S[MAXN];
int main(){
    SAM :: init();
    int n, last = 0;
    cin >> n;
    for(int i = 1;i ≤ n;++ i){
        scanf("%s", S);
        int m = strlen(S);
        last = 0;
        for(int j = 0;j < m;++ j){
            SAM :: extend(last, S[j]);
        }
    }
    SAM :: solve();
    cout << SAM :: s + 1 << endl;
    return 0;
}
```

8.10 后缀自动机

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;
```

```

const int MAXM= 26 + 3;
namespace SAM{
    const int SIZ = 2e6 + 3;
    int M[SIZ][MAXM];
    int L[SIZ], F[SIZ], S[SIZ];
    int last = 0, s = 0, h = 25;
    void init(){
        F[0] = -1, last = s = 0;
    }
    void extend(char c){
        int cur = ++ s, e = c - 'a';
        L[cur] = L[last] + 1;
        S[cur] = 1;
        int p = last;
        while(p != -1 && !M[p][e])
            M[p][e] = cur, p = F[p];
        if(p == -1){
            F[cur] = 0;
        } else {
            int q = M[p][e];
            if(L[p] + 1 == L[q]){
                F[cur] = q;
            } else {
                int clone = ++ s;
                L[clone] = L[p] + 1;
                F[clone] = F[q];
                S[clone] = 0;
                for(int i = 0;i <= h;++ i)
                    M[clone][i] = M[q][i];
                while(p != -1 && M[p][e] == q)
                    M[p][e] = clone, p = F[p];
                F[cur] = F[q] = clone;
            }
        }
        last = cur;
    }
    vector <int> E[SIZ];
    void build(){
        for(int i = 1;i <= s;++ i){
            E[F[i]].push_back(i);
        }
```

```
    }
    i64 ans = 0;
    void dfs(int u){
        for(auto &v : E[u]){
            dfs(v), S[u] += S[v];
        }
        if(S[u] > 1)
            ans = max(ans, 1ll * S[u] * L[u]);
    }
}
const int MAXN = 1e6 + 3;
char S[MAXN];
int main(){
    SAM :: init();
    scanf("%s", S); int n = strlen(S);
    for(int i = 0;i < n;++ i)
        SAM :: extend(S[i]);
    SAM :: build( );
    SAM :: dfs  (0);
    printf("%lld\n", SAM :: ans);
    return 0;
}
```

8.11 字典树

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

const int MAXM= 10 + 3;

namespace Trie{
    const int SIZ = 1e6 + 3;
    int M[SIZ][MAXM], s, h = 10;

    void extend(int &last, char c){
        int e = c - 'a';
        if(M[last][e]){
            last = M[last][e];
```

```
        } else {
            last = M[last][e] = ++ s;
        }
    }
    void insert(char *S){
        int p = 0;
        for(int i = 0;S[i];++ i){
            int e = S[i] - 'a';
            if(M[p][e]){
                p = M[p][e];
            } else
                p = M[p][e] = ++ s;
        }
    }
}
```

9 计算几何

9.1 二维凸包

9.1.1 例题

给定 n 个点，保证每三点不共线。要求找到一个简单多边形满足它不是凸包，使得该多边形面积最大。

```
#include<bits/stdc++.h>
using namespace std;

using i64 = long long;

const int MAXN = 2e5 + 3;
int X[MAXN], Y[MAXN];

struct Frac {
    int a, b;
    Frac (int _a, int _b){
        if(_b < 0){
            a = -_a, b = -_b;
        } else {
            a =  _a, b =  _b;
        }
    }
};

```

```
struct Node {
    int x, y;
}P[MAXN];

bool operator < (const Frac A, const Frac B){
    return 1ll * A.a * B.b - 1ll * A.b * B.a < 0;
}
bool operator < (const Node A, const Node B){
    return A.x == B.x ? A.y > B.y : A.x < B.x;
}

const Frac intersect(Node A, Node B){
    int a = B.y - A.y;
    int b = A.x - B.x;
    assert(b != 0);
    if(b < 0){
        a = -a, b = -b;
    }
    return Frac(a, b);
}
bool F[MAXN];
int main(){
    int TT;
    cin >> TT;
    while(TT -- ){
        int n;
        cin >> n;
        int maxx = -1e9, minx = 1e9;
        for(int i = 1;i <= n;++ i){
            auto &[x, y] = P[i];
            cin >> x >> y;
            F[i] = false;
        }
        sort(P + 1, P + 1 + n);
        vector <int> Q1, Q2, Q;
        // Q1 计算上凸壳, Q2 计算下凸壳
        for(int i = 1;i <= n;++ i){
            auto &[x, y] = P[i];
            if(Q1.size() <= 1){
                Q1.push_back(i);
            } else {
                while(Q1.size() >= 2){
```

```
                    auto &[x1, y1] = P[Q1[Q1.size() - 1]];
                    auto &[x2, y2] = P[Q1[Q1.size() - 2]];
                    long long cmp = 1ll * (y - y1) * (x1 - x2) - 1ll * (x - x1) * (y1 - y2);
                    if(cmp > 0){
                        Q1.pop_back();
                    } else break;
                }
                Q1.push_back(i);
            }
            if(Q2.size() ≤ 1){
                Q2.push_back(i);
            } else {
                while(Q2.size() ≥ 2){
                    auto &[x1, y1] = P[Q2[Q2.size() - 1]];
                    auto &[x2, y2] = P[Q2[Q2.size() - 2]];
                    long long cmp = 1ll * (y - y1) * (x1 - x2) - 1ll * (x - x1) * (y1 - y2);
                    if(cmp < 0){
                        Q2.pop_back();
                    } else break;
                }
                Q2.push_back(i);
            }
        }

        Q = Q1;
        for(int i = Q2.size();i ≠ 0;i --){
            if(i ≠ Q2.size())
                Q.push_back(Q2[i - 1]);
        }
        long long area = 0;
        int x0 = P[Q[0]].x;
        int y0 = P[Q[0]].y;
        for(int i = 1;i + 1 < Q.size();++ i){
            auto &[x1, y1] = P[Q[    i]];
            auto &[x2, y2] = P[Q[i + 1]];
            area += 1ll * (x1 - x0) * (y2 - y0) - 1ll * (x2 - x0) * (y1 - y0);
        }
        area = -area;
        for(auto &i: Q1) F[i] = true;
```

```
        for(auto &i: Q2) F[i] = true;
        bool ok = false;
        for(int i = 1;i ≤ n;++ i) if(!F[i]){
            ok = true;
            maxx = max(maxx, P[i].x);
            minx = min(minx, P[i].x);
        }
        if(!ok){
            cout << -1 << "\n";
            continue;
        }
        vector <int>  L1;
        vector <int>  L2;
        // L1 插入 kx + b 维护下凸壳
        for(int i = 1;i ≤ n;++ i) if(!F[i]){
            auto &[k, b] = P[i];
            if(!L1.empty() && k == P[L1.back()].x)
                continue;
            while(L1.size() ≥ 2){
                auto &P1 = P[L1[L1.size() - 1]];
                auto &P2 = P[L1[L1.size() - 2]];
                Frac i1 = intersect(P1, P[i]);
                Frac i2 = intersect(P2, P[i]);
                if(i1 < i2){
                    L1.pop_back();
                } else break;
            }
            L1.push_back(i);
        }
        // L2 插入 kx + b 维护上凸壳
        for(int i = n;i ≥ 1;-- i) if(!F[i]){
            auto &[k, b] = P[i];
            if(!L2.empty() && k == P[L2.back()].x)
                continue;
            while(L2.size() ≥ 2){
                auto &P1 = P[L2[L2.size() - 1]];
                auto &P2 = P[L2[L2.size() - 2]];

                Frac i1 = intersect(P1, P[i]);
                Frac i2 = intersect(P2, P[i]);
                if(i1 < i2){
                    L2.pop_back();
```

```
                } else break;
            }
            L2.push_back(i);
        }
        vector <Frac> E1;
        E1.push_back(Frac( -2e9, 1 ));
        for(int i = 0;i + 1 < L1.size();++ i){
            auto &P1 = P[L1[i    ]];
            auto &P2 = P[L1[i + 1]];
            E1.push_back(intersect(P1, P2));
        }
        vector <Frac> E2;
        E2.push_back(Frac( -2e9, 1 ));
        for(int i = 0;i + 1 < L2.size();++ i){
            auto &P1 = P[L2[i    ]];
            auto &P2 = P[L2[i + 1]];
            E2.push_back(intersect(P1, P2));
        }
        long long ans = 0;
        for(int i = 0;i + 1 < Q.size();++ i){
            auto &[x1, y1] = P[Q[i    ]];
            auto &[x2, y2] = P[Q[i + 1]];
            long long w = 1ll * x2 * y1 - 1ll * x1 * y2;
            int A = y2 - y1;
            int B = x1 - x2;
            int x = 0, y = 0;
            if(B == 0){
                if(A > 0){
                    x = minx, y = 0;
                } else {
                    x = maxx, y = 0;
                }
            } else
            if(B <  0){
                Frac K = Frac(-A, -B);
                int p = 0;
                for(int k = 20;k >= 0;-- k){
                    int pp = p | 1 << k;
                    if(pp < E1.size() && E1[pp] < K){
                        p = pp;
                    }
                }
```

```
                x = P[L1[p]].x;
                y = P[L1[p]].y;
            } else {
                Frac K = Frac( A,  B);
                int p = 0;
                for(int k = 20;k ≥ 0;-- k){
                    int pp = p | 1 << k;
                    if(pp < E2.size() && E2[pp] < K){
                        p = pp;
                    }
                }
                x = P[L2[p]].x;
                y = P[L2[p]].y;
            }
            ans = max(ans, area - (w + 1ll * A * x + 1ll * B * y));
        }
        // cerr << "ans = " << ans << endl;
        cout << ans << "\n";
    }
    return 0;
}
```

9.2 最小圆覆盖

```
#include "2d.cpp"

point geto(point a, point b, point c) {
    double a1, a2, b1, b2, c1, c2;
    point ans(0, 0);
    a1 = 2 * (b.x - a.x), b1 = 2 * (b.y - a.y),
    c1 = sqr(b.x) - sqr(a.x) + sqr(b.y) - sqr(a.y);
    a2 = 2 * (c.x - a.x), b2 = 2 * (c.y - a.y),
    c2 = sqr(c.x) - sqr(a.x) + sqr(c.y) - sqr(a.y);
    if (equal(a1, 0)) {
        ans.y = c1 / b1;
        ans.x = (c2 - ans.y * b2) / a2;
    } else if (equal(b1, 0)) {
        ans.x = c1 / a1;
        ans.y = (c2 - ans.x * a2) / b2;
    } else {
        ans.x = (c2 * b1 - c1 * b2) / (a2 * b1 - a1 * b2);
        ans.y = (c2 * a1 - c1 * a2) / (b2 * a1 - b1 * a2);
```

```
    }
    return ans;
}

mt19937 MT;
circ minimal(vector <point> V){
    shuffle(V.begin(), V.end(), MT);

    point  o = V[0];
    double r = 0;
    for(int i = 0;i < V.size();++ i) {
        if (sign(dis(o, V[i]) - r) != 1) continue;
        o.x = (V[i].x + V[0].x) / 2;
        o.y = (V[i].y + V[0].y) / 2;
        r = dis(V[i], V[0]) / 2;
        for(int j = 0;j < i;++ j) {
            if (sign(dis(o, V[j]) - r) != 1) continue;
            o.x = (V[i].x + V[j].x) / 2;
            o.y = (V[i].y + V[j].y) / 2;
            r = dis(V[i], V[j]) / 2;
            for(int k = 0;k < j;++ k) {
                if (sign(dis(o, V[k]) - r) != 1) continue;
                o = geto(V[i], V[j], V[k]);
                r = dis(o, V[i]);
            }
        }
    }
    circ res;
    res.o = o;
    res.r = r;
    return res;
}
```

9.3 最左转线

```
#include "2d.cpp"

namespace DSU{
    const int MAXN = 1e5 + 3;
    int F[MAXN];
    int getfa(int u){
        return u == F[u] ? u : F[u] = getfa(F[u]);
```

```
    }
}

namespace Dual{
    const int MAXN = 1e5 + 3;
    const int MAXM = 1e5 + 3;
    int A[MAXM], B[MAXM], W[MAXM], I[MAXM], n, m;
    int outer;
    bool cmp(int a, int b){
        return W[a] < W[b];
    }

    vector <pair<int, int> > E[MAXN];
    const int MAXT = 20 + 3;
    int F[MAXN][MAXT], G[MAXN][MAXT], D[MAXN], h = 20;
    void dfs(int u, int f){
        D[u] = D[f] + 1;
        for(int i = 1;i ≤ h;++ i)
            F[u][i] = F[F[u][i - 1]][i - 1],
            G[u][i] = max(G[u][i - 1], G[F[u][i - 1]][i - 1]);
        for(auto &[v, w] : E[u]) if(v ≠ f){
            G[v][0] = w;
            F[v][0] = u;
            dfs(v, u);
        }
    }
    void build(){
        for(int i = 1;i ≤ n;++ i)
            DSU :: F[i] = i;
        for(int i = 1;i ≤ m;++ i)
            I[i] = i;
        sort(I + 1, I + 1 + m, cmp);
        for(int i = 1;i ≤ m;++ i){
            int a = A[I[i]];
            int b = B[I[i]];
            int w = W[I[i]];
            int fa = DSU :: getfa(a);
            int fb = DSU :: getfa(b);
            if(fa ≠ fb){
                DSU :: F[fa] = fb;
                E[a].push_back({b, w});
                E[b].push_back({a, w});
```

```
            }
        }
        dfs(1, 0);
    }
    int solve(int u, int v){
        if(u == outer || v == outer)
            return -1;
        int ans = 0;
        if(D[u] < D[v]) swap(u, v);
        for(int i = h;i >= 0;-- i)
            if(D[F[u][i]] >= D[v]){
                ans = max(ans, G[u][i]);
                u = F[u][i];
            }
        if(u == v) return ans;
        for(int i = h;i >= 0;-- i)
            if(F[u][i] != F[v][i]){
                ans = max(ans, G[u][i]);
                ans = max(ans, G[v][i]);
                u = F[u][i];
                v = F[v][i];
            }
        ans = max(ans, G[u][0]);
        ans = max(ans, G[v][0]);
        return ans;
    }
}

namespace Planer{
    const int MAXN = 1e5 + 3 + 3;
    const int MAXE = 2e5 + 3;
    const int MAXG = 1e5 + 3;
    const int MAXQ = 2e5 + 3;
    point P[MAXN];

    using edge = tuple<int, int>;

    double gety(int a, int b, double x){
        return P[a].y + (x - P[a].x) / (P[b].x - P[a].x) * (P[b].y - P[
            a].y);
    }

```

```
    double scanx;
    struct Cmp1{
        bool operator ()(const pair<edge, int> l1, const pair<edge, int
            > l2) const{
            const edge &e1 = l1.first;
            const edge &e2 = l2.first;
            double h1 = gety(get<0>(e1), get<1>(e1), scanx);
            double h2 = gety(get<0>(e2), get<1>(e2), scanx);
            return h1 < h2;
        };
    };
    struct Cmp2{
        bool operator ()(const pair<edge, int> l1, const pair<edge, int
            > l2) const{
            if(l1.second == l2.second)
                return false;
            const edge &e1 = l1.first;
            const edge &e2 = l2.first;
            vec v1 = P[get<1>(e1)] - P[get<0>(e1)];
            vec v2 = P[get<1>(e2)] - P[get<0>(e2)];
            if(sign(v1.y) != sign(v2.y)){
                return v1.y > 0;
            } else {
                return sign(mulx(v1, v2)) == 1;
            }
        };
    };

    vector <pair<edge, int> > E[MAXN];

    vector <int> G[MAXG];
    int L[MAXE], R[MAXE], W[MAXE], n, m, q, o;
    double theta;

    int outer;

    void rotate(){
        srand(time(0));
        theta = PI * rand() / RAND_MAX;
    }

    int add(double x, double y){
```

```
        srand(time(0));
        P[++ n] = rotate(vec(x, y), theta);
        return n;
    }
    int link(int u, int v, int w){
        ++ m;
        E[u].push_back({{u, v}, ++ o});
        L[o] = u, R[o] = v, W[o] = w;
        E[v].push_back({{v, u}, ++ o});
        L[o] = v, R[o] = u, W[o] = w;
        return m;
    }

    int I[MAXE];
    int polys;
    pair<edge, int> findleft(int l, int r){
        auto it = lower_bound(E[r].begin(), E[r].end(), make_pair(edge(
            r, l), 0), Cmp2());
        if(it == E[r].begin())
            return E[r].back();
        else
            return *(it - 1);
    }
    void leftmost(){
        for(int i = 1;i ≤ n;++ i){
            sort(E[i].begin(), E[i].end(), Cmp2());
        }
        for(int p = 1;p ≤ n;++ p){
            for(auto &[e1, id1] : E[p]){
                auto &[x, y] = e1;
                if(!I[id1]){
                    int l = x;
                    int r = y;
                    I[id1] = ++ polys;
                    G[polys].push_back(id1);
                    while(r ≠ p){
                        auto [e2, id2] = findleft(l, r);
                        auto [a, b] = e2;
                        I[id2] = polys;
                        G[polys].push_back(id2);
                        l = r;
                        r = b;
```

```
                    }
                }
            }
        }
        for(int i = 1;i ≤ polys;++ i){
            double area = 0;
            for(int j = 0;j < G[i].size();++ j){
                area += mulx(P[L[G[i][j]]], P[R[G[i][j]]]);
            }
            if(area < 0)
                outer = i;
        }
    }

    void dual(){
        Dual :: n = polys;
        Dual :: m = 0;
        for(int i = 1;i ≤ m;++ i){
            int u = I[2 * i - 1], v = I[2 * i], w = W[2 * i];
            if(u == outer || v == outer)
                w = 1e9L + 1;
            ++ Dual :: m;
            Dual :: A[Dual :: m] = u;
            Dual :: B[Dual :: m] = v;
            Dual :: W[Dual :: m] = w;
        }
        Dual :: build();
        Dual :: outer = outer;
    }

    set <pair<edge, int>, Cmp1> S;

    vector <pair<double, int> > T;
    vector <pair<double, int> > Q;

    double X[MAXQ], Y[MAXQ];
    int    Z[MAXQ];

    int ask(double x, double y){
        ++ q;
        point p = rotate(vec(x, y), theta);
        X[q] = p.x;
```

```
        Y[q] = p.y;
        return q;
    }

    void locate(){
        T.clear(), Q.clear(), S.clear();
        for(int i = 1;i ≤ q;++ i){
            Q.push_back(make_pair(X[i], i));
        }
        for(int i = 1;i ≤ polys;++ i){
            for(auto &e : G[i]){
                int u = L[e];
                int v = R[e];
                if(P[u].x > P[v].x){
                    T.push_back(make_pair(P[v].x + 1e-5,  e));
                    T.push_back(make_pair(P[u].x - 1e-5, -e));
                }
            }
        }
        sort(T.begin(), T.end());
        sort(Q.begin(), Q.end());

        int p1 = 0, p2 = 0;
        scanx = -1e9;
        Cmp1 CMP;
        while(p1 < Q.size() || p2 < T.size()){
            // for(auto it1 = S.begin(), it2 = next(S.begin()); it2 ≠
               S.end();++ it1, ++ it2)
            //     assert(CMP(*it1, *it2));
            double x1 = p1 < Q.size() ? Q[p1].first : 1e9;
            double x2 = p2 < T.size() ? T[p2].first : 1e9;
            scanx = min(x1, x2);
            if(equal(scanx, x1)){
                auto &x = X[Q[p1].second];
                auto &y = Y[Q[p1].second];
                auto &z = Z[Q[p1].second];
                P[n + 1] = point(-1e9, y);
                P[n + 2] = point( 1e9, y);
                auto it = S.lower_bound({{n + 1, n + 2}, 0});
                if(it == S.end())
                    z = outer;
                else
```

```
                    z = it -> second;
                ++ p1;
            }
            if(equal(scanx, x2)){
                int g = T[p2].second;
                if(g > 0){
                    assert(!S.count({{L[g], R[g]}, I[g]}));
                    S.insert({{L[g], R[g]}, I[g]});
                } else {
                    g = -g;
                    assert( S.count({{L[g], R[g]}, I[g]}));
                    S.erase ({{L[g], R[g]}, I[g]});
                }
                ++ p2;
            }
        }
    }
}

const int MAXN = 1e5 + 3;
int A[MAXN], B[MAXN];

int main(){
#ifndef ONLINE_JUDGE
    freopen("test.in", "r", stdin);
    freopen("test.out", "w", stdout);
#endif
    int n, m, q;
    Planer :: rotate();

    cin >> n >> m;
    for(int i = 1;i <= n;++ i){
        double x, y;
        cin >> x >> y;
        Planer :: add(x, y);
    }
    for(int i = 1;i <= m;++ i){
        int u, v, w;
        cin >> u >> v >> w;
        Planer :: link(u, v, w);
    }
    Planer :: leftmost();
```

```
    Planer :: dual();
    cin >> q;
    for(int i = 1;i ≤ q;++ i){
        double a1, b1, a2, b2;
        cin >> a1 >> b1;
        A[i] = Planer :: ask(a1, b1);
        cin >> a2 >> b2;
        B[i] = Planer :: ask(a2, b2);
    }
    Planer :: locate();
    for(int i = 1;i ≤ q;++ i)
        A[i] = Planer :: Z[A[i]],
        B[i] = Planer :: Z[B[i]];
    for(int i = 1;i ≤ q;++ i){
        int ans = Dual :: solve(A[i], B[i]);
        cout << ans << endl;
    }
    return 0;
}
```

9.4 二维基础

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

int qread();

const double EPS = 1e-9;
const double PI  = acos(-1);

bool equal(double a, double b){
    return fabs(a - b) < EPS;
}

int sign(double a){
    if(equal(a, 0))
        return 0;
    return a > 0 ? 1 : -1;
```

```
}

double sqr(double x){
    return x * x;
}

struct vec{   // 二维向量
    double x;
    double y;
    vec(){}
    vec(double _x, double _y) : x(_x), y(_y){}
};

vec operator +(const vec &a, const vec &b){
    return vec(a.x + b.x, a.y + b.y);
}
vec operator -(const vec &a, const vec &b){
    return vec(a.x - b.x, a.y - b.y);
}
double mulp(const vec &a, const vec &b){
    return a.x * b.x + a.y * b.y;
}
double mulx(const vec &a, const vec &b){
    return a.x * b.y - a.y * b.x;
}
vec mul(const double &r, const vec &a){
    return vec(r * a.x, r * a.y);
}
bool equal(vec a, vec b){
    return equal(a.x, b.x) && equal(a.y, b.y);
}

using point = vec;

point rotate(point a, double t){
    double c = cos(t);
    double s = sin(t);
    return point(a.x * c - a.y * s, a.y * c + a.x * s);
}

bool cmpx(point a, point b){
    return sign(a.x - b.x) == -1;
```

```
}
bool cmpy(point a, point b){
    return sign(a.y - b.y) == -1;
}

struct line{    // 有向直线
    point o;
    vec p;
    line(point _o, vec _p) : o(_o), p(_p){}
};
struct segm{    // 有向线段
    point a, b;
    segm(point _a, point _b) : a(_a), b(_b){}
};

int side(line l, point p){
    return sign(mulx(l.p, p - l.o));
}
int side(segm s, point p){
    return sign(mulx(s.b - s.a, p - s.a));
}

bool parallel(line a, line b){
    return equal(0, mulx(a.p, b.p));
}

double abs(vec a){
    return sqrt(a.x * a.x + a.y * a.y);
}
double dis(point a, point b){
    return sqrt(sqr(a.x - b.x) + sqr(a.y - b.y));
}
double abs(segm s){
    return dis(s.a, s.b);
}
double dis(line a, point p){
    return abs(mulx(p - a.o, a.p)) / abs(a.p);
}

point intersection(line a, line b){
    return b.o + mul(mulx(b.o - a.o, a.p) / mulx(a.p, b.p), b.p);
}
```

```

bool intersect(double l1, double r1, double l2, double r2){
    if(l1 > r1) swap(l1, r1);
    if(l2 > r2) swap(l2, r2);
    if(equal(r1, l2) || equal(r2, l1))
        return true;
    return !equal(max(r1, r2) - min(l1, l2), r1 - l1 + r2 - l2);
}

bool intersect(segm s1, segm s2){
    bool fx = intersect(s1.a.x, s1.b.x, s2.a.x, s2.b.x);
    if(!fx) return false;
    bool fy = intersect(s1.a.y, s1.b.y, s2.a.y, s2.b.y);
    if(!fy) return false;
    bool g1 = side(s1, s2.a) * side(s1, s2.b) == 1;
    if(g1) return false;
    bool g2 = side(s2, s1.a) * side(s2, s1.b) == 1;
    if(g2) return false;
    return true;
}

struct circ{  // 二维圆形
    point o;
    double r;
};

struct poly{  // 二维多边形
    vector <point> P;
};

double area(point a, point b, point c){
    return abs(mulx(b - a, c - a)) / 2;
}

double area(const poly &P){
    double ans = 0;
    for(int i = 0;i < P.P.size();++ i){
        const point &l = P.P[i];
        const point &r = P.P[i + 1 == P.P.size() ? 0 : i + 1];
        ans += mulx(l, r);
    }
    return ans / 2;
```

```
}
```

10 其他

10.1 笛卡尔树

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

int qread();

const int MAXN = 1e7 + 3;
int n, L[MAXN], R[MAXN], A[MAXN];

void build(){
    stack <int> S;
    A[n + 1] = -1e9;
    for(int i = 1;i ≤ n + 1;++ i){
        int v = 0;
        while(!S.empty() && A[S.top()] > A[i]){
            auto u = S.top();
            R[u] = v;
            v    = u;
            S.pop();
        }
        L[i] = v;
        S.push(i);
    }
}

int main(){
    n = qread();
    for(int i = 1;i ≤ n;++ i)
        A[i] = qread();
    build();
    long long ans1 = 0, ans2 = 0;
    for(int i = 1;i ≤ n;++ i){
        // cout << L[i] << " " << R[i] << endl;
```

```
        ans1 ^= 1ll * i * (L[i] + 1);
        ans2 ^= 1ll * i * (R[i] + 1);
    }
    cout << ans1 << " " << ans2 << endl;
    return 0;
}
```

10.2 CDQ 分治

10.2.1 例题

给定三元组序列 (a_i, b_i, c_i)，求解 $f(i) = \sum_j [a_j \le a_i \wedge b_j \le b_i \wedge c_j \le c_i]$。

```
#include<bits/stdc++.h>
#define up(l, r, i) for(int i = l, END##i = r;i <= END##i;++ i)
#define dn(r, l, i) for(int i = r, END##i = l;i >= END##i;-- i)
using namespace std;
typedef long long i64;
const int INF = 2147483647;
const int MAXN= 1e5 + 3;
const int MAXM= 2e5 + 3;
struct Node{
    int id, a, b, c;
}A[MAXN], B[MAXN];
bool cmp(Node a, Node b){
    if(a.a != b.a) return a.a < b.a;
    if(a.b != b.b) return a.b < b.b;
    if(a.c != b.c) return a.c < b.c;
    return a.id < b.id;
}
int K[MAXN], H[MAXN];
int qread();
int n, m, D[MAXM];
namespace BIT{
    void increase(int x, int w){
        while(x <= m) D[x] += w, x += x & -x;
    }
    void decrease(int x, int w){
        while(x <= m) D[x] -= w, x += x & -x;
    }
    void query(int x, int &r){
        while(x) r += D[x], x -= x & -x;
    }
}
```

```
void cdq(int l, int r){
    if(l ≠ r){
        int t = l + r >> 1; cdq(l, t), cdq(t + 1, r);
        int p = l, q = t + 1, u = l;
        while(p ⩽ t && q ⩽ r){
            if(A[p].b ⩽ A[q].b)
                BIT :: increase(A[p].c, 1),       B[u ++] = A[p ++];
             else
                BIT :: query(A[q].c, K[A[q].id]), B[u ++] = A[q ++];
        }
        while(p ⩽ t) BIT :: increase(A[p].c, 1),       B[u ++] = A[p ++];
        while(q ⩽ r) BIT :: query(A[q].c, K[A[q].id]), B[u ++] = A[q ++];
        up(l, t, i) BIT :: decrease(A[i].c, 1);
        up(l, r, i) A[i] = B[i];
    }
}
int main(){
    n = qread(), m = qread();
    up(1, n, i) A[i].id = i, A[i].a = qread(), A[i].b = qread(), A[i].c = qread();
    sort(A + 1, A + 1 + n, cmp), cdq(1, n);
    sort(A + 1, A + 1 + n, cmp);
    dn(n, 1, i){
        if(A[i].a == A[i + 1].a && A[i].b == A[i + 1].b && A[i].c == A[i + 1].c)
            K[A[i].id] = K[A[i + 1].id];
        H[K[A[i].id]] ++;
    }
    up(0, n - 1, i) printf("%d\n", H[i]);
    return 0;
}
```

10.3 自适应辛普森

10.3.1 例题

计算

$$\int_0^{+\infty} x^{(a/x)-x}$$

```
#include<bits/stdc++.h>
```

```
using namespace std;

using i64 = long long;
const int INF  =  1e9;
const i64 INFL = 1e18;

double simpson(double (*f)(double), double l, double r){
    double mid = (l + r) / 2;
    return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6.0;
}

double adapt_simpson(double (*f)(double), double l, double r, double
    EPS, int step){
    double mid = (l + r) / 2;
    double w0 = simpson(f, l, r);
    double w1 = simpson(f, l, mid);
    double w2 = simpson(f, mid, r);
    if(fabs(w0 - w1 - w2) < EPS && step < 0)
        return w1 + w2;
    else
        return adapt_simpson(f, l, mid, EPS, step - 1) +
               adapt_simpson(f, mid, r, EPS, step - 1);
}

double a, l, r;
double fun(double x){
    return pow(x, a / x - x);
}

int main(){
    cin >> a;
    if(a < 0)
        cout << "orz" << endl;
    else {
        l = 1e-9;
        r = 150;
        cout << fixed << setprecision(5) << adapt_simpson(fun, l, r, 1e
            -9, 15);
    }
}
```

10.4 模拟退火

10.4.1 例题

给定 n 个物品挂在洞下，第 i 个物品坐标 (x_i, y_i) 重量为 w_i。询问平衡点。

```
#include<bits/stdc++.h>
using namespace std;

const double T0 = 2e3, Tk = 1e-14, delta = 0.993, R = 1e-3;
mt19937 MT(114514);
double distance(double x, double y, double a, double b){
    return sqrt(pow(a - x, 2) + pow(b - y, 2));
}
const int MAXN = 1e3 + 3;
double X[MAXN], Y[MAXN], W[MAXN]; int n;
double calculate(double x, double y){
    double gx, gy, a;
    for(int i = 0;i < n; ++i){
        a = atan2(y - Y[i], x - X[i]);
        gx += cos(a) * W[i];
        gy += sin(a) * W[i];
    }
    return pow(gx, 2) + pow(gy, 2);
}
double ex, ey, eans = 1e18;
void SA(){
    double T = T0, x = 0, y = 0, ans = calculate(x, y);
    double ansx, ansy;
    uniform_real_distribution<double> U;
    while(T > Tk){
        double nx, ny, nans;
        nx = x + 2 * (U(MT) - .5) * T;
        ny = y + 2 * (U(MT) - .5) * T;
        if((nans = calculate(nx, ny)) < ans){
            ans = nans;
            ansx = x = nx;
            ansy = y = ny;
        } else if(exp(-distance(nx, ny, x, y) / T / R) > U(MT)){
            x = nx, y = ny;
        }
        T *= delta;
    }
    if(ans < eans) eans = ans, ex = ansx, ey = ansy;
```

```
}
int main(){
    cin >> n;
    for(int i = 0;i < n;++ i)
        cin >> X[i] >> Y[i] >> W[i];
    cout << fixed << setprecision(3);
    if(n == 1){
        cout << X[0] << " " << Y[0] << endl;
    } else {
        SA(), SA(), SA();
        cout << ex << " " << ey << endl;
    }
    return 0;
}
```

10.5 伪随机生成

```
#include<bits/stdc++.h>
using namespace std;

using u32 = uint32_t;
using u64 = uint64_t;

u32 xorshift32(u32 &x){
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return x;
}
u64 xorshift64(u64 &x){
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    return x;
}
```